

|        |     |          |
|--------|-----|----------|
| Name : | ID: | Section: |
|--------|-----|----------|

### Question 1 [C01] [10 Points]

You are preparing iftar plates for family members and each plate must contain **3 dates** and served with **1 glass of water**. However you only have a certain amount of dates and water glasses ready. Write a Java program, that takes as input the total number of people at iftar, total dates available and total water glasses available. Your program must

1. Calculate the total number of dates and glasses of water required to serve all the people
2. Display appropriate message based on if everyone can be served or not:
  - a. Both dates and water glasses are sufficient, display "All plates can be served."
  - b. Only dates are insufficient display "Not enough dates."
  - c. Only water glasses are insufficient display "Not enough water glasses."
  - d. Both are insufficient "Not enough dates and water glasses."
4. Moreover, if all plates can be served, display remaining dates and water glasses and if you can serve additional people or not.

| Sample Input                                                                                                    | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total number of people: 5<br>Enter total dates available: 18<br>Enter total glasses of water available: 6 | Dates Required: 15 , Water Required: 5<br>All plates can be served.<br>Remaining dates: 3<br>Remaining water glasses: 1<br>Can serve additional people.<br><b>Explanation:</b><br>5 people need 5 water glasses and 15 dates. Since given amount of water is 5 and dates are 18, so everyone can be served. Moreover there are 3 additional dates and one additional water. Using 3 dates and 1 glass of water another person can be served. |
| Enter total number of people: 3<br>Enter total dates available: 10<br>Enter total glasses of water available: 3 | Dates Required: 9 , Water Required: 3<br>All plates can be served<br>Remaining dates: 1<br>Remaining water glasses: 0<br>Can NOT serve additional people.<br><b>Explanation:</b><br>At least 3 dates and 1 water needed to serve additional people                                                                                                                                                                                           |
| Enter total number of people: 3<br>Enter total dates available: 8<br>Enter total glasses of water available: 4  | Dates Required: 9 , Water Required: 3<br>Not enough dates.                                                                                                                                                                                                                                                                                                                                                                                   |

|        |     |          |
|--------|-----|----------|
| Name : | ID: | Section: |
|--------|-----|----------|

### Question 1 [C01] [10 Points]

You want to donate clothes and socks to people in need. Each person will receive **1 piece of clothing** and **2 socks**. However, you have a limited number of clothes and socks.

Write a Java program, that takes as input the total number of people you plan on donating to, total clothes available and total socks available. Your program must

1. Calculate the total number of clothes and socks required to serve all the people
2. Display appropriate message based on if everyone can be served or not:
  - a. Both clothes and socks are sufficient, display "All people can receive donation."
  - b. Only clothes are insufficient display "Not enough clothes."
  - c. Only socks are insufficient display "Not enough socks."
  - d. Both are insufficient "Not enough clothes and socks."
4. Moreover, if all people can receive donation, display remaining clothes and socks and if you can donate to additional people or not.

| Sample Input                                                                                           | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total number of people: 6<br>Enter total clothes available: 8<br>Enter total socks available: 16 | Clothes Required: 6 , Socks Required: 12<br>All people can receive donation.<br>Remaining clothes: 2<br>Remaining socks: 4<br>Can donate to additional people.<br><b>Explanation:</b><br>6 people need 6 clothes and 12 socks. Since given amount of clothes is 8 and socks are 16, so everyone can be served.<br>Moreover there are 2 additional clothes and 4 additional socks.<br>Using 2 socks and 1 cloth another person can be served. |
| Enter total number of people: 4<br>Enter total clothes available: 4<br>Enter total socks available: 9  | Clothes Required: 4 , Socks Required: 8<br>All people can receive donation.<br>Remaining clothes: 0<br>Remaining socks: 1<br>Can NOT donate to additional people.<br><b>Explanation:</b><br>At least 1 cloth and 2 socks are needed to serve one more person                                                                                                                                                                                 |
| Enter total number of people: 6<br>Enter total clothes available: 8<br>Enter total socks available: 5  | Clothes Required: 6 , Socks Required: 12<br>Not enough socks.                                                                                                                                                                                                                                                                                                                                                                                |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

In the futuristic city of **Energon Prime**, engineers have discovered a powerful **P-digit** Energy Code(Number) that controls the entire city's power grid. This code must be carefully analyzed to unlock its hidden energy potential. The system responds only to a **special digit N** embedded within the code.

As you examine the Energy Code from Right to Left:

- If the digit N is not found anywhere in the number, then the system shuts down and no energy is generated.
- If the digit N is found, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as  $\text{digit} \times (\text{frequency})$ . **All nonzero contributions** are multiplied to get the **final energy**.

Write a Java program to print whether N was found and the resulting energy value, or indicate that no energy is generated.

| Sample Input 1 | Sample Output 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 122331<br>3    | Digit 0 -> Frequency: 0, Contribution: 0<br>Digit 1 -> Frequency: 2, Contribution: 2<br>Digit 2 -> Frequency: 2, Contribution: 4<br>Digit 3 -> Frequency: 2, Contribution: 6<br>Digit 4 -> Frequency: 0, Contribution: 0<br>Digit 5 -> Frequency: 0, Contribution: 0<br>Digit 6 -> Frequency: 0, Contribution: 0<br>Digit 7 -> Frequency: 0, Contribution: 0<br>Digit 8 -> Frequency: 0, Contribution: 0<br>Digit 9 -> Frequency: 0, Contribution: 0<br>Total Energy: 48<br><b>Explanation:</b><br>Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times<br>$(1 \times 2) \times (2 \times 2) \times (3 \times 2)$<br>= 48 |
| Sample Input 2 | Sample Output 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 456789<br>2    | 2 NOT FOUND<br>NO ENERGY GENERATED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

In the ancient cyber realm of **Aetherion Grid**, engineers have discovered a powerful **P-digit Signal Key (Number)** that controls the planetary communication network. This code must be analyzed to unlock its hidden energy. The system responds only to a special digit **N** embedded within the number.

As you examine the Signal Key from Right to Left:

- If the digit **N** is not found anywhere in the number, the system shuts down and no energy is generated.
- If the digit **N** is found anywhere in the number, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as **digit × (frequency<sup>2</sup>)**. All contributions are summed to get the final energy.

Write a Java program to print whether **N** was found and the resulting energy value, or indicate that no energy is generated.

| Sample Input 1 | Sample Output 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 122331<br>3    | Digit 0 -> Frequency: 0, Contribution: 0<br>Digit 1 -> Frequency: 2, Contribution: 4<br>Digit 2 -> Frequency: 2, Contribution: 8<br>Digit 3 -> Frequency: 2, Contribution: 12<br>Digit 4 -> Frequency: 0, Contribution: 0<br>Digit 5 -> Frequency: 0, Contribution: 0<br>Digit 6 -> Frequency: 0, Contribution: 0<br>Digit 7 -> Frequency: 0, Contribution: 0<br>Digit 8 -> Frequency: 0, Contribution: 0<br>Digit 9 -> Frequency: 0, Contribution: 0<br>Total Energy: 24<br><u>Explanation:</u> 3 is found!<br>Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times<br>$1 \times 2^2 + 2 \times 2^2 + 3 \times 2^2$<br>= 24 |
| Sample Input 2 | Sample Output 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 456789<br>2    | 2 NOT FOUND<br>NO ENERGY GENERATED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |



CSE110: Programming Language I (Lab)  
Spring 2026  
Lab Exam - 02  
Duration: 25 Minutes

**B**

|        |     |          |
|--------|-----|----------|
| Name : | ID: | Section: |
|--------|-----|----------|

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer  $d$ , where  $0 \leq d < n$  and  $n$  is the length of the string. The program should perform a **left rotation** on the given string by  $d$  positions.

In a left rotation:

- The first  **$d$  characters** of the string are moved to the end.
- The remaining characters are shifted to the left.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

| Sample Input:                                                                                                                  | Sample Output:                                |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Enter string: ABCD<br>Enter d: 1<br>Enter string: ABCD<br>Enter d: 2<br>Enter string: HELLO<br>Enter d: 3<br>Enter string: END | BCDA<br>CDAB<br>LOHEL<br>The program stopped. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer  $d$ , where  $0 \leq d < n$  and  $n$  is the length of the string. The program should perform a **right rotation** on the given string by  $d$  positions.

A right rotation means:

- moving the last  **$d$  characters** of the string to the beginning
- shifting the remaining characters to the right.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

| Sample Input:                                                                                                                  | Sample Output:                                |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Enter string: ABCD<br>Enter d: 1<br>Enter string: ABCD<br>Enter d: 2<br>Enter string: HELLO<br>Enter d: 3<br>Enter string: END | DABC<br>CDAB<br>LLOHE<br>The program stopped. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **Friends**. A user is considered a **true Friends binge watcher** only if they watched episodes for at least **3 consecutive days** where the number of episodes watched each day was **strictly increasing**. Among all such increasing streaks, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *Friends* a user watched each day. Write a Java program that prints "**True Binge Watcher**" if the selected streak has a total of **10 or more episodes**; otherwise print "**Casual Watcher**".

|                                                                                             |                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sample Input 1:</b><br><br>Given Array:<br><pre>int [] watch = {2, 3, 5, 6, 1, 2};</pre> | <b>Sample Output 1:</b><br><br>True Binge Watcher<br><b>Explanation:</b><br>Increasing streaks:<br>2, 3, 5, 6 → length = 4, sum = 16<br>1, 2 → length = 2 (ignored)<br>Selected streak = 2, 3, 5, 6<br>sum = 16 >= 10 |
| <b>Sample Input 2:</b><br><br>Given Array:<br><pre>int [] watch = {1, 2, 3, 0, 1};</pre>    | <b>Sample Output 2:</b><br><br>Casual Watcher<br><b>Explanation:</b><br>Increasing streaks:<br>1, 2, 3 → length = 3, sum = 6<br>0, 1 → length = 2 (ignored)<br>Selected streak = 1, 2, 3<br>sum = 6 < 10              |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **The Big Bang Theory**. A user is considered a true *The Big Bang Theory* binge watcher only if they watched episodes for **at least 3 consecutive days** where the number of episodes watched each day was strictly decreasing. Among all such **decreasing streaks**, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *The Big Bang Theory* a user watched each day. Write a Java program that prints **"True Binge Watcher"** if the selected streak has a total of 12 or more episodes; otherwise print **"Casual Watcher"**.

|                                                                                                |                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sample Input 1:</b><br><br>Given Array:<br><pre>int [] watch = {8, 6, 4, 2, 5, 3, 1};</pre> | <b>Sample Output 1:</b><br><br>True Binge Watcher<br><b>Explanation:</b><br>Decreasing streaks are:<br>8, 6, 4, 2 → length = 4, sum = 20<br>5, 3, 1 → length = 3, sum = 9<br>Longest streak = 8, 6, 4, 2<br>Sum = 20 >= 12                                                       |
| <b>Sample Input 2:</b><br><br>Given Array:<br><pre>int [] watch = {5, 4, 2, 6, 2, 1};</pre>    | <b>Sample Output 2:</b><br><br>Casual Watcher<br><b>Explanation:</b><br>Decreasing streaks are:<br>5, 4, 2 → length = 3, sum = 11<br>6, 2, 1 → length = 3, sum = 9<br>Both streaks have same length, so choose the one with the higher total episodes watched:<br>sum = 11 <= 12 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate calorie intake during Iftar. The program should ask the user to enter a six-digit number, and the product of the last three digits of this number will be used as the total calorie limit. The user must then input how many dates they want to eat and how many jilapis they want to eat. Each date contains 80 calories and each jilapi contains 140 calories, and the person must always eat dates first. The program should first calculate the total calories from the dates; if the calories from the dates exceed the calorie limit, then the person cannot eat any jilapis and must drink water instead. If the calories from the dates are within the limit, the program should calculate how many jilapis can be eaten without exceeding the total calorie limit.

| Input                                                                                    | Output                                                                                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Enter a 6-digit number: 123456<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 120<br>Calories from dates: 160<br>Dates exceed the calorie limit.<br>You cannot eat jilapis. Drink water instead. |
| Enter a 6-digit number: 243798<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 504<br>Calories from dates: 160<br>You can eat up to 2 jilapis.<br>Total calories consumed: 440                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a **Java program** that calculates calorie intake during Sehri. The program should take input for the number of rice plates, glasses of milk, and bananas a person wants to eat. Each plate of rice has 200 calories, each glass of milk has 120 calories, and each banana has 100 calories, with a total calorie limit of 800. The program must calculate rice calories first, and if rice alone exceeds 800 calories, the person can only drink water. If not, then calculate the calories from milk and bananas and make sure the total does not go over 800 calories.

| Sample Input                                                                                          | Sample Output                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of plates of rice: 5<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 1000<br>Calorie limit exceeded by rice alone!<br>You should drink only water.                                                                      |
| Enter number of plates of rice: 2<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 400<br>Milk calories = 120<br>Banana calories = 100<br>Total calorie intake = 620<br>You are within the 800 calorie limit. Good Sehri!             |
| Enter number of plates of rice: 3<br>Enter number of glasses of milk: 2<br>Enter number of bananas: 2 | Rice calories = 600<br>Adding milk and bananas exceeds the 800 calorie limit.<br>You should reduce milk or bananas.<br>Maximum remaining calories after rice = 200 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze food sales during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of student stalls in the festival. For each student stall, the number of items sold is equal to the stall number. For example, Stall 1 sells 1 item, Stall 2 sells 2 items, Stall 3 sells 3 items, and so on. For each item, the user must input its price.

For every student stall, the program should display all items that cost more than 200 taka (premium items) and calculate the total sales of that stall. If a stall has 2 or more premium items, the program should print **"Popular Stall!"**. After processing all stalls, the program should display the overall total sales of the festival.

| Input                                                                                                                                                                                                                                                        | Output                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of stalls: 2<br><br>Stall 1:<br>Enter price of item 1: 220<br><br>Stall 2:<br>Enter price of item 1: 150<br>Enter price of item 2: 250                                                                                                          | Stall 1:<br>Premium items: 220<br>Total sales: 220<br><br>Stall 2:<br>Premium items: 250<br>Total sales: 400<br><br>Overall Festival Sales: 620                                                                                     |
| Enter number of stalls: 3<br><br>Stall 1:<br>Enter price of item 1: 150<br><br>Stall 2:<br>Enter price of item 1: 250<br>Enter price of item 2: 300<br><br>Stall 3:<br>Enter price of item 1: 100<br>Enter price of item 2: 220<br>Enter price of item 3: 50 | Stall 1:<br>Premium items:<br>Total sales: 150<br><br>Stall 2:<br>Premium items: 250 300<br>Total sales: 550<br><b>Popular Stall!</b><br><br>Stall 3:<br>Premium items: 220<br>Total sales: 370<br><br>Overall Festival Sales: 1070 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze cultural sessions during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of sessions. For each session, the program should first take input of the number of performances. Then, for each performance, the user must input its duration (in minutes).

For every session, the program should count how many performances have a duration greater than 30 minutes and calculate the total duration of that session. If more than half of the performances in a session have a duration greater than 30 minutes, the program should print **"Long Session!"**. After processing all sessions, the program should display the total duration of all sessions combined.

| Input                                                                                                                                                                                                                                                                                                                                    | Output                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions: 1<br><br>Session 1:<br>Enter number of performances: 4<br>Enter duration of performance 1: 10<br>Enter duration of performance 2: 35<br>Enter duration of performance 3: 45<br>Enter duration of performance 4: 20                                                                                             | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><br>Overall Total Duration: 110                                                                                               |
| Enter number of sessions: 2<br><br>Session 1:<br>Enter number of performances: 3<br>Enter duration of performance 1: 20<br>Enter duration of performance 2: 40<br>Enter duration of performance 3: 50<br><br>Session 2:<br>Enter number of performances: 2<br>Enter duration of performance 1: 25<br>Enter duration of performance 2: 30 | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><b>Long Session!</b><br><br>Session 2:<br>Long performances count: 0<br>Total duration: 55<br><br>Overall Total Duration: 165 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a string from the user and produces its run-length encoded form. In run-length encoding, each group of consecutive identical characters is replaced by the character followed by its count. If a character appears only once in a row, write the character without a number. After producing the encoded string, compare its length with the original. If the encoded version is shorter, print the encoded string. Otherwise, print the original string unchanged.

| Sample Input | Sample Output                                                |
|--------------|--------------------------------------------------------------|
| aabbbccdddee | a2b3c2d4e2                                                   |
| abcd         | abcd<br>(encoded "a1b1c1d1" is longer, so<br>print original) |
| aaabbaaa     | a3b2a3                                                       |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a main string and a pattern string, then prints how many times the pattern appears in the main string (non-overlapping occurrences only). You must implement the search manually, no built-in method may be used to find or detect the substring.

| Sample Input                      | Sample Output |
|-----------------------------------|---------------|
| main: ababab<br>pattern: ab       | Count: 3      |
| main: aaaaaa<br>pattern: aa       | Count: 3      |
| main: hello world<br>pattern: xyz | Count: 0      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate calorie intake during Iftar. The program should ask the user to enter a six-digit number, and the product of the last three digits of this number will be used as the total calorie limit. The user must then input how many dates they want to eat and how many jilapis they want to eat. Each date contains 80 calories and each jilapi contains 140 calories, and the person must always eat dates first. The program should first calculate the total calories from the dates; if the calories from the dates exceed the calorie limit, then the person cannot eat any jilapis and must drink water instead. If the calories from the dates are within the limit, the program should calculate how many jilapis can be eaten without exceeding the total calorie limit.

| Input                                                                                    | Output                                                                                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Enter a 6-digit number: 123456<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 120<br>Calories from dates: 160<br>Dates exceed the calorie limit.<br>You cannot eat jilapis. Drink water instead. |
| Enter a 6-digit number: 243798<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 504<br>Calories from dates: 160<br>You can eat up to 2 jilapis.<br>Total calories consumed: 440                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a **Java program** that calculates calorie intake during Sehri. The program should take input for the number of rice plates, glasses of milk, and bananas a person wants to eat. Each plate of rice has 200 calories, each glass of milk has 120 calories, and each banana has 100 calories, with a total calorie limit of 800. The program must calculate rice calories first, and if rice alone exceeds 800 calories, the person can only drink water. If not, then calculate the calories from milk and bananas and make sure the total does not go over 800 calories.

| Sample Input                                                                                          | Sample Output                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of plates of rice: 5<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 1000<br>Calorie limit exceeded by rice alone!<br>You should drink only water.                                                                      |
| Enter number of plates of rice: 2<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 400<br>Milk calories = 120<br>Banana calories = 100<br>Total calorie intake = 620<br>You are within the 800 calorie limit. Good Sehri!             |
| Enter number of plates of rice: 3<br>Enter number of glasses of milk: 2<br>Enter number of bananas: 2 | Rice calories = 600<br>Adding milk and bananas exceeds the 800 calorie limit.<br>You should reduce milk or bananas.<br>Maximum remaining calories after rice = 200 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate calorie intake during Iftar. The program should ask the user to enter a six-digit number, and the product of the last three digits of this number will be used as the total calorie limit. The user must then input how many dates they want to eat and how many jilapis they want to eat. Each date contains 80 calories and each jilapi contains 140 calories, and the person must always eat dates first. The program should first calculate the total calories from the dates; if the calories from the dates exceed the calorie limit, then the person cannot eat any jilapis and must drink water instead. If the calories from the dates are within the limit, the program should calculate how many jilapis can be eaten without exceeding the total calorie limit.

| Input                                                                                    | Output                                                                                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Enter a 6-digit number: 123456<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 120<br>Calories from dates: 160<br>Dates exceed the calorie limit.<br>You cannot eat jilapis. Drink water instead. |
| Enter a 6-digit number: 243798<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 504<br>Calories from dates: 160<br>You can eat up to 2 jilapis.<br>Total calories consumed: 440                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a **Java program** that calculates calorie intake during Sehri. The program should take input for the number of rice plates, glasses of milk, and bananas a person wants to eat. Each plate of rice has 200 calories, each glass of milk has 120 calories, and each banana has 100 calories, with a total calorie limit of 800. The program must calculate rice calories first, and if rice alone exceeds 800 calories, the person can only drink water. If not, then calculate the calories from milk and bananas and make sure the total does not go over 800 calories.

| Sample Input                                                                                          | Sample Output                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of plates of rice: 5<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 1000<br>Calorie limit exceeded by rice alone!<br>You should drink only water.                                                                      |
| Enter number of plates of rice: 2<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 400<br>Milk calories = 120<br>Banana calories = 100<br>Total calorie intake = 620<br>You are within the 800 calorie limit. Good Sehri!             |
| Enter number of plates of rice: 3<br>Enter number of glasses of milk: 2<br>Enter number of bananas: 2 | Rice calories = 600<br>Adding milk and bananas exceeds the 800 calorie limit.<br>You should reduce milk or bananas.<br>Maximum remaining calories after rice = 200 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze food sales during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of student stalls in the festival. For each student stall, the number of items sold is equal to the stall number. For example, Stall 1 sells 1 item, Stall 2 sells 2 items, Stall 3 sells 3 items, and so on. For each item, the user must input its price.

For every student stall, the program should display all items that cost more than 200 taka (premium items) and calculate the total sales of that stall. If a stall has 2 or more premium items, the program should print **"Popular Stall!"**. After processing all stalls, the program should display the overall total sales of the festival.

| Input                                                                                                                                                                                                                                                        | Output                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of stalls: 2<br><br>Stall 1:<br>Enter price of item 1: 220<br><br>Stall 2:<br>Enter price of item 1: 150<br>Enter price of item 2: 250                                                                                                          | Stall 1:<br>Premium items: 220<br>Total sales: 220<br><br>Stall 2:<br>Premium items: 250<br>Total sales: 400<br><br>Overall Festival Sales: 620                                                                                     |
| Enter number of stalls: 3<br><br>Stall 1:<br>Enter price of item 1: 150<br><br>Stall 2:<br>Enter price of item 1: 250<br>Enter price of item 2: 300<br><br>Stall 3:<br>Enter price of item 1: 100<br>Enter price of item 2: 220<br>Enter price of item 3: 50 | Stall 1:<br>Premium items:<br>Total sales: 150<br><br>Stall 2:<br>Premium items: 250 300<br>Total sales: 550<br><b>Popular Stall!</b><br><br>Stall 3:<br>Premium items: 220<br>Total sales: 370<br><br>Overall Festival Sales: 1070 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze cultural sessions during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of sessions. For each session, the program should first take input of the number of performances. Then, for each performance, the user must input its duration (in minutes).

For every session, the program should count how many performances have a duration greater than 30 minutes and calculate the total duration of that session. If more than half of the performances in a session have a duration greater than 30 minutes, the program should print **"Long Session!"**. After processing all sessions, the program should display the total duration of all sessions combined.

| Input                                                                                                                                                                                                                                                                                                                                    | Output                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions: 1<br><br>Session 1:<br>Enter number of performances: 4<br>Enter duration of performance 1: 10<br>Enter duration of performance 2: 35<br>Enter duration of performance 3: 45<br>Enter duration of performance 4: 20                                                                                             | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><br>Overall Total Duration: 110                                                                                               |
| Enter number of sessions: 2<br><br>Session 1:<br>Enter number of performances: 3<br>Enter duration of performance 1: 20<br>Enter duration of performance 2: 40<br>Enter duration of performance 3: 50<br><br>Session 2:<br>Enter number of performances: 2<br>Enter duration of performance 1: 25<br>Enter duration of performance 2: 30 | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><b>Long Session!</b><br><br>Session 2:<br>Long performances count: 0<br>Total duration: 55<br><br>Overall Total Duration: 165 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

The water park needs a program to manage visitor eligibility for various activities and special offers. Write a Java program that first takes the number of visitor sessions, followed by the number of visitors in each session. For each visitor, the program should further take the Activity Level (1 for Low, 2 for Moderate, 3 for High) and whether the visitor has visited the park previously (true for previous visitors, false for first-time visitors). The park offers the following activities and special offers: the **Lazy Pool** is available for visitors with an activity level of 2 or 3; the **Water Coaster** is available for visitors with an activity level of 3 who have visited the park before; and the **Special Offer** is available to first-time visitors. The program should count and display the total number of eligible visitors for each activity and for the Special Offer.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of visitor sessions: 2<br><br>Enter number of visitors in session 1: 3<br>Enter activity level of visitor 1: 2<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 1<br>Enter previous visit of visitor 2: false<br>Enter activity level of visitor 3: 2<br>Enter previous visit of visitor 3: false<br><br>Enter number of visitors in session 2: 2<br>Enter activity level of visitor 1: 3<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 2<br>Enter previous visit of visitor 2: false | Lazy Pool Riders: 4<br>Water Coaster Riders: 1<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>For session 1, Visitor 1 qualifies for the Lazy Pool but not the Water Coaster due to an activity level of 2. Visitor 2 qualifies for the Special Offer as a first-time visitor. Visitor 3 qualifies for the Lazy Pool and Special Offer.<br><br>For session 2, Visitor 1 qualifies for both the Lazy Pool and the Water Coaster. Visitor 2 qualifies for the Lazy Pool and the Special Offer. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

You are asked to write a Java program for an amusement park that manages visitor eligibility for multiple rides and special offers. The program first takes the number of entry sessions from the management team. For each session, the program takes the number of visitors, followed by each visitor's height (in cm) and age (in years). The park has multiple rides with eligibility rules: the **Roller Coaster** requires a minimum height of 140 cm and a minimum age of 12 years, the **Ferris Wheel** allows visitors aged 5 years or older, and the **Bumper Cars** allow visitors aged 8 years or older with a maximum height of 200 cm for safety. Additionally, visitors under 12 years are eligible for a **Special Discount Offer**. The program should count the number of visitors who are eligible for each ride, as well as how many receive the discount. Finally, the program should display these counts after processing all sessions.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions = 3<br><br>Enter number of visitors in session 1 = 3<br>Enter height of visitor 1: 150<br>Enter age of visitor 1: 14<br>Enter height of visitor 2: 120<br>Enter age of visitor 2: 10<br>Enter height of visitor 3: 160<br>Enter age of visitor 3: 20<br><br>Enter number of visitors in session 2 = 2<br>Enter height of visitor 1: 130<br>Enter age of visitor 1: 6<br>Enter height of visitor 2: 170<br>Enter age of visitor 2: 13<br><br>Enter number of visitors in session 3 = 1<br>Enter height of visitor 1: 180<br>Enter age of visitor 1: 9 | Roller Coaster Riders: 3<br>Ferris Wheel Riders: 6<br>Bumper Cars Riders: 5<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>Visitor 1 in session 1 qualifies for all rides, Visitor 2 qualifies for the Ferris Wheel and Bumper Cars and gets the Special Offer, and Visitor 3 qualifies for all rides.<br>In session 2, Visitor 1 qualifies for the Ferris Wheel and the Special Offer, while Visitor 2 qualifies for all rides.<br>In session 3, the visitor qualifies for the Ferris Wheel and Bumper Cars and receives the Special Offer. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a string from the user and produces its run-length encoded form. In run-length encoding, each group of consecutive identical characters is replaced by the character followed by its count. If a character appears only once in a row, write the character without a number. After producing the encoded string, compare its length with the original. If the encoded version is shorter, print the encoded string. Otherwise, print the original string unchanged.

| Sample Input | Sample Output                                                |
|--------------|--------------------------------------------------------------|
| aabbbccdddee | a2b3c2d4e2                                                   |
| abcd         | abcd<br>(encoded "a1b1c1d1" is longer, so<br>print original) |
| aaabbaaa     | a3b2a3                                                       |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a main string and a pattern string, then prints how many times the pattern appears in the main string (non-overlapping occurrences only). You must implement the search manually, no built-in method may be used to find or detect the substring.

| Sample Input                      | Sample Output |
|-----------------------------------|---------------|
| main: ababab<br>pattern: ab       | Count: 3      |
| main: aaaaaa<br>pattern: aa       | Count: 3      |
| main: hello world<br>pattern: xyz | Count: 0      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that will take two Strings of the same length and an operator character (\* or /) from the user. Your task is to create two numbers from the two input strings by considering only the odd digits and perform mathematical operations based on the third input (operator). Finally, print the newly created two numbers and the output value. The input String can contain other characters than digits. For simplicity, assume that both input strings will always be of the same length. You are not allowed to use any built-in String methods other than .length() and .charAt() methods.

| Sample Input                                                                                                                                                                                                                                                                    | Sample Output        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| ab4s3F67h%21<br>mnP543&1b80Q<br>*                                                                                                                                                                                                                                               | 371<br>531<br>197001 |
| 5dQ!87<br>26f0&3<br>/                                                                                                                                                                                                                                                           | 57<br>3<br>19        |
| Explanation:<br>For sample Input 1,<br><br>ab4s3F67h%21 contains odd digits 3,7,1, that's why the first number is 371<br>mnP543&1b86Q contains odd digits 5,3,1, that's why the second number is 531<br>After performing the addition operation (*), the final output is 197001 |                      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that will take two Strings of the same length and an operator character (+ or -) from the user. Your task is to create two numbers from the two input strings by considering only the even digits and perform mathematical operations based on the third input (operator). Finally, print the newly created two numbers and the output value. The input String can contain other characters than digits. For simplicity, assume that both input strings will always be of the same length. You are not allowed to use any built-in String methods other than `.length()` and `.charAt()` methods.

| Sample Input                                                                                                                                                                                                                                                                   | Sample Output     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| ab4s3F67h%21<br>mnP543&1b80Q<br>+                                                                                                                                                                                                                                              | 462<br>480<br>942 |
| 2dQ!83<br>36f1&5<br>-                                                                                                                                                                                                                                                          | 28<br>6<br>22     |
| Explanation:<br>For sample Input 1,<br><br>ab4s3F67h%21 contains even digits 4,6,2, that's why the first number is 462<br>mnP543&1b86Q contains even digits 4,8,0, that's why the second number is 480<br>After performing the addition operation (+), the final output is 942 |                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

A weekly attendance system stores 7 digits in an array, where **each index** represents a day of the week in order from Saturday to Friday. In this system, classes on **Saturday** (index 0) and **Tuesday** (index 3) are conducted **online**, while all other days are conducted **in-person**. A day is considered balanced if the number of elements before it (to the left) with lower values is equal to the number of elements after it (to the right) with higher values.

Write a Java program to determine and print **all** balanced days **only among the in-person days**, and also print the **total** number of such balanced in-person days.

| Given Array                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Output                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <pre>int[] arr = {40, 10, 20, 100, 15, 30, 5};</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Balanced in-person days are: 2 4 6</p> <p>Total balanced in-person days: 3</p> |
| <p><b>Explanation:</b></p> <p>Monday (index 2, value 20): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> <p>Wednesday (index 4, value 15): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> <p>Friday (index 6, value 5): left lower count = 0, right higher count = 0, ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> |                                                                                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

A FIFA World Cup performance tracker stores the number of goal chances created for 7 matches in an array, where **each index** represents a match in sequence from Match 1 to Match 7. In this system, matches played on **Monday** (index 1) and **Thursday** (index 4) are against **rival teams**, while all other matches are **non-rival**. A match is considered Growth-Stable if the number of non-rival matches before it (to the left) with higher values is equal to the number of non-rival matches after it (to the right) with lower values.

Write a Java program to determine and print **all** Growth-Stable matches **only among the non-rival matches**, and also print the **total** number of such Growth-Stable matches.

| Given Array                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Output                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <pre>int[] scores = {40, 10, 20, 25, 15, 30, 5};</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Growth-Stable matches are: 2 3 5</p> <p>Total Growth-Stable matches: 3</p> |
| <p><b>Explanation:</b></p> <p>Match 3 (index 2, value 20): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> <p>Match 4 (index 3, value 25): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> <p>Match 6 (index 5, value 30): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> |                                                                               |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C03] [10 Points]**

Write a Java program that rearranges the elements of a given array in such a way that all even numbers appear before all odd numbers. *[Do it in-place. You can't create a new array to solve it. The relative order among even or odd numbers does not matter.] [Your code should work for any given integer array]*

| Given Array                                        | Expected Output                            |
|----------------------------------------------------|--------------------------------------------|
| <code>int [] arr = { 3, 6, 4, 7, 1, 9, 2};</code>  | Rearranged Array:<br>6, 4, 2, 7, 1, 9, 3   |
| <code>int[] arr = { 2, 11, 1, 7, 12, 8, 3};</code> | Rearranged Array:<br>2, 12, 8, 7, 11, 1, 3 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C03] [10 Points]**

Write a Java program that rearranges the elements of a given array in such a way that all odd numbers appear before all even numbers. *[Do it in-place. You can't create a new array to solve it. The relative order among even or odd numbers does not matter.] [Your code should work for any given integer array]*

| Given Array                                         | Expected Output                             |
|-----------------------------------------------------|---------------------------------------------|
| <code>int [] arr = {5, 4, 2, 7, 8, 9};</code>       | Rearranged Array:<br>5, 7, 9, 4, 8, 2       |
| <code>int[] arr = { 4, 16, 14, 3, 7, 12, 5};</code> | Rearranged Array:<br>3, 7, 5, 4, 16, 12, 14 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

The water park needs a program to manage visitor eligibility for various activities and special offers. Write a Java program that first takes the number of visitor sessions, followed by the number of visitors in each session. For each visitor, the program should further take the Activity Level (1 for Low, 2 for Moderate, 3 for High) and whether the visitor has visited the park previously (true for previous visitors, false for first-time visitors). The park offers the following activities and special offers: the **Lazy Pool** is available for visitors with an activity level of 2 or 3; the **Water Coaster** is available for visitors with an activity level of 3 who have visited the park before; and the **Special Offer** is available to first-time visitors. The program should count and display the total number of eligible visitors for each activity and for the Special Offer.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of visitor sessions: 2<br><br>Enter number of visitors in session 1: 3<br>Enter activity level of visitor 1: 2<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 1<br>Enter previous visit of visitor 2: false<br>Enter activity level of visitor 3: 2<br>Enter previous visit of visitor 3: false<br><br>Enter number of visitors in session 2: 2<br>Enter activity level of visitor 1: 3<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 2<br>Enter previous visit of visitor 2: false | Lazy Pool Riders: 4<br>Water Coaster Riders: 1<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>For session 1, Visitor 1 qualifies for the Lazy Pool but not the Water Coaster due to an activity level of 2. Visitor 2 qualifies for the Special Offer as a first-time visitor. Visitor 3 qualifies for the Lazy Pool and Special Offer.<br><br>For session 2, Visitor 1 qualifies for both the Lazy Pool and the Water Coaster. Visitor 2 qualifies for the Lazy Pool and the Special Offer. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

You are asked to write a Java program for an amusement park that manages visitor eligibility for multiple rides and special offers. The program first takes the number of entry sessions from the management team. For each session, the program takes the number of visitors, followed by each visitor's height (in cm) and age (in years). The park has multiple rides with eligibility rules: the **Roller Coaster** requires a minimum height of 140 cm and a minimum age of 12 years, the **Ferris Wheel** allows visitors aged 5 years or older, and the **Bumper Cars** allow visitors aged 8 years or older with a maximum height of 200 cm for safety. Additionally, visitors under 12 years are eligible for a **Special Discount Offer**. The program should count the number of visitors who are eligible for each ride, as well as how many receive the discount. Finally, the program should display these counts after processing all sessions.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions = 3<br><br>Enter number of visitors in session 1 = 3<br>Enter height of visitor 1: 150<br>Enter age of visitor 1: 14<br>Enter height of visitor 2: 120<br>Enter age of visitor 2: 10<br>Enter height of visitor 3: 160<br>Enter age of visitor 3: 20<br><br>Enter number of visitors in session 2 = 2<br>Enter height of visitor 1: 130<br>Enter age of visitor 1: 6<br>Enter height of visitor 2: 170<br>Enter age of visitor 2: 13<br><br>Enter number of visitors in session 3 = 1<br>Enter height of visitor 1: 180<br>Enter age of visitor 1: 9 | Roller Coaster Riders: 3<br>Ferris Wheel Riders: 6<br>Bumper Cars Riders: 5<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>Visitor 1 in session 1 qualifies for all rides, Visitor 2 qualifies for the Ferris Wheel and Bumper Cars and gets the Special Offer, and Visitor 3 qualifies for all rides.<br>In session 2, Visitor 1 qualifies for the Ferris Wheel and the Special Offer, while Visitor 2 qualifies for all rides.<br>In session 3, the visitor qualifies for the Ferris Wheel and Bumper Cars and receives the Special Offer. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that will take two Strings of the same length and an operator character (\* or /) from the user. Your task is to create two numbers from the two input strings by considering only the odd digits and perform mathematical operations based on the third input (operator). Finally, print the newly created two numbers and the output value. The input String can contain other characters than digits. For simplicity, assume that both input strings will always be of the same length. You are not allowed to use any built-in String methods other than .length() and .charAt() methods.

| Sample Input                                                                                                                                                                                                                                                                    | Sample Output        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| ab4s3F67h%21<br>mnP543&1b80Q<br>*                                                                                                                                                                                                                                               | 371<br>531<br>197001 |
| 5dQ!87<br>26f0&3<br>/                                                                                                                                                                                                                                                           | 57<br>3<br>19        |
| Explanation:<br>For sample Input 1,<br><br>ab4s3F67h%21 contains odd digits 3,7,1, that's why the first number is 371<br>mnP543&1b86Q contains odd digits 5,3,1, that's why the second number is 531<br>After performing the addition operation (*), the final output is 197001 |                      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that will take two Strings of the same length and an operator character (+ or -) from the user. Your task is to create two numbers from the two input strings by considering only the even digits and perform mathematical operations based on the third input (operator). Finally, print the newly created two numbers and the output value. The input String can contain other characters than digits. For simplicity, assume that both input strings will always be of the same length. You are not allowed to use any built-in String methods other than .length() and .charAt() methods.

| Sample Input                                                                                                                                                                                                                                                                   | Sample Output     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| ab4s3F67h%21<br>mnP543&1b80Q<br>+                                                                                                                                                                                                                                              | 462<br>480<br>942 |
| 2dQ!83<br>36f1&5<br>-                                                                                                                                                                                                                                                          | 28<br>6<br>22     |
| Explanation:<br>For sample Input 1,<br><br>ab4s3F67h%21 contains even digits 4,6,2, that's why the first number is 462<br>mnP543&1b86Q contains even digits 4,8,0, that's why the second number is 480<br>After performing the addition operation (+), the final output is 942 |                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to **calculate the final payable amount** of a customer after applying discounts based on **membership type, festival season, and purchase amount**.

The program should:

- Input total purchase amount.
- Input membership type (1 = Regular, 2 = Silver, 3 = Gold).
- Input festival season (true/false).

Discount rules:

- Regular: 5%
- Silver: 10%
- Gold: 20%
- Add 5% if it is festival season.
- Add 7% if the purchase amount is greater than 10,000.

Display the **total discount percentage** and the **final payable amount**.

| Sample Input                                                                                     | Output                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total purchase amount: 15000<br>Enter membership type : 3<br>Is it festival season?: true  | Total Discount: 32.0%<br>Final Payable Amount: 10200.0<br><hr/> <b>Explanation:</b><br>Membership (20%) + Festival season (5%) + Amount > 10000 (7%)<br>Total discount = 32% |
| Enter total purchase amount: 8000<br>Enter membership type : 2<br>Is it festival season? : false | Total Discount: 10.0%<br>Final Payable Amount: 7200.0<br><hr/> <b>Explanation:</b><br>Membership 10% only                                                                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate the final fee to be paid for a training workshop based on **participant category** and **early registration status and fee**.

The program should:

- Input the workshop fee.
- Input participant category (1 = Student, 2 = Professional, 3 = Corporate).
- Input whether the participant registered early (true/false).

Discount rules:

- Student: 15% discount
- Professional: 8% discount
- Corporate: 5% discount
- If registered early, add an extra 6% discount.
- If the workshop fee is more than 25,000, add an additional 4% discount.

Display the **total discount percentage** and the **final payable amount**.

| Sample Input                                                                                       | Output                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter workshop fee: 30000<br>Enter participant category: 1<br>Was it an early registration? : true | Total Discount: 25.0%<br>Final Amount to be Paid: 22500.0<br><hr/> <b>Explanation:</b><br>Student (15%) + Early registration (6%) + Fee > 25000 (4%)<br>Total discount = 25% |
| Enter workshop fee: 18000<br>Enter participant category: 2<br>Was it an early registration?: false | Total Discount: 8.0%<br>Final Amount to be Paid: 16560.0<br><hr/> <b>Explanation:</b><br>Professional (8%) only                                                              |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

The water park needs a program to manage visitor eligibility for various activities and special offers. Write a Java program that first takes the number of visitor sessions, followed by the number of visitors in each session. For each visitor, the program should further take the Activity Level (1 for Low, 2 for Moderate, 3 for High) and whether the visitor has visited the park previously (true for previous visitors, false for first-time visitors). The park offers the following activities and special offers: the **Lazy Pool** is available for visitors with an activity level of 2 or 3; the **Water Coaster** is available for visitors with an activity level of 3 who have visited the park before; and the **Special Offer** is available to first-time visitors. The program should count and display the total number of eligible visitors for each activity and for the Special Offer.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of visitor sessions: 2<br><br>Enter number of visitors in session 1: 3<br>Enter activity level of visitor 1: 2<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 1<br>Enter previous visit of visitor 2: false<br>Enter activity level of visitor 3: 2<br>Enter previous visit of visitor 3: false<br><br>Enter number of visitors in session 2: 2<br>Enter activity level of visitor 1: 3<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 2<br>Enter previous visit of visitor 2: false | Lazy Pool Riders: 4<br>Water Coaster Riders: 1<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>For session 1, Visitor 1 qualifies for the Lazy Pool but not the Water Coaster due to an activity level of 2. Visitor 2 qualifies for the Special Offer as a first-time visitor. Visitor 3 qualifies for the Lazy Pool and Special Offer.<br><br>For session 2, Visitor 1 qualifies for both the Lazy Pool and the Water Coaster. Visitor 2 qualifies for the Lazy Pool and the Special Offer. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

You are asked to write a Java program for an amusement park that manages visitor eligibility for multiple rides and special offers. The program first takes the number of entry sessions from the management team. For each session, the program takes the number of visitors, followed by each visitor's height (in cm) and age (in years). The park has multiple rides with eligibility rules: the **Roller Coaster** requires a minimum height of 140 cm and a minimum age of 12 years, the **Ferris Wheel** allows visitors aged 5 years or older, and the **Bumper Cars** allow visitors aged 8 years or older with a maximum height of 200 cm for safety. Additionally, visitors under 12 years are eligible for a **Special Discount Offer**. The program should count the number of visitors who are eligible for each ride, as well as how many receive the discount. Finally, the program should display these counts after processing all sessions.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions = 3<br><br>Enter number of visitors in session 1 = 3<br>Enter height of visitor 1: 150<br>Enter age of visitor 1: 14<br>Enter height of visitor 2: 120<br>Enter age of visitor 2: 10<br>Enter height of visitor 3: 160<br>Enter age of visitor 3: 20<br><br>Enter number of visitors in session 2 = 2<br>Enter height of visitor 1: 130<br>Enter age of visitor 1: 6<br>Enter height of visitor 2: 170<br>Enter age of visitor 2: 13<br><br>Enter number of visitors in session 3 = 1<br>Enter height of visitor 1: 180<br>Enter age of visitor 1: 9 | Roller Coaster Riders: 3<br>Ferris Wheel Riders: 6<br>Bumper Cars Riders: 5<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>Visitor 1 in session 1 qualifies for all rides, Visitor 2 qualifies for the Ferris Wheel and Bumper Cars and gets the Special Offer, and Visitor 3 qualifies for all rides.<br>In session 2, Visitor 1 qualifies for the Ferris Wheel and the Special Offer, while Visitor 2 qualifies for all rides.<br>In session 3, the visitor qualifies for the Ferris Wheel and Bumper Cars and receives the Special Offer. |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class WaterPark {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lazyPool = 0, waterCoaster = 0, specialOffer = 0;

        System.out.print("Enter number of visitor sessions: ");
        int sessions = sc.nextInt();

        for (int i = 1; i <= sessions; i++) {
            System.out.print("Enter number of visitors in session " + i + ": ");
            int visitors = sc.nextInt();

            for (int j = 1; j <= visitors; j++) {
                System.out.print("Enter activity level of visitor " + j + ": ");
                int activityLevel = sc.nextInt();
                System.out.print("Enter previous visit of visitor " + j + ": ");
                boolean previousVisit = sc.nextBoolean();

                if (activityLevel == 2 || activityLevel == 3) {
                    lazyPool++;
                }

                if (activityLevel == 3 && previousVisit==true) {
                    waterCoaster++;
                }

                if (previousVisit==false) {
                    specialOffer++;
                }
            }
        }

        System.out.println("Lazy Pool Riders: " + lazyPool);
        System.out.println("Water Coaster Riders: " + waterCoaster);
        System.out.println("Visitors with Special Offer: " + specialOffer);
    }
}
```

**RUBRIC**

| SL | Points to Meet                                                                               | Marks [10] |
|----|----------------------------------------------------------------------------------------------|------------|
| 1  | The program takes input for sessions, number of visitors, activity level, and previous visit | 1          |
| 2  | Correctly calculates eligibility for all activities (Lazy Pool, Water Coaster)               | 3          |
| 3  | Correctly calculates eligibility for Special Offer                                           | 1          |
| 4  | Displays total counts for all activities and Special Offers correctly                        | 2          |
| 5  | Proper use of nested loops and conditionals                                                  | 3          |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;

public class AmusementPark {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int rollerCoaster = 0, ferrisWheel = 0, bumperCars = 0, specialOffer = 0;

        System.out.print("Enter number of sessions = ");
        int sessions = sc.nextInt();

        for (int i = 1; i <= sessions; i++) {
            System.out.print("Enter number of visitors in session " + i + "= ");
            int visitors = sc.nextInt();

            for (int j = 1; j <= visitors; j++) {
                System.out.print("Enter height of visitor " + j + ": ");
                int height = sc.nextInt();
                System.out.print("Enter age of visitor " + j + ": ");
                int age = sc.nextInt();

                if (height >= 140 && age >= 12) {
                    rollerCoaster++;
                }

                if (age >= 5) {
                    ferrisWheel++;
                }

                if (age >= 8 && height <= 200) {
                    bumperCars++;
                }

                if (age < 12) {
                    specialOffer++;
                }
            }
        }

        System.out.println("Roller Coaster Riders: " + rollerCoaster);
        System.out.println("Ferris Wheel Riders: " + ferrisWheel);
        System.out.println("Bumper Cars Riders: " + bumperCars);
        System.out.println("Visitors with Special Offer: " + specialOffer);
    }
}
```

| RUBRIC |                                                                                            |               |
|--------|--------------------------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                                             | Marks<br>[10] |
| 1      | The program takes input for sessions, number of visitors, height, and age                  | 1             |
| 2      | Correctly calculates eligibility for all rides (Roller Coaster, Ferris Wheel, Bumper Cars) | 3             |
| 3      | Correctly calculates Special Offer eligibility                                             | 1             |
| 4      | Displays total counts for all rides and Special Offers correctly                           | 2             |
| 5      | Proper use of nested loops                                                                 | 3             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that will take two Strings of the same length and an operator character (\* or /) from the user. Your task is to create two numbers from the two input strings by considering only the odd digits and perform mathematical operations based on the third input (operator). Finally, print the newly created two numbers and the output value. The input String can contain other characters than digits. For simplicity, assume that both input strings will always be of the same length. You are not allowed to use any built-in String methods other than .length() and .charAt() methods.

| Sample Input                                                                                                                                                                                                                                                                    | Sample Output        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| ab4s3F67h%21<br>mnP543&1b80Q<br>*                                                                                                                                                                                                                                               | 371<br>531<br>197001 |
| 5dQ!87<br>26f0&3<br>/                                                                                                                                                                                                                                                           | 57<br>3<br>19        |
| Explanation:<br>For sample Input 1,<br><br>ab4s3F67h%21 contains odd digits 3,7,1, that's why the first number is 371<br>mnP543&1b86Q contains odd digits 5,3,1, that's why the second number is 531<br>After performing the addition operation (*), the final output is 197001 |                      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that will take two Strings of the same length and an operator character (+ or -) from the user. Your task is to create two numbers from the two input strings by considering only the even digits and perform mathematical operations based on the third input (operator). Finally, print the newly created two numbers and the output value. The input String can contain other characters than digits. For simplicity, assume that both input strings will always be of the same length. You are not allowed to use any built-in String methods other than .length() and .charAt() methods.

| Sample Input                                                                                                                                                                                                                                                                   | Sample Output     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| ab4s3F67h%21<br>mnP543&1b80Q<br>+                                                                                                                                                                                                                                              | 462<br>480<br>942 |
| 2dQ!83<br>36f1&5<br>-                                                                                                                                                                                                                                                          | 28<br>6<br>22     |
| Explanation:<br>For sample Input 1,<br><br>ab4s3F67h%21 contains even digits 4,6,2, that's why the first number is 462<br>mnP543&1b86Q contains even digits 4,8,0, that's why the second number is 480<br>After performing the addition operation (+), the final output is 942 |                   |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;
public class A{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        String s1 = sc.next();
        String s2 = sc.next();
        char op = sc.next().charAt(0);
        int num1 = 0, num2 = 0;
        for(int i = 0; i<s1.length(); i++){
            char ch1 = s1.charAt(i);
            char ch2 = s2.charAt(i);
            if(ch1>='0' && ch1<='9'){
                int digit = (int)ch1 - 48;
                if(digit%2 != 0){
                    num1 = num1*10 + digit;
                }
            }
            if(ch2>='0' && ch2<='9'){
                int digit = (int)ch2 - 48;
                if(digit%2 != 0){
                    num2 = num2*10 + digit;
                }
            }
        }
        System.out.println(num1);
        System.out.println(num2);
        if(op == '*'){
            System.out.println(num1*num2);
        }
        else if(op == '/'){
            System.out.println(num1/num2);
        }
    }
}
```

**RUBRIC**

| SL | Points to Meet                                            | Marks [10] |
|----|-----------------------------------------------------------|------------|
| 1  | Importing Scanner class, main method, and Java syntax     | 0.5        |
| 2  | Correctly takes two strings and an operator from the user | 1          |

|   |                                                                               |     |
|---|-------------------------------------------------------------------------------|-----|
| 3 | Properly handles the iteration and extracts the characters of both Strings    | 1.5 |
| 4 | Checking the digits using characters ('0'-'9') or ASCII values [both Strings] | 1   |
| 5 | Converting the character to a digit for both Strings                          | 1   |
| 6 | Searching the odd digits [both Strings]                                       | 0.5 |
| 7 | Initialize num1, num2, and correctly prepare the numbers                      | 3   |
| 8 | Checking the operations (*,/) and performing the correct operation            | 1   |
| 9 | Printing all three outputs correctly                                          | 0.5 |

### SET - [B]

#### SOLUTION

```
import java.util.Scanner;
public class B{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        String s1 = sc.next();
        String s2 = sc.next();
        char op = sc.next().charAt(0);
        int num1 = 0, num2 = 0;
        for(int i = 0; i<s1.length(); i++){
            char ch1 = s1.charAt(i);
            char ch2 = s2.charAt(i);
            if(ch1>='0' && ch1<='9'){
                int digit = (int)ch1 - 48;
                if(digit%2 == 0){
                    num1 = num1*10 + digit;
                }
            }
            if(ch2>='0' && ch2<='9'){
                int digit = (int)ch2 - 48;
                if(digit%2 == 0){
                    num2 = num2*10 + digit;
                }
            }
        }
        System.out.println(num1);
        System.out.println(num2);
        if(op == '+'){
            System.out.println(num1+num2);
        }
    }
}
```

```

    }
    else if(op == '-'){
        System.out.println(num1-num2);
    }
}
}
}

```

#### RUBRIC

| SL | Points to Meet                                                                | Marks [10] |
|----|-------------------------------------------------------------------------------|------------|
| 1  | Importing Scanner class, main method, and Java syntax                         | 0.5        |
| 2  | Correctly takes two strings and an operator from the user                     | 1          |
| 3  | Properly handles the iteration and extracts the characters of both Strings    | 1.5        |
| 4  | Checking the digits using characters ('0'-'9') or ASCII values [both Strings] | 1          |
| 5  | Converting the character to a digit for both Strings                          | 1          |
| 6  | Searching the even digits [both Strings]                                      | 0.5        |
| 7  | Initialize num1, num2, and correctly prepare the numbers                      | 3          |
| 8  | Checking the operations (+,-) and performing the correct operation            | 1          |
| 9  | Printing all three outputs correctly                                          | 0.5        |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C03] [10 Points]**

Write a Java program that rearranges the elements of a given array in such a way that all even numbers appear before all odd numbers. *[Do it in-place. You can't create a new array to solve it. The relative order among even or odd numbers does not matter.] [Your code should work for any given integer array]*

| Given Array                                        | Expected Output                            |
|----------------------------------------------------|--------------------------------------------|
| <code>int [] arr = { 3, 6, 4, 7, 1, 9, 2};</code>  | Rearranged Array:<br>6, 4, 2, 7, 1, 9, 3   |
| <code>int[] arr = { 2, 11, 1, 7, 12, 8, 3};</code> | Rearranged Array:<br>2, 12, 8, 7, 11, 1, 3 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C03] [10 Points]**

Write a Java program that rearranges the elements of a given array in such a way that all odd numbers appear before all even numbers. *[Do it in-place. You can't create a new array to solve it. The relative order among even or odd numbers does not matter.] [Your code should work for any given integer array]*

| Given Array                                         | Expected Output                             |
|-----------------------------------------------------|---------------------------------------------|
| <code>int [] arr = {5, 4, 2, 7, 8, 9};</code>       | Rearranged Array:<br>5, 7, 9, 4, 8, 2       |
| <code>int[] arr = { 4, 16, 14, 3, 7, 12, 5};</code> | Rearranged Array:<br>3, 7, 5, 4, 16, 12, 14 |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
public class EvenOddInPlace {
    public static void main(String[] args) {
        int[] arr = { 3, 6, 4, 7, 1, 9, 2};
        int pos = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0) {
                int temp = arr[i];
                arr[i] = arr[pos];
                arr[pos] = temp;
                pos++;
            }
        }
        System.out.println("Rearranged Array:");
        for (int i = 0; i < arr.length; i++) {
            if(i==arr.length-1){
                System.out.println(arr[i]);
            }
            else{
                System.out.print(arr[i] + ", ");
            }
        }
    }
}
```

Alternate Solution

```
public class EvenOddNested {
    public static void main(String[] args) {
        int[] arr = {3, 6, 4, 7, 1, 9, 2};
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 != 0) {
                for (int j = i + 1; j < arr.length; j++) {
                    if (arr[j] % 2 == 0) {
                        int temp = arr[i];
                        arr[i] = arr[j];
                        arr[j] = temp;
                        break;
                    }
                }
            }
        }
        System.out.println("Rearranged Array:");
        for (int i = 0; i < arr.length; i++) {
            if (i == arr.length - 1) {
                System.out.println(arr[i]);
            } else {
                System.out.print(arr[i] + ", ");
            }
        }
    }
}
```

| RUBRIC |                                                                    |            |
|--------|--------------------------------------------------------------------|------------|
| SL     | Points to Meet                                                     | Marks [10] |
| 1      | Main method, and basic Java syntax                                 | 0.5        |
| 2      | Correct array declaration and initialization of required variables | 1          |
| 3      | Loop for traversing the array                                      | 1.25       |
| 4      | Checking even numbers using condition (% 2 == 0)                   | 1.5        |
| 5      | Swapping logic (in-place rearrangement)                            | 2          |
| 6      | Updating variables correctly                                       | 1.5        |
| 7      | Loop for printing array elements                                   | 1.25       |
| 8      | Proper output formatting (handling comma at the end)               | 1          |

### SET - [B]

| SOLUTION                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> public class EvenOddInPlace {     public static void main(String[] args) {         int[] arr = { 3, 6, 4, 7, 1, 9, 2};         int pos = 0;         for (int i = 0; i &lt; arr.length; i++) {             if (arr[i] % 2 != 0) {                 int temp = arr[i];                 arr[i] = arr[pos];                 arr[pos] = temp;                 pos++;             }         }         System.out.println("Rearranged Array:");         for (int i = 0; i &lt; arr.length; i++) {             if(i==arr.length-1){                 System.out.println(arr[i]);             }             else{                 System.out.print(arr[i] + ", ");             }         }     } } </pre> |

### Alternate Solution

```
public class EvenOddNested {
    public static void main(String[] args) {
        int[] arr = {3, 6, 4, 7, 1, 9, 2};
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0) {
                for (int j = i + 1; j < arr.length; j++) {
                    if (arr[j] % 2 != 0) {
                        int temp = arr[i];
                        arr[i] = arr[j];
                        arr[j] = temp;
                        break;
                    }
                }
            }
        }

        System.out.println("Rearranged Array:");
        for (int i = 0; i < arr.length; i++) {
            if (i == arr.length - 1) {
                System.out.println(arr[i]);
            } else {
                System.out.print(arr[i] + ", ");
            }
        }
    }
}
```

### RUBRIC

| SL | Points to Meet                                                     | Marks [10] |
|----|--------------------------------------------------------------------|------------|
| 1  | Main method, and basic Java syntax                                 | 0.5        |
| 2  | Correct array declaration and initialization of required variables | 1          |
| 3  | Loop for traversing the array                                      | 1.25       |
| 4  | Checking odd numbers using condition (% 2 != 0)                    | 1.5        |
| 5  | Swapping logic (in-place rearrangement)                            | 2          |
| 6  | Updating variables correctly                                       | 1.5        |
| 7  | Loop for printing array elements                                   | 1.25       |
| 8  | Proper output formatting (handling comma at the end)               | 1          |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate the final payable amount of a customer after applying discounts based on membership type, festival season, and purchase amount.

The program should:

- Input total purchase amount.
- Input membership type (1 = Regular, 2 = Silver, 3 = Gold).
- Input festival season (true/false).

Discount rules:

- Regular: 5%
- Silver: 10%
- Gold: 20%
- Add 5% if it is festival season.
- Add 7% if the purchase amount is greater than 10,000.

Display the total discount percentage and the final payable amount.

| Sample Input                                                                                     | Output                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total purchase amount: 15000<br>Enter membership type : 3<br>Is it festival season?: true  | Total Discount: 32.0%<br>Final Payable Amount: 10200.0<br><hr/> <b>Explanation:</b><br>Membership (20%) + Festival season (5%) + Amount > 10000 (7%)<br>Total discount = 32% |
| Enter total purchase amount: 8000<br>Enter membership type : 2<br>Is it festival season? : false | Total Discount: 10.0%<br>Final Payable Amount: 7200.0<br><hr/> <b>Explanation:</b><br>Membership 10% only                                                                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate the final fee to be paid for a training workshop based on participant category and early registration status and fee.

The program should:

- Input the workshop fee.
- Input participant category (1 = Student, 2 = Professional, 3 = Corporate).
- Input whether the participant registered early (true/false).

Discount rules:

- Student: 15% discount
- Professional: 8% discount
- Corporate: 5% discount
- If registered early, add an extra 6% discount.
- If the workshop fee is more than 25,000, add an additional 4% discount.

Display the total discount percentage and the final payable amount.

| Sample Input                                                                                       | Output                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter workshop fee: 30000<br>Enter participant category: 1<br>Was it an early registration? : true | Total Discount: 25.0%<br>Final Amount to be Paid: 22500.0<br><hr/> Explanation:<br>Student (15%) + Early registration (6%) + Fee > 25000 (4%)<br>Total discount = 25% |
| Enter workshop fee: 18000<br>Enter participant category: 2<br>Was it an early registration?: false | Total Discount: 8.0%<br>Final Amount to be Paid: 16560.0<br><hr/> Explanation:<br>Professional (8%) only                                                              |

## pSET - [A]

## SOLUTION

```
import java.util.Scanner;
public class ShoppingDiscount {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total purchase amount: ");
        double amount = sc.nextDouble();

        System.out.print("Enter membership type: ");
        int membership = sc.nextInt();

        System.out.print("Is it festival season?: ");
        boolean festival = sc.nextBoolean();

        double discount = 0.0;

        // Membership discount
        if (membership == 3) {           // Gold
            discount = discount + 20;
        }
        else if (membership == 2) {      // Silver
            discount = discount + 10;
        }
        else if (membership == 1) {      // Regular
            discount = discount + 5;
        }

        // Festival discount
        if (festival == true) {
            discount = discount + 5;
        }

        // Purchase above 10,000 discount
        if (amount > 10000) {
            discount = discount + 7;
        }

        // Calculate final amount
        double finalAmount = amount - (amount * discount / 100);

        System.out.println("Total Discount: " + discount + "%");
        System.out.println("Final Payable Amount: " + finalAmount);
    }
}
```

## RUBRIC

| SL | Points to Meet                                                                                                                                                                                                                         | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1  | Correct Input Handling (Scanner import, correct datatype)                                                                                                                                                                              | 2             |
| 2  | Membership Discount Logic <ul style="list-style-type: none"> <li>• Correct use of if-else if structure</li> <li>• Correct percentage assigned for each membership type</li> <li>• Adding the applied discount in a variable</li> </ul> | 3             |
| 3  | Additional Discount Conditions (Festival and workshop amount threshold) <ul style="list-style-type: none"> <li>• Correct if-else logic</li> <li>• Adding the applied discounts in a variable</li> </ul>                                | 1+1           |
| 4  | Final Amount Calculation                                                                                                                                                                                                               | 2             |
| 5  | Output and Code Structure                                                                                                                                                                                                              | 1             |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;
public class WorkshopFeeCalculator {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter workshop fee: ");
        double fee = sc.nextDouble();

        System.out.print("Enter participant category: ");
        int category = sc.nextInt();

        System.out.print("Was it an early registration?: ");
        boolean early = sc.nextBoolean();

        double discount = 0.0;

        // Category discount
        if (category == 1) {           // Student
            discount = discount + 15;
        }
        else if (category == 2) {      // Professional
            discount = discount + 8;
        }
        else if (category == 3) {      // Corporate
            discount = discount + 5;
        }

        // Early registration discount
        if (early == true) {
            discount = discount + 6;
        }

        // High fee discount
        if (fee > 25000) {
            discount = discount + 4;
        }

        // Calculate final fee
        double finalAmount = fee - (fee * discount / 100);

        System.out.println("Total Discount: " + discount + "%");
        System.out.println("Final Amount to be Paid: " + finalAmount);
    }
}
```

| RUBRIC |                                                                                                                                                                                                                                    |               |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                                                                                                                                                                                     | Marks<br>[10] |
| 1      | Correct Input Handling (Scanner import, correct datatype)                                                                                                                                                                          | 2             |
| 2      | Category Discount Logic <ul style="list-style-type: none"> <li>• Correct use of if-else if structure</li> <li>• Correct percentage assigned for each category type</li> <li>• Adding the applied discount in a variable</li> </ul> | 3             |
| 3      | Additional Discount Conditions (Registration status and fee) <ul style="list-style-type: none"> <li>• Correct if-else logic</li> <li>• Adding the applied discounts in a variable</li> </ul>                                       | 1+1           |
| 4      | Final Amount Calculation                                                                                                                                                                                                           | 2             |
| 5      | Output and Code Structure                                                                                                                                                                                                          | 1             |



CSE110: Programming Language I (Lab)  
Spring 2026  
Lab Exam - 02  
Duration: 15 Minutes

**A**

Name:

ID:

Section:

**Question 1 [C01] [10 Points]**

You are provided with a number  $N$  from the user. Your task is to print a pattern of numbers in descending order, where each row starts from  $n$  and decreases by 1 until the row number. The total number of rows will be equal to the value of  $N$ .

| Sample Input | Sample Output                             |
|--------------|-------------------------------------------|
| 4            | 4<br>43<br>432<br>4321                    |
| 6            | 6<br>65<br>654<br>6543<br>65432<br>654321 |

|           |       |          |
|-----------|-------|----------|
| Roll No.: | Date: | Section: |
|-----------|-------|----------|

### Question 1 [CO1] [10 Points]

You are given a number  $N$  from the user. Your task is to print a pattern where the number of characters in each row increases by 1, and the symbols alternate between two characters. The pattern should continue for  $N$  rows.

| Sample Input | Sample Output                           |
|--------------|-----------------------------------------|
| 4            | <pre> a aa aaa aaaa </pre>              |
| 6            | <pre> a aa aaa aaaa aaaaa aaaaaa </pre> |

```
1  import java.util.Scanner;
2  public class T1{
3      public static void main(String[]args){
4          Scanner sc = new Scanner(System.in);
5          int n = sc.nextInt();
6          for(int i=1;i<=n;i++){
7              for(int j=1;j<=i;j++){
8                  if(i%2==0){System.out.print("$");}
9                  else if(i%2!=0){System.out.print("&");}
10             }
11             System.out.println();
12         }
13     }
14 }
```

```
1  import java.util.Scanner;
2  public class T2{
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          int n = sc.nextInt();
6          for(int i=1;i<=n;i++){
7              for(int j=n;j>=n-i+1;j--){
8                  System.out.print(j);
9              }
10         System.out.println();
```

I

SET-A

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are building a text filtering tool for a data cleaning application. Users often input strings where words are separated by a special character. Some characters in each word appear only once (considered noise), while others appear multiple times (considered significant).

Your task is to develop a program that removes all unique (single-occurrence) characters from each word while keeping all occurrences of characters that appear more than once.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, remove characters that appear only once
- Print the filtered words

| Input                   | Output                            |
|-------------------------|-----------------------------------|
| heellllloo-worrlld<br>- | The words are:<br>eellllloo<br>rr |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are developing a text analyzer where users input strings with repeated characters and special separators. In this case, characters that appear only once are considered meaningful, while repeated characters are treated as noise.

Your task is to develop a program that keeps only the characters that appear exactly once in each word and removes all repeated characters.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, keep only characters that appear once
- Print the cleaned words

| Input                   | Output                      |
|-------------------------|-----------------------------|
| heellllloo#worrlld<br># | The words are:<br>h<br>wold |

# Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class Question1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            Count++;
                        }
                    }

                    // keep only repeated characters
                    if (count > 1) {
                        result = result + current;
                    }
                }

                System.out.println(result);
                word = "";
            } else {
                word = word + ch;
            }
        }
    }
}
```

| RUBRIC |                                                     |               |
|--------|-----------------------------------------------------|---------------|
| SL     | Points to Meet                                      | Marks<br>[10] |
| 1      | Correct input handling (string + separator)         | 2             |
| 2      | Proper manual splitting using loop (no built-in)    | 2             |
| 3      | Correct logic for counting character frequency      | 2             |
| 4      | Correct filtering condition (Set A: >1, Set B: ==1) | 2             |
| 5      | Correct output formatting and handling all words    | 2             |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;

public class Question2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            count++;
                        }
                    }

                    // keep only unique characters
                    if (count == 1) {
                        result = result + current;
                    }
                }

                System.out.println(result);
                word = "";
            } else {
                word = word + ch;
            }
        }
    }
}
```

| RUBRIC |                                                     |               |
|--------|-----------------------------------------------------|---------------|
| SL     | Points to Meet                                      | Marks<br>[10] |
| 1      | Correct input handling (string + separator)         | 2             |
| 2      | Proper manual splitting using loop (no built-in)    | 2             |
| 3      | Correct logic for counting character frequency      | 2             |
| 4      | Correct filtering condition (Set A: >1, Set B: ==1) | 2             |
| 5      | Correct output formatting and handling all words    | 2             |

SET-A

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to update the array by replacing the duplicate values of the array with zero. The updated array should be sorted in ascending order. Finally, you have to print the position of the number (you took as input from the user) from the updated array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem. Can't create a new array for this problem.**

| Input                                             | Output                                                                                                          |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=2</pre> | Updated array:<br>6, 2, 0, 1, 0, 9, 0<br>Sorted array:<br>0,0,0,1,2,6,9<br>Output:<br>Position of 2 = 5         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=5</pre> | Updated array:<br>6, 2, 0, 1, 0, 9, 0<br>Sorted array:<br>0,0,0,1,2,6,9<br>Output:<br>Position of 5 = Not found |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to create a new array that will contain only the unique (distinct) elements of the given array. The new array should be sorted in descending order. Finally, you have to print the position of the number (you took as input from the user) in the new array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem.**

| Input                                             | Output                                                                                   |
|---------------------------------------------------|------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=6</pre> | <pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 6 = 2</pre>         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=7</pre> | <pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 7 = Not found</pre> |

# Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 5;

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    arr[i] = 0;
                    break;
                }
            }
        }

        System.out.println("Updated Array:");
        for (int num : arr) {
            System.out.print(num + " ");
        }

        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
            if (arr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }else{
            System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
        }
    }
}
```

```
}
```

### RUBRIC

| SL | Points to Meet                                                        | Marks<br>[10] |
|----|-----------------------------------------------------------------------|---------------|
| 1  | Handle Duplicates by replacing with 0 in the array                    | 3             |
| 2  | Sort the array in ascending order                                     | 3             |
| 3  | Search and find the position                                          | 2             |
| 4  | Follow the constraint (No built-in sort + No new array)               | 1             |
| 5  | Maintain Output format (updated array, sorted array, output position) | 1             |

### SET - [B]

### SOLUTION

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 6;

        int[] newArr = new int[arr.length];
        int count = 0;

        for (int i = 0; i < arr.length; i++) {
            boolean isDuplicate = false;

            for (int j = 0; j < count; j++) {
                if (arr[i] == newArr[j]) {
                    isDuplicate = true;
                    break;
                }
            }

            if (!isDuplicate) {
                newArr[count] = arr[i];
                count++;
            }
        }

        System.out.println("\nNew array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
        }

        for (int i = 0; i < newArr.length - 1; i++) {
            int maxIndex = i;

            for (int j = i + 1; j < newArr.length; j++) {
                if (newArr[j] > newArr[maxIndex]) {
                    maxIndex = j;
                }
            }

            int temp = newArr[i];
            newArr[i] = newArr[maxIndex];
            newArr[maxIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
            if (newArr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }
    }
}
```

```

    }else{
        System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
    }
}
}

```

#### RUBRIC

| SL | Points to Meet                                                    | Marks<br>[10] |
|----|-------------------------------------------------------------------|---------------|
| 1  | Create unique elements in new array                               | 3             |
| 2  | Sort the array in descending order                                | 3             |
| 3  | Search and find the position                                      | 2             |
| 4  | Follow the constraint (No built-in sort)                          | 1             |
| 5  | Maintain Output Format (new array, sorted array, output position) | 1             |

SET-A

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate electricity bills using a slab system. The program should take **total electricity consumption** (kWh), **connected load** (kW), and **customer type** (1 = Residential, 2 = Commercial, 3 = Agricultural) as input. Then calculate the bill as described below:

| Energy Charge Slabs                                                                                                                                       | Apply discount/rebate based on customer type                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>0 - 100 KWh → 7 Tk per KWh</li> <li>101 - 300 KWh → 9 Tk per KWh</li> <li>Above 300 KWh → 12 Tk per KWh</li> </ul> | <ul style="list-style-type: none"> <li>Residential → 5%</li> <li>Commercial → 3%</li> <li>Agricultural → 10%</li> </ul> |
| <ul style="list-style-type: none"> <li>Demand charge = 30 Tk per kW of connected load.</li> </ul>                                                         |                                                                                                                         |

Bill Calculation:

- Energy charge = Calculated according to slab rates
- Demand charge = Connected load × 30
- Discount = (Energy charge + Demand charge) × Discount Rate
- Final bill = (Energy charge + Demand charge) – Discount

| Input                                                                          | output                                                                                                                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Total electricity consumption = 309<br>Connected load = 2<br>Customer type = 1 | <b>BILL DETAILS:</b><br><br>Energy consumption charge = 2608 Tk<br><br>Demand charge = 60 Tk<br><br>Discount = 133.4 Tk<br><br>Total bill = 2534.6 Tk | <b>Energy consumption charge:</b><br>100*7 = 700tk (for first 100kwh)<br>200*9 = 1800 (for next 200 kwh)<br>9*12= 108 (above 300kwh)<br><br><b>Total energy consumption charge = 700+1800+108 = 2608</b><br><br><b>Demand charge = 2*30 = 60</b><br><br><b>Discount / rebate =</b><br>(2608+60)*5% = 133.4<br><br><b>Total bill = ((2608+ 60)-(2608+ 60)*.05) = 2534.6</b> |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate water bills using a slab system. The program should take **total water consumption** (in cubic meters, m<sup>3</sup>), **connection size**, and **customer type** (1 = Residential, 2 = Commercial, 3 = Agricultural) as input. Then calculate the bill as described below:

| Water Charge Slabs                                                                                                                                                                                                   | Apply discount/rebate based on customer type                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>0 - 50 m<sup>3</sup> → 10 Tk per m<sup>3</sup></li> <li>51- 100 m<sup>3</sup> → 15 Tk per m<sup>3</sup></li> <li>Above 100 m<sup>3</sup> → 20 Tk per m<sup>3</sup></li> </ul> | <ul style="list-style-type: none"> <li>Residential → 5%</li> <li>Commercial → 3%</li> <li>Agricultural → 10%</li> </ul> |
| <ul style="list-style-type: none"> <li>Connection charge = 50 Tk per unit</li> </ul>                                                                                                                                 |                                                                                                                         |

Bill Calculation:

- Water charge = Calculated according to slab rates
- Connection charge = Connection size × 50
- Discount = (Water charge + Connection charge) × Discount Rate
- Final bill = (Water charge + Connection charge) – Discount

| Input                                                                    | output                                                                                                                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Total water consumption = 109<br>Connected size = 2<br>Customer type = 1 | <b>BILL DETAILS:</b><br><br>Water consumption charge = 1430 Tk<br><br>Connection charge = 100 Tk<br><br>Discount = 76.5<br><br>Total bill = 1453.5 Tk | <b>Water consumption charge:</b><br>50*10 = 500tk (for first 50m <sup>3</sup> )<br>50*15 = 750 (for next 50 m <sup>3</sup> )<br>9*20= 180 (above 100 m <sup>3</sup> )<br><br><b>Total water consumption charge</b><br>= 500+750+180 = 1430<br><br><b>Connection charge</b> = 2*50=100<br><br><b>Discount / rebate =</b><br>(1430+100)*5% = 76.5<br><br><b>Total bill =</b><br>((1430+100)-(1430+100)*.05) = 1453.5 |

# Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;
public class ElectricityBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Total electricity consumption = ");
        int consumption = sc.nextInt();

        System.out.print("Connected load = ");
        int load = sc.nextInt();

        System.out.print("Customer type = ");
        int type = sc.nextInt();

        double energyCharge = 0;

        // Slab Calculation
        if (consumption <= 100) {
            energyCharge = consumption * 7;
        } else if (consumption <= 300) {
            energyCharge = (100 * 7) + (consumption - 100) * 9;
        } else {
            energyCharge = (100 * 7) + (200 * 9) + (consumption - 300) * 12;
        }

        // Demand Charge
        double demandCharge = load * 30;

        double total = energyCharge + demandCharge;

        // Discount
        double discount = 0;
        if (type == 1) {
            discount = total * 0.05;    // Residential
        } else if (type == 2) {
            discount = total * 0.03;    // Commercial
        } else if (type == 3) {
            discount = total * 0.10;    // Agricultural
        } else {
            discount = 0;
        }

        double finalBill = total - discount;

        // Output
        System.out.println("\n----- BILL DETAILS -----");
        System.out.println("Energy Charge = " + energyCharge + " Tk");
        System.out.println("Demand Charge = " + demandCharge + " Tk");
    }
}
```

```

        System.out.println("Discount = " + discount + " Tk");
        System.out.println("Total Bill = " + finalBill + " Tk");
    }
}

```

#### RUBRIC

| SL | Points to Meet                                                                                     | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object) | 2             |
| 2  | Taking 3 inputs properly                                                                           | 1             |
| 3  | Calculate energy consumption charge properly                                                       | 2             |
| 4  | Calculate demand charge properly                                                                   | 1             |
| 5  | Calculate discount properly                                                                        | 2             |
| 6  | Calculate final bill properly                                                                      | 1             |
| 7  | Proper print statements according to sample input and output                                       | 1             |

#### SET - [B]

#### SOLUTION

```

import java.util.Scanner;
public class WaterBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Total water consumption = ");
        int consumption = sc.nextInt();

        System.out.print("Enter connection size = ");
        int size = sc.nextInt();

        System.out.print("Customer type = ");
        int type = sc.nextInt();

        double waterCharge = 0;

        // Slab Calculation
        if (consumption <= 50) {
            waterCharge = consumption * 10;
        } else if (consumption <= 100) {
            waterCharge = (50 * 10) + (consumption - 50) * 15;
        } else {
            waterCharge = (50 * 10) + (50 * 15) + (consumption - 100) * 20;
        }

        // Connection Charge
        double connectionCharge = size * 50;

        double total = waterCharge + connectionCharge;

        // Discount
        double discount = 0;

        if (type == 1) {
            discount = total * 0.05;    // Residential
        } else if (type == 2) {
            discount = total * 0.03;    // Commercial
        } else if (type == 3) {
            discount = total * 0.10;    // Agricultural
        } else {
            discount = 0;
        }

        double finalBill = total - discount;

        // Output
        System.out.println("\n----- BILL DETAILS -----");
        System.out.println("Water Consumption charge = " + waterCharge + " Tk");
        System.out.println("Connection Charge = " + connectionCharge + " Tk");
        System.out.println("Discount = " + discount + " Tk");
        System.out.println("Total Bill = " + finalBill + " Tk");
    }
}

```

**RUBRIC**

| SL | Points to Meet                                                                                     | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object) | 2             |
| 2  | Taking 3 inputs properly                                                                           | 1             |
| 3  | Calculate Water consumption charge properly                                                        | 2             |
| 4  | Calculate Connection charge properly                                                               | 1             |
| 5  | Calculate discount properly                                                                        | 2             |
| 6  | Calculate final bill properly                                                                      | 1             |
| 7  | Proper print statements according to sample input and output                                       | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze food sales during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of student stalls in the festival. For each student stall, the number of items sold is equal to the stall number. For example, Stall 1 sells 1 item, Stall 2 sells 2 items, Stall 3 sells 3 items, and so on. For each item, the user must input its price.

For every student stall, the program should display all items that cost more than 200 taka (premium items) and calculate the total sales of that stall. If a stall has 2 or more premium items, the program should print **"Popular Stall!"**. After processing all stalls, the program should display the overall total sales of the festival.

| Input                                                                                                                                                                                                                                                        | Output                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of stalls: 2<br><br>Stall 1:<br>Enter price of item 1: 220<br><br>Stall 2:<br>Enter price of item 1: 150<br>Enter price of item 2: 250                                                                                                          | Stall 1:<br>Premium items: 220<br>Total sales: 220<br><br>Stall 2:<br>Premium items: 250<br>Total sales: 400<br><br>Overall Festival Sales: 620                                                                                     |
| Enter number of stalls: 3<br><br>Stall 1:<br>Enter price of item 1: 150<br><br>Stall 2:<br>Enter price of item 1: 250<br>Enter price of item 2: 300<br><br>Stall 3:<br>Enter price of item 1: 100<br>Enter price of item 2: 220<br>Enter price of item 3: 50 | Stall 1:<br>Premium items:<br>Total sales: 150<br><br>Stall 2:<br>Premium items: 250 300<br>Total sales: 550<br><b>Popular Stall!</b><br><br>Stall 3:<br>Premium items: 220<br>Total sales: 370<br><br>Overall Festival Sales: 1070 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze cultural sessions during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of sessions. For each session, the program should first take input of the number of performances. Then, for each performance, the user must input its duration (in minutes).

For every session, the program should count how many performances have a duration greater than 30 minutes and calculate the total duration of that session. If more than half of the performances in a session have a duration greater than 30 minutes, the program should print **"Long Session!"**. After processing all sessions, the program should display the total duration of all sessions combined.

| Input                                                                                                                                                                                                                                                                                                                                    | Output                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions: 1<br><br>Session 1:<br>Enter number of performances: 4<br>Enter duration of performance 1: 10<br>Enter duration of performance 2: 35<br>Enter duration of performance 3: 45<br>Enter duration of performance 4: 20                                                                                             | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><br>Overall Total Duration: 110                                                                                               |
| Enter number of sessions: 2<br><br>Session 1:<br>Enter number of performances: 3<br>Enter duration of performance 1: 20<br>Enter duration of performance 2: 40<br>Enter duration of performance 3: 50<br><br>Session 2:<br>Enter number of performances: 2<br>Enter duration of performance 1: 25<br>Enter duration of performance 2: 30 | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><b>Long Session!</b><br><br>Session 2:<br>Long performances count: 0<br>Total duration: 55<br><br>Overall Total Duration: 165 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a string from the user and produces its run-length encoded form. In run-length encoding, each group of consecutive identical characters is replaced by the character followed by its count. If a character appears only once in a row, write the character without a number. After producing the encoded string, compare its length with the original. If the encoded version is shorter, print the encoded string. Otherwise, print the original string unchanged.

| Sample Input | Sample Output                                                |
|--------------|--------------------------------------------------------------|
| aabbbccdddee | a2b3c2d4e2                                                   |
| abcd         | abcd<br>(encoded "a1b1c1d1" is longer, so<br>print original) |
| aaabbaaa     | a3b2a3                                                       |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a main string and a pattern string, then prints how many times the pattern appears in the main string (non-overlapping occurrences only). You must implement the search manually, no built-in method may be used to find or detect the substring.

| Sample Input                      | Sample Output |
|-----------------------------------|---------------|
| main: ababab<br>pattern: ab       | Count: 3      |
| main: aaaaaa<br>pattern: aa       | Count: 3      |
| main: hello world<br>pattern: xyz | Count: 0      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

A weekly attendance system stores 7 digits in an array, where **each index** represents a day of the week in order from Saturday to Friday. In this system, classes on **Saturday** (index 0) and **Tuesday** (index 3) are conducted **online**, while all other days are conducted **in-person**. A day is considered balanced if the number of elements before it (to the left) with lower values is equal to the number of elements after it (to the right) with higher values.

Write a Java program to determine and print **all** balanced days **only among the in-person days**, and also print the **total** number of such balanced in-person days.

| Given Array                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Output                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <pre>int[] arr = {40, 10, 20, 100, 15, 30, 5};</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Balanced in-person days are: 2 4 6</p> <p>Total balanced in-person days: 3</p> |
| <p><b>Explanation:</b></p> <p>Monday (index 2, value 20): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> <p>Wednesday (index 4, value 15): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> <p>Friday (index 6, value 5): left lower count = 0, right higher count = 0, ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> |                                                                                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

A FIFA World Cup performance tracker stores the number of goal chances created for 7 matches in an array, where **each index** represents a match in sequence from Match 1 to Match 7. In this system, matches played on **Monday** (index 1) and **Thursday** (index 4) are against **rival teams**, while all other matches are **non-rival**. A match is considered Growth-Stable if the number of non-rival matches before it (to the left) with higher values is equal to the number of non-rival matches after it (to the right) with lower values.

Write a Java program to determine and print **all** Growth-Stable matches **only** among the **non-rival** matches, and also print the **total** number of such Growth-Stable matches.

| Given Array                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Output                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <pre>int[] scores = {40, 10, 20, 25, 15, 30, 5};</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Growth-Stable matches are: 2 3 5</p> <p>Total Growth-Stable matches: 3</p> |
| <p><b>Explanation:</b></p> <p>Match 3 (index 2, value 20): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> <p>Match 4 (index 3, value 25): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> <p>Match 6 (index 5, value 30): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> |                                                                               |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate calorie intake during Iftar. The program should ask the user to enter a six-digit number, and the product of the last three digits of this number will be used as the total calorie limit. The user must then input how many dates they want to eat and how many jilapis they want to eat. Each date contains 80 calories and each jilapi contains 140 calories, and the person must always eat dates first. The program should first calculate the total calories from the dates; if the calories from the dates exceed the calorie limit, then the person cannot eat any jilapis and must drink water instead. If the calories from the dates are within the limit, the program should calculate how many jilapis can be eaten without exceeding the total calorie limit.

| Input                                                                                    | Output                                                                                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Enter a 6-digit number: 123456<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 120<br>Calories from dates: 160<br>Dates exceed the calorie limit.<br>You cannot eat jilapis. Drink water instead. |
| Enter a 6-digit number: 243798<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 504<br>Calories from dates: 160<br>You can eat up to 2 jilapis.<br>Total calories consumed: 440                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a **Java program** that calculates calorie intake during Sehri. The program should take input for the number of rice plates, glasses of milk, and bananas a person wants to eat. Each plate of rice has 200 calories, each glass of milk has 120 calories, and each banana has 100 calories, with a total calorie limit of 800. The program must calculate rice calories first, and if rice alone exceeds 800 calories, the person can only drink water. If not, then calculate the calories from milk and bananas and make sure the total does not go over 800 calories.

| Sample Input                                                                                          | Sample Output                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of plates of rice: 5<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 1000<br>Calorie limit exceeded by rice alone!<br>You should drink only water.                                                                      |
| Enter number of plates of rice: 2<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 400<br>Milk calories = 120<br>Banana calories = 100<br>Total calorie intake = 620<br>You are within the 800 calorie limit. Good Sehri!             |
| Enter number of plates of rice: 3<br>Enter number of glasses of milk: 2<br>Enter number of bananas: 2 | Rice calories = 600<br>Adding milk and bananas exceeds the 800 calorie limit.<br>You should reduce milk or bananas.<br>Maximum remaining calories after rice = 200 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C01] [10 Points]**

You are provided with a number N from the user. Your task is to print a pattern of numbers in descending order, where each row starts from n and decreases by 1 until the row number. The total number of rows will be equal to the value of N.

| Sample Input | Sample Output                             |
|--------------|-------------------------------------------|
| 4            | 4<br>43<br>432<br>4321                    |
| 6            | 6<br>65<br>654<br>6543<br>65432<br>654321 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

You are given a number N from the user. Your task is to print a pattern where the number of characters in each row increases by 1, and the symbols alternate between two characters. The pattern should continue for N rows.

| Sample Input | Sample Output                                           |
|--------------|---------------------------------------------------------|
| 4            | &<br>\$\$<br>&&&<br>\$\$\$\$                            |
| 6            | &<br>\$\$<br>&&&<br>\$\$\$\$<br>&&&&&<br>\$\$\$\$\$\$\$ |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are building a text filtering tool for a data cleaning application. Users often input strings where words are separated by a special character. Some characters in each word appear only once (considered noise), while others appear multiple times (considered significant).

Your task is to develop a program that removes all unique (single-occurrence) characters from each word while keeping all occurrences of characters that appear more than once.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, remove characters that appear only once
- Print the filtered words

| Input                   | Output                            |
|-------------------------|-----------------------------------|
| heellllloo-worrlld<br>- | The words are:<br>eellllloo<br>rr |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are developing a text analyzer where users input strings with repeated characters and special separators. In this case, characters that appear only once are considered meaningful, while repeated characters are treated as noise.

Your task is to develop a program that keeps only the characters that appear exactly once in each word and removes all repeated characters.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, keep only characters that appear once
- Print the cleaned words

| Input                   | Output                      |
|-------------------------|-----------------------------|
| heellllloo#worrlld<br># | The words are:<br>h<br>wold |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to update the array by replacing the duplicate values of the array with zero. The updated array should be sorted in ascending order. Finally, you have to print the position of the number (you took as input from the user) from the updated array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem. Can't create a new array for this problem.**

| Input                                             | Output                                                                                                                      |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=2</pre> | <p>Updated array:<br/>6, 2, 0, 1, 0, 9, 0<br/>Sorted array:<br/>0,0,0,1,2,6,9<br/>Output:<br/>Position of 2 = 5</p>         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=5</pre> | <p>Updated array:<br/>6, 2, 0, 1, 0, 9, 0<br/>Sorted array:<br/>0,0,0,1,2,6,9<br/>Output:<br/>Position of 5 = Not found</p> |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to create a new array that will contain only the unique (distinct) elements of the given array. The new array should be sorted in descending order. Finally, you have to print the position of the number (you took as input from the user) in the new array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem.**

| Input                                             | Output                                                                                       |
|---------------------------------------------------|----------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=6</pre> | New array:<br>6, 2, 1, 9<br>Sorted array:<br>9,6,2,1<br>Output:<br>Position of 6 = 2         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=7</pre> | New array:<br>6, 2, 1, 9<br>Sorted array:<br>9,6,2,1<br>Output:<br>Position of 7 = Not found |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate electricity bills using a slab system. The program should take **total electricity consumption** (kWh), **connected load** (kW), and **customer type** (1 = Residential, 2 = Commercial, 3 = Agricultural) as input. Then calculate the bill as described below:

| Energy Charge Slabs                                                                                                                                       | Apply discount/rebate based on customer type                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>0 - 100 KWh → 7 Tk per KWh</li> <li>101 - 300 KWh → 9 Tk per KWh</li> <li>Above 300 KWh → 12 Tk per KWh</li> </ul> | <ul style="list-style-type: none"> <li>Residential → 5%</li> <li>Commercial → 3%</li> <li>Agricultural → 10%</li> </ul> |
| <ul style="list-style-type: none"> <li>Demand charge = 30 Tk per kW of connected load.</li> </ul>                                                         |                                                                                                                         |

Bill Calculation:

- Energy charge = Calculated according to slab rates
- Demand charge = Connected load × 30
- Discount = (Energy charge + Demand charge) × Discount Rate
- Final bill = (Energy charge + Demand charge) – Discount

| Input                                                                          | output                                                                                                                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Total electricity consumption = 309<br>Connected load = 2<br>Customer type = 1 | <b>BILL DETAILS:</b><br><br>Energy consumption charge = 2608 Tk<br><br>Demand charge = 60 Tk<br><br>Discount = 133.4 Tk<br><br>Total bill = 2534.6 Tk | <b>Energy consumption charge:</b><br>100*7 = 700tk (for first 100kwh)<br>200*9 = 1800 (for next 200 kwh)<br>9*12= 108 (above 300kwh)<br><br><b>Total energy consumption charge = 700+1800+108 = 2608</b><br><br><b>Demand charge = 2*30 = 60</b><br><br><b>Discount / rebate =</b><br>(2608+60)*5% = 133.4<br><br><b>Total bill = ((2608+ 60)-(2608+ 60)*.05) = 2534.6</b> |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate water bills using a slab system. The program should take **total water consumption** (in cubic meters, m<sup>3</sup>), **connection size**, and **customer type** (1 = Residential, 2 = Commercial, 3 = Agricultural) as input. Then calculate the bill as described below:

| Water Charge Slabs                                                                                                                                                                                                   | Apply discount/rebate based on customer type                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>0 - 50 m<sup>3</sup> → 10 Tk per m<sup>3</sup></li> <li>51- 100 m<sup>3</sup> → 15 Tk per m<sup>3</sup></li> <li>Above 100 m<sup>3</sup> → 20 Tk per m<sup>3</sup></li> </ul> | <ul style="list-style-type: none"> <li>Residential → 5%</li> <li>Commercial → 3%</li> <li>Agricultural → 10%</li> </ul> |
| <ul style="list-style-type: none"> <li>Connection charge = 50 Tk per unit</li> </ul>                                                                                                                                 |                                                                                                                         |

Bill Calculation:

- Water charge = Calculated according to slab rates
- Connection charge = Connection size × 50
- Discount = (Water charge + Connection charge) × Discount Rate
- Final bill = (Water charge + Connection charge) – Discount

| Input                                                                    | output                                                                                                                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Total water consumption = 109<br>Connected size = 2<br>Customer type = 1 | <b>BILL DETAILS:</b><br><br>Water consumption charge = 1430 Tk<br><br>Connection charge = 100 Tk<br><br>Discount = 76.5<br><br>Total bill = 1453.5 Tk | <b>Water consumption charge:</b><br>50*10 = 500tk (for first 50m <sup>3</sup> )<br>50*15 = 750 (for next 50 m <sup>3</sup> )<br>9*20= 180 (above 100 m <sup>3</sup> )<br><br><b>Total water consumption charge</b><br>= 500+750+180 = 1430<br><br><b>Connection charge</b> = 2*50=100<br><br><b>Discount / rebate =</b><br>(1430+100)*5% = 76.5<br><br><b>Total bill =</b><br>((1430+100)-(1430+100)*.05) = 1453.5 |

SET-A

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

The water park needs a program to manage visitor eligibility for various activities and special offers. Write a Java program that first takes the number of visitor sessions, followed by the number of visitors in each session. For each visitor, the program should further take the Activity Level (1 for Low, 2 for Moderate, 3 for High) and whether the visitor has visited the park previously (true for previous visitors, false for first-time visitors). The park offers the following activities and special offers: the **Lazy Pool** is available for visitors with an activity level of 2 or 3; the **Water Coaster** is available for visitors with an activity level of 3 who have visited the park before; and the **Special Offer** is available to first-time visitors. The program should count and display the total number of eligible visitors for each activity and for the Special Offer.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of visitor sessions: 2<br><br>Enter number of visitors in session 1: 3<br>Enter activity level of visitor 1: 2<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 1<br>Enter previous visit of visitor 2: false<br>Enter activity level of visitor 3: 2<br>Enter previous visit of visitor 3: false<br><br>Enter number of visitors in session 2: 2<br>Enter activity level of visitor 1: 3<br>Enter previous visit of visitor 1: true<br>Enter activity level of visitor 2: 2<br>Enter previous visit of visitor 2: false | Lazy Pool Riders: 4<br>Water Coaster Riders: 1<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>For session 1, Visitor 1 qualifies for the Lazy Pool but not the Water Coaster due to an activity level of 2. Visitor 2 qualifies for the Special Offer as a first-time visitor. Visitor 3 qualifies for the Lazy Pool and Special Offer.<br><br>For session 2, Visitor 1 qualifies for both the Lazy Pool and the Water Coaster. Visitor 2 qualifies for the Lazy Pool and the Special Offer. |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

You are asked to write a Java program for an amusement park that manages visitor eligibility for multiple rides and special offers. The program first takes the number of entry sessions from the management team. For each session, the program takes the number of visitors, followed by each visitor's height (in cm) and age (in years). The park has multiple rides with eligibility rules: the **Roller Coaster** requires a minimum height of 140 cm and a minimum age of 12 years, the **Ferris Wheel** allows visitors aged 5 years or older, and the **Bumper Cars** allow visitors aged 8 years or older with a maximum height of 200 cm for safety. Additionally, visitors under 12 years are eligible for a **Special Discount Offer**. The program should count the number of visitors who are eligible for each ride, as well as how many receive the discount. Finally, the program should display these counts after processing all sessions.

| Sample Input                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions = 3<br><br>Enter number of visitors in session 1 = 3<br>Enter height of visitor 1: 150<br>Enter age of visitor 1: 14<br>Enter height of visitor 2: 120<br>Enter age of visitor 2: 10<br>Enter height of visitor 3: 160<br>Enter age of visitor 3: 20<br><br>Enter number of visitors in session 2 = 2<br>Enter height of visitor 1: 130<br>Enter age of visitor 1: 6<br>Enter height of visitor 2: 170<br>Enter age of visitor 2: 13<br><br>Enter number of visitors in session 3 = 1<br>Enter height of visitor 1: 180<br>Enter age of visitor 1: 9 | Roller Coaster Riders: 3<br>Ferris Wheel Riders: 6<br>Bumper Cars Riders: 5<br>Visitors with Special Offer: 3<br><br><b>Explanation:</b><br>Visitor 1 in session 1 qualifies for all rides, Visitor 2 qualifies for the Ferris Wheel and Bumper Cars and gets the Special Offer, and Visitor 3 qualifies for all rides.<br>In session 2, Visitor 1 qualifies for the Ferris Wheel and the Special Offer, while Visitor 2 qualifies for all rides.<br>In session 3, the visitor qualifies for the Ferris Wheel and Bumper Cars and receives the Special Offer. |

# Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class WaterPark {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lazyPool = 0, waterCoaster = 0, specialOffer = 0;

        System.out.print("Enter number of visitor sessions: ");
        int sessions = sc.nextInt();

        for (int i = 1; i <= sessions; i++) {
            System.out.print("Enter number of visitors in session " + i + ": ");
            int visitors = sc.nextInt();

            for (int j = 1; j <= visitors; j++) {
                System.out.print("Enter activity level of visitor " + j + ": ");
                int activityLevel = sc.nextInt();
                System.out.print("Enter previous visit of visitor " + j + ": ");
                boolean previousVisit = sc.nextBoolean();

                if (activityLevel == 2 || activityLevel == 3) {
                    lazyPool++;
                }

                if (activityLevel == 3 && previousVisit==true) {
                    waterCoaster++;
                }

                if (previousVisit==false) {
                    specialOffer++;
                }
            }
        }

        System.out.println("Lazy Pool Riders: " + lazyPool);
        System.out.println("Water Coaster Riders: " + waterCoaster);
        System.out.println("Visitors with Special Offer: " + specialOffer);
    }
}
```

**RUBRIC**

| SL | Points to Meet                                                                               | Marks [10] |
|----|----------------------------------------------------------------------------------------------|------------|
| 1  | The program takes input for sessions, number of visitors, activity level, and previous visit | 1          |
| 2  | Correctly calculates eligibility for all activities (Lazy Pool, Water Coaster)               | 3          |
| 3  | Correctly calculates eligibility for Special Offer                                           | 1          |
| 4  | Displays total counts for all activities and Special Offers correctly                        | 2          |
| 5  | Proper use of nested loops and conditionals                                                  | 3          |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;

public class AmusementPark {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int rollerCoaster = 0, ferrisWheel = 0, bumperCars = 0, specialOffer = 0;

        System.out.print("Enter number of sessions = ");
        int sessions = sc.nextInt();

        for (int i = 1; i <= sessions; i++) {
            System.out.print("Enter number of visitors in session " + i + "= ");
            int visitors = sc.nextInt();

            for (int j = 1; j <= visitors; j++) {
                System.out.print("Enter height of visitor " + j + ": ");
                int height = sc.nextInt();
                System.out.print("Enter age of visitor " + j + ": ");
                int age = sc.nextInt();

                if (height >= 140 && age >= 12) {
                    rollerCoaster++;
                }

                if (age >= 5) {
                    ferrisWheel++;
                }

                if (age >= 8 && height <= 200) {
                    bumperCars++;
                }

                if (age < 12) {
                    specialOffer++;
                }
            }
        }

        System.out.println("Roller Coaster Riders: " + rollerCoaster);
        System.out.println("Ferris Wheel Riders: " + ferrisWheel);
        System.out.println("Bumper Cars Riders: " + bumperCars);
        System.out.println("Visitors with Special Offer: " + specialOffer);
    }
}
```

| RUBRIC |                                                                                            |            |
|--------|--------------------------------------------------------------------------------------------|------------|
| SL     | Points to Meet                                                                             | Marks [10] |
| 1      | The program takes input for sessions, number of visitors, height, and age                  | 1          |
| 2      | Correctly calculates eligibility for all rides (Roller Coaster, Ferris Wheel, Bumper Cars) | 3          |
| 3      | Correctly calculates Special Offer eligibility                                             | 1          |
| 4      | Displays total counts for all rides and Special Offers correctly                           | 2          |
| 5      | Proper use of nested loops                                                                 | 3          |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

## Lab Quiz-3 Question

Write a Java program that reads two strings and a character representing an arithmetic operator (\* or /). Each string may contain digits and other characters. The program should extract only the **odd digits** from each string, form two integers from those digits, and then perform the operation indicated by the operator. Finally, print both integers and the result of the operation.

| Input        | Output |
|--------------|--------|
| s1 = "a123b" | 135    |
| s2 = "x456y" | 57     |
| op = *       | 7695   |
| s1 = "p987q" | 987    |
| s2 = "r654s" | 5      |
| op = /       | 197    |

Write a Java program that reads two strings and a character representing an arithmetic operator (+ or -). Each string may contain digits and other characters. The program should extract only the **even digits** from each string, form two integers from those digits, and then perform the operation indicated by the operator. Finally, print both integers and the result of the operation.

| Input        | Output |
|--------------|--------|
| s1 = "a123b" | 2      |
| s2 = "x456y" | 46     |

|              |     |
|--------------|-----|
| op = +       | 48  |
| s1 = "p987q" | 8   |
| s2 = "r654s" | 64  |
| op = -       | -56 |

### Set-A

#### SOLUTION

```
import java.util.Scanner;
public class A{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        String s1 = sc.next();
        String s2 = sc.next();
        char op = sc.next().charAt(0);
        int num1 = 0, num2 = 0;
        for(int i = 0; i<s1.length(); i++){
            char ch1 = s1.charAt(i);
            char ch2 = s2.charAt(i);
            if(ch1>='0' && ch1<='9'){
                int digit = (int)ch1 - 48;
                if(digit%2 != 0){
                    num1 = num1*10 + digit;
                }
            }
            if(ch2>='0' && ch2<='9'){
                int digit = (int)ch2 - 48;
                if(digit%2 != 0){
                    num2 = num2*10 + digit;
                }
            }
        }
        System.out.println(num1);
        System.out.println(num2);
        if(op == '*'){
            System.out.println(num1*num2);
        }
        else if(op == '/'){
            System.out.println(num1/num2);
        }
    }
}
```

```
}  
}  
}
```

#### RUBRIC

| SL | Points to Meet                                                                | Marks [10] |
|----|-------------------------------------------------------------------------------|------------|
| 1  | Importing Scanner class, main method, and Java syntax                         | 0.5        |
| 2  | Correctly takes two strings and an operator from the user                     | 1          |
| 3  | Properly handles the iteration and extracts the characters of both Strings    | 1.5        |
| 4  | Checking the digits using characters ('0'-'9') or ASCII values [both Strings] | 1          |
| 5  | Converting the character to a digit for both Strings                          | 1          |
| 6  | Searching the odd digits [both Strings]                                       | 0.5        |
| 7  | Initialize num1, num2, and correctly prepare the numbers                      | 3          |

|   |                                                                    |     |
|---|--------------------------------------------------------------------|-----|
| 8 | Checking the operations (*,/) and performing the correct operation | 1   |
| 9 | Printing all three outputs correctly                               | 0.5 |

### SET - [B]

#### SOLUTION

```
import java.util.Scanner;
public class B{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        String s1 = sc.next();
        String s2 = sc.next();
        char op = sc.next().charAt(0);
        int num1 = 0, num2 = 0;
        for(int i = 0; i<s1.length(); i++){
            char ch1 = s1.charAt(i);
            char ch2 = s2.charAt(i);
            if(ch1>='0' && ch1<='9'){
                int digit = (int)ch1 - 48;
                if(digit%2 == 0){
                    num1 = num1*10 + digit;
                }
            }
            if(ch2>='0' && ch2<='9'){
                int digit = (int)ch2 - 48;
                if(digit%2 == 0){
                    num2 = num2*10 + digit;
                }
            }
        }
        System.out.println(num1);
        System.out.println(num2);
        if(op == '+'){
            System.out.println(num1+num2);
        }
        else if(op == '-'){
            System.out.println(num1-num2);
        }
    }
}
```

#### RUBRIC

| SL | Points to Meet                                                                | Marks [10] |
|----|-------------------------------------------------------------------------------|------------|
| 1  | Importing Scanner class, main method, and Java syntax                         | 0.5        |
| 2  | Correctly takes two strings and an operator from the user                     | 1          |
| 3  | Properly handles the iteration and extracts the characters of both Strings    | 1.5        |
| 4  | Checking the digits using characters ('0'-'9') or ASCII values [both Strings] | 1          |
| 5  | Converting the character to a digit for both Strings                          | 1          |
| 6  | Searching the even digits [both Strings]                                      | 0.5        |
| 7  | Initialize num1, num2, and correctly prepare the numbers                      | 3          |
| 8  | Checking the operations (+,-) and performing the correct operation            | 1          |
| 9  | Printing all three outputs correctly                                          | 0.5        |

SET-A

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate the final payable amount of a customer after applying discounts based on membership type, festival season, and purchase amount.

The program should:

- Input total purchase amount.
- Input membership type (1 = Regular, 2 = Silver, 3 = Gold).
- Input festival season (true/false).

Discount rules:

- Regular: 5%
- Silver: 10%
- Gold: 20%
- Add 5% if it is festival season.
- Add 7% if the purchase amount is greater than 10,000.

Display the total discount percentage and the final payable amount.

| Sample Input                                                                                     | Output                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total purchase amount: 15000<br>Enter membership type : 3<br>Is it festival season?: true  | Total Discount: 32.0%<br>Final Payable Amount: 10200.0<br><hr/> <b>Explanation:</b><br>Membership (20%) + Festival season (5%) + Amount > 10000 (7%)<br>Total discount = 32% |
| Enter total purchase amount: 8000<br>Enter membership type : 2<br>Is it festival season? : false | Total Discount: 10.0%<br>Final Payable Amount: 7200.0<br><hr/> <b>Explanation:</b><br>Membership 10% only                                                                    |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate the final fee to be paid for a training workshop based on participant category and early registration status and fee.

The program should:

- Input the workshop fee.
- Input participant category (1 = Student, 2 = Professional, 3 = Corporate).
- Input whether the participant registered early (true/false).

Discount rules:

- Student: 15% discount
- Professional: 8% discount
- Corporate: 5% discount
- If registered early, add an extra 6% discount.
- If the workshop fee is more than 25,000, add an additional 4% discount.

Display the total discount percentage and the final payable amount.

| Sample Input                                                                                       | Output                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter workshop fee: 30000<br>Enter participant category: 1<br>Was it an early registration? : true | Total Discount: 25.0%<br>Final Amount to be Paid: 22500.0<br><hr/> Explanation:<br>Student (15%) + Early registration (6%) + Fee > 25000 (4%)<br>Total discount = 25% |
| Enter workshop fee: 18000<br>Enter participant category: 2<br>Was it an early registration?: false | Total Discount: 8.0%<br>Final Amount to be Paid: 16560.0<br><hr/> Explanation:<br>Professional (8%) only                                                              |

# Solution & Rubric

## SET - [A]

## SOLUTION

```
import java.util.Scanner;
public class ShoppingDiscount {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total purchase amount: ");
        double amount = sc.nextDouble();

        System.out.print("Enter membership type: ");
        int membership = sc.nextInt();

        System.out.print("Is it festival season?: ");
        boolean festival = sc.nextBoolean();

        double discount = 0.0;

        // Membership discount
        if (membership == 3) {           // Gold
            discount = discount + 20;
        }
        else if (membership == 2) {      // Silver
            discount = discount + 10;
        }
        else if (membership == 1) {      // Regular
            discount = discount + 5;
        }

        // Festival discount
        if (festival == true) {
            discount = discount + 5;
        }

        // Purchase above 10,000 discount
        if (amount > 10000) {
            discount = discount + 7;
        }

        // Calculate final amount
        double finalAmount = amount - (amount * discount / 100);

        System.out.println("Total Discount: " + discount + "%");
        System.out.println("Final Payable Amount: " + finalAmount);
    }
}
```

## RUBRIC

| SL | Points to Meet                                                                                                                                                                                                                         | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1  | Correct Input Handling (Scanner import, correct datatype)                                                                                                                                                                              | 2             |
| 2  | Membership Discount Logic <ul style="list-style-type: none"> <li>• Correct use of if-else if structure</li> <li>• Correct percentage assigned for each membership type</li> <li>• Adding the applied discount in a variable</li> </ul> | 3             |
| 3  | Additional Discount Conditions (Festival and workshop amount threshold) <ul style="list-style-type: none"> <li>• Correct if-else logic</li> <li>• Adding the applied discounts in a variable</li> </ul>                                | 1+1           |
| 4  | Final Amount Calculation                                                                                                                                                                                                               | 2             |
| 5  | Output and Code Structure                                                                                                                                                                                                              | 1             |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;
public class WorkshopFeeCalculator {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter workshop fee: ");
        double fee = sc.nextDouble();

        System.out.print("Enter participant category: ");
        int category = sc.nextInt();

        System.out.print("Was it an early registration?: ");
        boolean early = sc.nextBoolean();

        double discount = 0.0;

        // Category discount
        if (category == 1) {           // Student
            discount = discount + 15;
        }
        else if (category == 2) {      // Professional
            discount = discount + 8;
        }
        else if (category == 3) {      // Corporate
            discount = discount + 5;
        }

        // Early registration discount
        if (early == true) {
            discount = discount + 6;
        }

        // High fee discount
        if (fee > 25000) {
            discount = discount + 4;
        }

        // Calculate final fee
        double finalAmount = fee - (fee * discount / 100);

        System.out.println("Total Discount: " + discount + "%");
        System.out.println("Final Amount to be Paid: " + finalAmount);
    }
}
```

| RUBRIC |                                                                                                                                                                                                                                    |               |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                                                                                                                                                                                     | Marks<br>[10] |
| 1      | Correct Input Handling (Scanner import, correct datatype)                                                                                                                                                                          | 2             |
| 2      | Category Discount Logic <ul style="list-style-type: none"> <li>• Correct use of if-else if structure</li> <li>• Correct percentage assigned for each category type</li> <li>• Adding the applied discount in a variable</li> </ul> | 3             |
| 3      | Additional Discount Conditions (Registration status and fee) <ul style="list-style-type: none"> <li>• Correct if-else logic</li> <li>• Adding the applied discounts in a variable</li> </ul>                                       | 1+1           |
| 4      | Final Amount Calculation                                                                                                                                                                                                           | 2             |
| 5      | Output and Code Structure                                                                                                                                                                                                          | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A robotics lab is testing **N mini robots over R rounds**. In each round, each robot sends a **two-digit signal code**. From each **valid code**, the **bigger digit** is the **power level**, and the **smaller digit** is the **stability level**. If the **difference between them is 5 or more**, the robot is **unstable**. Otherwise, it is **stable**. You are required to write a Java program that **takes R and N as input**, followed by the signal code of each robot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 3** and the **smaller digit is divisible by 7**, the code is **invalid**, and **input must be taken again** for that robot in that round. After each round, print the number of **stable and unstable robots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                                                | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds (R) = 2<br>Robots (N) = 3<br>Round 1:<br>Robot 1 code = 42<br>Robot 2 code = 97<br>Robot 2 code = 81<br>Robot 3 code = 65<br>Round 2:<br>Robot 1 code = 54<br>Robot 2 code = 20<br>Robot 3 code = 31 | Round 1:<br>Stable robots: 2<br>Unstable robots: 1<br>Round 2:<br>Stable robots: 3<br>Unstable robots: 0 | Round 1:<br>42 → bigger = 4, smaller = 2 → difference = 2 → Stable<br>97 → bigger = 9, smaller = 7 → bigger divisible by 3 and smaller divisible by 7 → Invalid<br>81 → taken again for robot 2 → bigger = 8, smaller = 1 → difference = 7 → Unstable<br>65 → bigger = 6, smaller = 5 → difference = 1 → Stable<br>So, in Round 1:<br>Stable robots = 2<br>Unstable robots = 1 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A game company is testing  $N$  mini game bots over  $R$  rounds. In each round, each bot sends a **two-digit performance code**. From each **valid code**, the **bigger digit** is the **attack level**, and the **smaller digit** is the **defense level**. If the **difference** between them is **less than 4**, the bot is **balanced**. Otherwise, it is **imbalanced**. You are required to write a Java program that takes  $R$  and  $N$  as **input**, followed by the **performance code** of each bot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 5** and the **smaller digit is divisible by 3**, the code is **invalid**, and **input must be taken again** for that bot in that round. After each round, print the number of **balanced and imbalanced bots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                              | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds (R)= 2<br>Bots (N)= 3<br>Round 1:<br>Bot 1 code = 39<br>Bot 2 code = 11<br>Bot 3 Code = 81<br>Round 2:<br>Bot 1 code = 42<br>Bot 2 code = 53<br>Bot 2 code = 87<br>Bot 3 Code = 64 | Round 1:<br>Balanced bots: 1<br>Imbalanced bots: 2<br>Round 2:<br>Balanced bots: 3<br>Imbalanced bots: 0 | Round 2:<br>42 → bigger = 4, smaller = 2 → difference = 2 → Balanced<br>53 → bigger = 5, smaller = 3 → bigger is divisible by 5 and smaller is divisible by 3 → Invalid<br>87 → taken again for bot 2 → bigger = 8, smaller = 7 → difference = 1 → Balanced<br>64 → bigger = 6, smaller = 4 → difference = 2 → Balanced<br>So, in Round 2:<br>Balanced bots = 3<br>Imbalanced bots = 0 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A cybersecurity team is checking encrypted message logs to identify hidden access codes. A valid code **starts with an uppercase letter (A-Z)** and **ends with an odd digit (0-9)**, including all characters in between. Once a valid code is found, **it is separated by #**, and the search for the next valid code must continue from the next character. Write a Java program that takes the encrypted message as input from the user, extracts all valid codes according to the rules, and displays them. If no valid code is found, "No valid code found." is displayed.

**You are only allowed to use `.charAt()` and `.length()` as built-in methods.**

| Sample Input    | Sample Output                                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| abAxy3mnBz2pqC1 | Axy3#C1<br><br><b>Explanation</b><br>Axy3 → starts with A, ends with an odd digit 3 (valid)<br>Bz2 → starts with B, ends with an even digit 2 (invalid)<br>C1 → starts with C, ends with an odd digit 1 (valid)<br>Both valid codes are separated by #. |
| helloXabcYz     | No valid code found.                                                                                                                                                                                                                                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A research team is scanning DNA samples to identify hidden sample labels. A valid label **starts with an even digit (0-9)** and **ends with a lowercase letter (a-z)**, including all characters in between. Once a valid label is found, **it is separated by @**, and the search for the next valid code must continue from the next character. Write a Java program that takes a DNA sequence as input from the user, extracts all valid labels according to the rules, and displays them. If no valid label is found, "No valid label found." is displayed.

**You are only allowed to use .charAt() and .length() as built-in methods.**

| Sample Input    | Sample Output                                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2ADxBc5mnDe6qfs | 2ADx@6q<br><br><b>Explanation</b><br>2ADx → starts with an even digit 2, ends with x (valid)<br>5m → starts with an odd digit 5, ends with m (invalid)<br>6q → starts with an even digit 6, ends with q (valid)<br>Both valid codes are separated by @. |
| 135BCD79EFG     | No valid label found.                                                                                                                                                                                                                                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **ascending order** and elements at **odd indices** are in **descending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                              | Expected Output                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>int [] marks = {9, -5, 7, 9, -5, 5};</code>                                                                                                                                                  | Unique count: 4<br>Sorted array: 7 5 9 -5 |
| <b>Explanation:</b><br>The unique elements are 9, -5, 7, and 5, so the unique count is 4.<br>Even indices are sorted in ascending order: 7, 9<br>Odd indices are sorted in descending order: 5, -5 |                                           |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **descending order** and elements at **odd indices** are in **ascending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                            | Expected Output                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <code>int [] scores= {4, 2, 6, 1, 4, 1};</code>                                                                                                                                                  | Unique count: 4<br>Sorted array: 6 1 4 2 |
| <b>Explanation:</b><br>The unique elements are 4, 2, 6, and 1, so the unique count is 4.<br>Even indices are sorted in descending order: 6, 4<br>Odd indices are sorted in ascending order: 1, 2 |                                          |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

An online traditional clothing store is offering special Eid discounts. Write a Java program that asks the user to enter the cloth type (1 for Panjabi at Tk. 2100 per piece, 2 for Salwar Kameez at Tk. 2500 per piece, 3 for Sari at Tk. 4000 per piece), the quantity of items, and the 6-digit membership card number. The program must check that the cloth type is valid and that the quantity is greater than 0. Otherwise, it should display **"Invalid cloth type or quantity."** If the inputs are valid, the program should calculate the total amount and apply an Eid discount: **10% for a total amount between Tk. 5000 and Tk. 10000**, and **15% for a total amount above Tk. 10000**. If the updated price **exceeds Tk. 2000** after the standard discount and the **last 2-digit of the membership card is even**, an **additional 5% Eid discount** should be applied. However, if this 5% discount causes the final price to **fall below Tk. 4300**, then the discount for the membership card number is removed. Finally, the program should display the final payable amount after all applicable discounts.

| Sample Input                                                                                                                       | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 2<br>Enter quantity: 2<br>Enter membership card number (6-digit): 125468 | Total payable amount: Tk. 4500.0<br><br><b>Explanation:</b><br>The user buys 2 Salwar Kameez for Tk. 5000 total, which gets a 10% discount, reducing the price to Tk. 4500. Since Tk. 4500 is over Tk. 2000 and the last 2-digit of the membership card is even, the 5% Eid discount is applied, dropping the price to Tk. 4275. However, this falls below the Tk. 4300 limit and the 5% discount is removed. |
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 1<br>Enter quantity: 3<br>Enter membership card number (6-digit): 125468 | Total payable amount: Tk. 5386.5<br><br><b>Explanation:</b><br>The user buys 3 Panjabis for a total of Tk. 6300, which gets a 10% discount, reducing the price to Tk. 5670. Since Tk. 5670 is over Tk. 2000 and the last 2-digit of the membership card is even, the 5% Eid discount is applied, dropping the price to Tk. 5386.5.                                                                            |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

An online traditional clothing store is offering special Eid reward points. Write a Java program that asks the user to enter the cloth type (1 for Panjabi at Tk. 2100 per piece, 2 for Salwar Kameez at Tk. 2500 per piece, 3 for Sari at Tk. 4000 per piece), the quantity, and the phone number. The program must check that the cloth type is valid and the quantity is greater than 0. Otherwise, it should display **"Invalid cloth type or quantity."** If the inputs are valid, the program should calculate the initial total amount and determine the Eid reward points as follows: the user earns **2 points for every full Tk. 1000** of the initial purchase if the **last 2-digit of the phone number is odd**. Otherwise, the user earns **5 points** if the initial purchase is between **Tk. 5000 and Tk. 10000**. Each reward point is worth **Tk. 50**. However, the reward point discount is **only** adjusted from the bill if the initial total amount exceeds **Tk. 5000**. Finally, the program should display the final payable amount after any applicable point adjustments.

| Sample Input                                                                                                        | Sample Output                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 2<br>Enter quantity: 2<br>Enter phone number: 01234567862 | Total payable amount: Tk. 5000.0<br><br><b>Explanation:</b><br>The user buys 2 Salwar Kameez for a total of Tk. 5000. The last 2-digit of the phone number is even, so they earn 5 points, which has a value of Tk.250. However, the discount is only applied if the total exceeds Tk. 5000. Since the total does not exceed Tk. 5000, no discount is applied. |
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 1<br>Enter quantity: 3<br>Enter phone number: 01234567891 | Total payable amount: Tk. 5670.0<br><br><b>Explanation:</b><br>The user buys 3 Panjabis for a total of Tk. 6300. The last 2-digit of the phone number is odd, so they earn 12.6 points (6300*2/1000), which has a value of Tk. 630. Since the total exceeds Tk. 5000, the discount is applied, resulting in a final payable amount of Tk. 5670.0               |

A

**CSE110 - Quiz 4 (A) (Time: 35 Minutes)****Name:****ID:****Section:**

You are given an integer array. You have to **update the given array [don't create a new one]** by reversing the digits of each element in it. You also need to create a boolean array that stores true if an element is a palindrome after reversal; otherwise, it stores false. Finally, print both the updated integer array and the newly created boolean array. **You are not allowed to convert numbers into strings, and you must not use any built-in methods for reversing or checking the numbers. Your code should work for any given integer array of any length.** [Marks: 10]

| Given Array                                                | Sample Output                                                                                       |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>int [ ] arr = {31213, 465, 23, 5, 545, 9084};</code> | <code>[31213, 564, 32, 5, 545, 4809]</code><br><code>[true, false, false, true, true, false]</code> |
| <code>int [ ] arr = {452, -34, 65121, 0, 454};</code>      | <code>[254, -43, 12156, 0, 454]</code><br><code>[false, false, false, true, true]</code>            |



B

**CSE110 - Quiz 4 (B) (Time: 35 Minutes)****Name:****ID:****Section:**

You are given an integer array. You have to **update the given array [don't create a new one]** by reversing the digits of each element in it. You also need to create a boolean array that stores true if the reversed value of an element is greater than the original element; otherwise, it stores false. Finally, print both the updated integer array and the newly created boolean array. **You are not allowed to convert numbers into strings, and you must not use any built-in methods for reversing or checking the numbers. Your code should work for any given integer array of any length.** [Marks: 12]

| Given Array                                               | Sample Output                                                                                       |
|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>int [ ] arr = {452, -34, 15126, 0, 45};</code>      | <code>[254, -43, 62151, 0, 54]</code><br><code>[false, false, true, false, true]</code>             |
| <code>int [ ] arr = {3121, 465, 23, 5, 545, 9084};</code> | <code>[1213, 564, 32, 5, 545, 4809]</code><br><code>[false, true, true, false, false, false]</code> |

Trace the code below and write the outputs in the provided boxes. Your outputs won't be accepted without the workings. [Marks: 8]

```

1 public class B{
2     public static void main(String [] args){
3         System.out.println("Start");
4         char [] arr = {'A', 'B', 'C'};
5         int s = metA(arr);
6         System.out.println(s);
7         System.out.println("End");
8         System.out.println(arr[0]+10);
9     }
10    public static int [] metB(int n, int m){
11        System.out.println(n+m);
12        int [] arr = {65, 66, 67};
13        System.out.println(arr[0]%10);
14        return arr;
15    }
16    public static int metA(char [] s){
17        int m = 34, n = 23;
18        int [] arr = metB(m++, ++n);
19        if((int)s[0] == arr[0]){
20            System.out.println(true);
21            System.out.println(m);
22        }
23        s[0] = "01".charAt(1);
24        return s[0]+s[1];
25    }
26 }

```

### Outputs:

[illegible]

# Solution

SET A:

```
import java.util.Arrays;
public class Q4A{
    public static void main(String [] args){
        //int [] arr = {31213, 465, 23, 5, 545, 9084};
        int [] arr = {452, -34, 65121, 0, 454};
        boolean [] ans = new boolean[arr.length];
        for(int i = 0; i<arr.length; i++){
            int num = arr[i];
            int temp = arr[i];
            int revNum = 0;
            while(num!=0){
                int digit = num%10;
                revNum = revNum*10 + digit;
                num/=10;
            }
            arr[i] = revNum;
            if(temp == arr[i]){
                ans[i] = true;
            }
        }
        System.out.println(Arrays.toString(arr));
        System.out.println(Arrays.toString(ans));
    }
}
```

**Begin**  
**68**  
**8**  
**false**  
**14**  
**146**  
**Finish**  
**63**

SET B:

```
import java.util.Arrays;
public class Q4B{
    public static void main(String [] args){
        int [] arr = {3121, 465, 23, 5, 545, 9084};
        //int [] arr = {452, -34, 15126, 0, 45};
        boolean [] ans = new boolean[arr.length];
        for(int i = 0; i<arr.length; i++){
            int num = arr[i];
            int temp = arr[i];
            int revNum = 0;
            while(num!=0){
                int digit = num%10;
                revNum = revNum*10 + digit;
                num/=10;
            }
            arr[i] = revNum;
            if(arr[i] > temp){
                ans[i] = true;
            }
        }
        System.out.println(Arrays.toString(arr));
        System.out.println(Arrays.toString(ans));
    }
}
```

**Start**

**58**

**5**

**true**

**35**

**115**

**End**

**59**

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

In the futuristic city of **Energon Prime**, engineers have discovered a powerful **P-digit** Energy Code(Number) that controls the entire city's power grid. This code must be carefully analyzed to unlock its hidden energy potential. The system responds only to a **special digit N** embedded within the code.

As you examine the Energy Code from Right to Left:

- If the digit N is not found anywhere in the number, then the system shuts down and no energy is generated.
- If the digit N is found, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as  $\text{digit} \times (\text{frequency})$ . All nonzero contributions are multiplied to get the **final energy**.

Write a Java program to print whether N was found and the resulting energy value, or indicate that no energy is generated.

| Sample Input 1 | Sample Output 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 122331<br>3    | Digit 0 -> Frequency: 0, Contribution: 0<br>Digit 1 -> Frequency: 2, Contribution: 2<br>Digit 2 -> Frequency: 2, Contribution: 4<br>Digit 3 -> Frequency: 2, Contribution: 6<br>Digit 4 -> Frequency: 0, Contribution: 0<br>Digit 5 -> Frequency: 0, Contribution: 0<br>Digit 6 -> Frequency: 0, Contribution: 0<br>Digit 7 -> Frequency: 0, Contribution: 0<br>Digit 8 -> Frequency: 0, Contribution: 0<br>Digit 9 -> Frequency: 0, Contribution: 0<br>Total Energy: 48<br><u>Explanation:</u><br>Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times<br>$(1 \times 2) \times (2 \times 2) \times (3 \times 2)$<br>= 48 |
| Sample Input 2 | Sample Output 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 456789<br>2    | 2 NOT FOUND<br>NO ENERGY GENERATED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer  $d$ , where  $0 \leq d < n$  and  $n$  is the length of the string. The program should perform a **left rotation** on the given string by  $d$  positions.

In a left rotation:

- The first  **$d$  characters** of the string are moved to the end.
- The remaining characters are shifted to the left.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

| Sample Input:                                                                                                                  | Sample Output:                                |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Enter string: ABCD<br>Enter d: 1<br>Enter string: ABCD<br>Enter d: 2<br>Enter string: HELLO<br>Enter d: 3<br>Enter string: END | BCDA<br>CDAB<br>LOHEL<br>The program stopped. |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **Friends**. A user is considered a **true Friends binge watcher** only if they watched episodes for at least **3 consecutive days** where the number of episodes watched each day was **strictly increasing**. Among all such increasing streaks, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *Friends* a user watched each day. Write a Java program that prints "True Binge Watcher" if the selected streak has a total of **10 or more episodes**; otherwise print "Casual Watcher".

|                                                                                             |                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sample Input 1:</b><br><br>Given Array:<br><pre>int [] watch = {2, 3, 5, 6, 1, 2};</pre> | <b>Sample Output 1:</b><br><br>True Binge Watcher<br><b>Explanation:</b><br>Increasing streaks:<br>2, 3, 5, 6 → length = 4, sum = 16<br>1, 2 → length = 2 (ignored)<br>Selected streak = 2, 3, 5, 6<br>sum = 16 >= 10 |
| <b>Sample Input 2:</b><br><br>Given Array:<br><pre>int [] watch = {1, 2, 3, 0, 1};</pre>    | <b>Sample Output 2:</b><br><br>Casual Watcher<br><b>Explanation:</b><br>Increasing streaks:<br>1, 2, 3 → length = 3, sum = 6<br>0, 1 → length = 2 (ignored)<br>Selected streak = 1, 2, 3<br>sum = 6 < 10              |

BRAC UNIVERSITY

Department of Computer Science and Engineering

SET: A

Course: CSE110

Examination: Quiz 1

Semester: Spring 2026

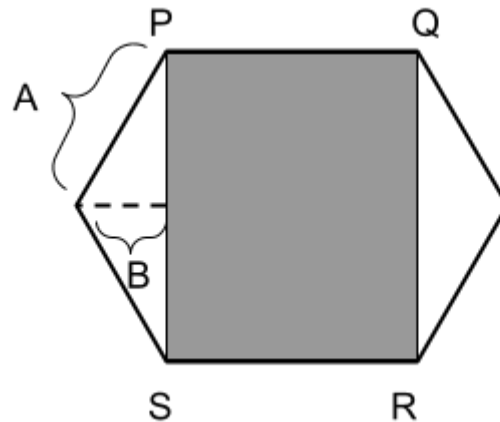
Name:

Duration: 15 min

Marks: 10

ID:

1.



Consider a regular **Hexagon** where each of the six sides is of equal length. You take two inputs **A & B** from the user. Your task is to **draw a flowchart** to calculate and print the **area of the Rectangle PQRS**.

[To find square root use `Math.sqrt(number)`]

**BRAC UNIVERSITY**

**Department of Computer Science and Engineering**

**SET: B**

Course: CSE110

Examination: Quiz 1

Semester: Spring 2026

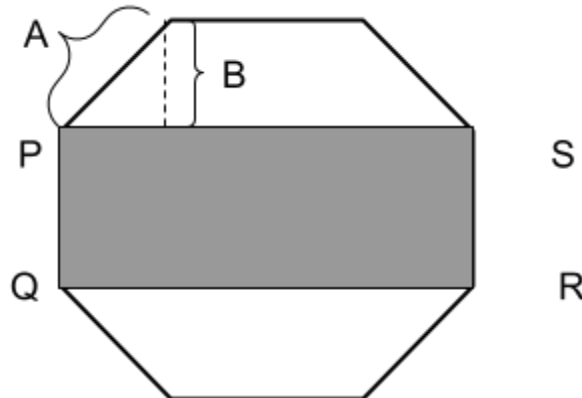
**Name:**

Duration: 15 min

Marks: 10

**ID:**

1.



Consider a regular **Octagon** where each of the eight sides is of equal length. You take two inputs **A & B** from the user. Your task is to **draw a flowchart** to calculate and print the **area of the Rectangle PQRS**.

[To find square root use `Math.sqrt(number)`]

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A robotics lab is testing  $N$  mini robots over  $R$  rounds. In each round, each robot sends a **two-digit signal code**. From each **valid code**, the **bigger digit** is the **power level**, and the **smaller digit** is the **stability level**. If the **difference between them is 5 or more**, the robot is **unstable**. Otherwise, it is **stable**. You are required to write a Java program that **takes  $R$  and  $N$  as input**, followed by the signal code of each robot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 3** and the **smaller digit is divisible by 7**, the code is **invalid**, and **input must be taken again** for that robot in that round. After each round, print the number of **stable and unstable robots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                                                        | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds ( $R$ ) = 2<br>Robots ( $N$ ) = 3<br>Round 1:<br>Robot 1 code = 42<br>Robot 2 code = 97<br>Robot 2 code = 81<br>Robot 3 code = 65<br>Round 2:<br>Robot 1 code = 54<br>Robot 2 code = 20<br>Robot 3 code = 31 | Round 1:<br>Stable robots: 2<br>Unstable robots: 1<br>Round 2:<br>Stable robots: 3<br>Unstable robots: 0 | Round 1:<br>42 $\rightarrow$ bigger = 4, smaller = 2 $\rightarrow$ difference = 2 $\rightarrow$ Stable<br>97 $\rightarrow$ bigger = 9, smaller = 7 $\rightarrow$ bigger divisible by 3 and smaller divisible by 7 $\rightarrow$ Invalid<br>81 $\rightarrow$ taken again for robot 2 $\rightarrow$ bigger = 8, smaller = 1 $\rightarrow$ difference = 7 $\rightarrow$ Unstable<br>65 $\rightarrow$ bigger = 6, smaller = 5 $\rightarrow$ difference = 1 $\rightarrow$ Stable<br>So, in Round 1:<br>Stable robots = 2<br>Unstable robots = 1 |

## SET - [A]

## SOLUTION

```

import java.util.Scanner;
public class LabExam3A {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String message = sc.nextLine();
        String result = "";
        int start = -1;
        for (int i = 0; i < message.length(); i++) {
            char ch = message.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                start = i;
            }
            if (start != -1 && (ch=='1' || ch=='3' || ch=='5' || ch=='7' || ch=='9')) {
                String code = "";
                for (int k = start; k <= i; k++) {
                    code = code + message.charAt(k);
                }
                if (result.length() > 0) {
                    result = result + "#";
                }
                result = result + code;
                start = -1;
            }
        }
        if (result.length() == 0) {
            System.out.println("No valid code found.");
        } else {
            System.out.println(result);
        }
    }
}

```

## SET - [B]

## SOLUTION

```
import java.util.Scanner;
public class DNALabelExtractor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sequence = sc.nextLine();
        String result = "";
        int start = -1;
        for (int i = 0; i < sequence.length(); i++) {
            char ch = sequence.charAt(i);
            if (ch == '0' || ch == '2' || ch == '4' || ch == '6' || ch == '8') {
                start = i;
            }
            if (start != -1 && ch >= 'a' && ch <= 'z') {
                String label = "";
                for (int k = start; k <= i; k++) {
                    label = label + sequence.charAt(k);
                }
                if (result.length() > 0) {
                    result = result + "@";
                }
                result = result + label;
                start = -1;
            }
        }
        if (result.length() == 0) {
            System.out.println("No valid label found.");
        } else {
            System.out.println(result);
        }
    }
}
```

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **ascending order** and elements at **odd indices** are in **descending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                              | Expected Output                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>int [] marks = {9, -5, 7, 9, -5, 5};</code>                                                                                                                                                  | Unique count: 4<br>Sorted array: 7 5 9 -5 |
| <b>Explanation:</b><br>The unique elements are 9, -5, 7, and 5, so the unique count is 4.<br>Even indices are sorted in ascending order: 7, 9<br>Odd indices are sorted in descending order: 5, -5 |                                           |

SET-A

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

An online traditional clothing store is offering special Eid discounts. Write a Java program that asks the user to enter the cloth type (1 for Panjabi at Tk. 2100 per piece, 2 for Salwar Kameez at Tk. 2500 per piece, 3 for Sari at Tk. 4000 per piece), the quantity of items, and the 6-digit membership card number. The program must check that the cloth type is valid and that the quantity is greater than 0. Otherwise, it should display **"Invalid cloth type or quantity."** If the inputs are valid, the program should calculate the total amount and apply an Eid discount: **10% for a total amount between Tk. 5000 and Tk. 10000**, and **15% for a total amount above Tk. 10000**. If the updated price **exceeds Tk. 2000** after the standard discount and the **last 2-digit of the membership card is even**, an **additional 5% Eid discount** should be applied. However, if this 5% discount causes the final price to **fall below Tk. 4300**, then the discount for the membership card number is removed. Finally, the program should display the final payable amount after all applicable discounts.

| Sample Input                                                                                                                       | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 2<br>Enter quantity: 2<br>Enter membership card number (6-digit): 125468 | Total payable amount: Tk. 4500.0<br><br><b>Explanation:</b><br>The user buys 2 Salwar Kameez for Tk. 5000 total, which gets a 10% discount, reducing the price to Tk. 4500. Since Tk. 4500 is over Tk. 2000 and the last 2-digit of the membership card is even, the 5% Eid discount is applied, dropping the price to Tk. 4275. However, this falls below the Tk. 4300 limit and the 5% discount is removed. |
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 1<br>Enter quantity: 3<br>Enter membership card number (6-digit): 125468 | Total payable amount: Tk. 5386.5<br><br><b>Explanation:</b><br>The user buys 3 Panjabis for a total of Tk. 6300, which gets a 10% discount, reducing the price to Tk. 5670. Since Tk. 5670 is over Tk. 2000 and the last 2-digit of the membership card is even, the 5% Eid discount is applied, dropping the price to Tk. 5386.5.                                                                            |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

An online traditional clothing store is offering special Eid reward points. Write a Java program that asks the user to enter the cloth type (1 for Panjabi at Tk. 2100 per piece, 2 for Salwar Kameez at Tk. 2500 per piece, 3 for Sari at Tk. 4000 per piece), the quantity, and the phone number. The program must check that the cloth type is valid and the quantity is greater than 0. Otherwise, it should display **"Invalid cloth type or quantity."** If the inputs are valid, the program should calculate the initial total amount and determine the Eid reward points as follows: the user earns **2 points for every full Tk. 1000** of the initial purchase if the **last 2-digit of the phone number is odd**. Otherwise, the user earns **5 points** if the initial purchase is between **Tk. 5000 and Tk. 10000**. Each reward point is worth **Tk. 50**. However, the reward point discount is **only** adjusted from the bill if the initial total amount exceeds **Tk. 5000**. Finally, the program should display the final payable amount after any applicable point adjustments.

| Sample Input                                                                                                        | Sample Output                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 2<br>Enter quantity: 2<br>Enter phone number: 01234567862 | Total payable amount: Tk. 5000.0<br><br><b>Explanation:</b><br>The user buys 2 Salwar Kameez for a total of Tk. 5000. The last 2-digit of the phone number is even, so they earn 5 points, which has a value of Tk.250. However, the discount is only applied if the total exceeds Tk. 5000. Since the total does not exceed Tk. 5000, no discount is applied. |
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 1<br>Enter quantity: 3<br>Enter phone number: 01234567891 | Total payable amount: Tk. 5670.0<br><br><b>Explanation:</b><br>The user buys 3 Panjabis for a total of Tk. 6300. The last 2-digit of the phone number is odd, so they earn 12.6 points (6300*2/1000), which has a value of Tk. 630. Since the total exceeds Tk. 5000, the discount is applied, resulting in a final payable amount of Tk. 5670.0               |

# Solution & Rubric

## SET - [A]

## SOLUTION

```
import java.util.Scanner;
public class LabExam1SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): ");
        int clothType = sc.nextInt();
        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();
        System.out.print("Enter membership card number (6-digit): ");
        int cardNo = sc.nextInt();

        if ((clothType == 1 || clothType == 2 || clothType == 3) && quantity > 0) {

            double totalAmount = 0;
            if (clothType == 1) {
                totalAmount = 2100 * quantity;
            } else if (clothType == 2) {
                totalAmount = 2500 * quantity;
            } else if (clothType == 3) {
                totalAmount = 4000 * quantity;
            }

            if (totalAmount > 10000) {
                totalAmount = totalAmount - (totalAmount * 0.15);
            } else if (totalAmount >= 5000 && totalAmount <= 10000) {
                totalAmount = totalAmount - (totalAmount * 0.10);
            }

            if (totalAmount > 2000 && (cardNo%100)%2 == 0) {
                double tempAmount = totalAmount - (totalAmount * 0.05);
                if (tempAmount >= 4300) {
                    totalAmount = tempAmount;
                }
            }

            System.out.println("\nTotal payable amount: Tk. " + totalAmount);

        } else {
            System.out.println("\nInvalid quantity or cloth type.");
        }
    }
}
```

# RUBRIC

| SL | Points to Meet                                                                                                                        | Marks<br>[10] |
|----|---------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1  | Correctly imports and initializes Scanner, and takes three inputs using appropriate data types                                        | 1             |
| 2  | Checks that cloth type is 1,2 or 3 and quantity is greater than 0                                                                     | 1             |
| 3  | Accurately determines the price per piece and calculates the initial amount                                                           | 1             |
| 4  | Correctly checks and calculates the 15% discount (for > 10000) and the 10% discount (for >= 5000 and <= 10000).                       | 2             |
| 5  | Checks the last two digits of the membership card to be an even and updated amount > 2000, then correctly calculates the 5% discount. | 2             |
| 6  | Implements the logic to apply or remove the premium membership discount based on the minimum limit of 4300.                           | 2             |
| 7  | Correctly prints the final payable amount and invalid inputs                                                                          | 1             |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;
public class LabExam1SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): ");
        int clothType = sc.nextInt();
        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();
        System.out.print("Enter phone number: ");
        int phoneNo = sc.nextInt();

        if ((clothType == 1 || clothType == 2 || clothType == 3) && quantity > 0) {

            double totalAmount = 0;
            if (clothType == 1) {
                totalAmount = 2100 * quantity;
            } else if (clothType == 2) {
                totalAmount = 2500 * quantity;
            } else if (clothType == 3) {
                totalAmount = 4000 * quantity;
            }

            double rewardPoints = 0;
            if ((phoneNo%100)%2 == 0) {
                if (totalAmount >= 5000 && totalAmount <= 10000) {
                    rewardPoints = 5;
                }
            } else {
                rewardPoints = (totalAmount / 1000) * 2;
            }
            if (totalAmount > 5000) {
                totalAmount = totalAmount - (rewardPoints * 50);
            }
            System.out.println("\nTotal payable amount: Tk. " + totalAmount);
        } else {
            System.out.println("\nInvalid cloth type or quantity.");
        }
    }
}
```

| RUBRIC |                                                                                                                                  |               |
|--------|----------------------------------------------------------------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                                                                                   | Marks<br>[10] |
| 1      | Correctly imports and initializes Scanner, and takes three inputs using appropriate data types                                   | 1             |
| 2      | Checks that cloth type is 1,2 or 3 and quantity is greater than 0                                                                | 1             |
| 3      | Accurately determines the price per piece and calculates the initial amount                                                      | 1.5           |
| 4      | Correctly checks for odd last 2-digit of phone number.                                                                           | 1.5           |
| 5      | Correctly implements the reward point logic (2 points for every Tk. 1000 for odd last 2-digit phone number; otherwise, 5 points) | 2             |
| 6      | Implements the logic to apply the point deduction (points x 50) only if the total amount exceeds 5000.                           | 2             |
| 7      | Correctly prints the final payable amount and invalid inputs                                                                     | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A robotics lab is testing  $N$  mini robots over  $R$  rounds. In each round, each robot sends a **two-digit signal code**. From each **valid code**, the **bigger digit** is the **power level**, and the **smaller digit** is the **stability level**. If the **difference between them is 5 or more**, the robot is **unstable**. Otherwise, it is **stable**. You are required to write a Java program that **takes  $R$  and  $N$  as input**, followed by the signal code of each robot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 3** and the **smaller digit is divisible by 7**, the code is **invalid**, and **input must be taken again** for that robot in that round. After each round, print the number of **stable and unstable robots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                                                        | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds ( $R$ ) = 2<br>Robots ( $N$ ) = 3<br>Round 1:<br>Robot 1 code = 42<br>Robot 2 code = 97<br>Robot 2 code = 81<br>Robot 3 code = 65<br>Round 2:<br>Robot 1 code = 54<br>Robot 2 code = 20<br>Robot 3 code = 31 | Round 1:<br>Stable robots: 2<br>Unstable robots: 1<br>Round 2:<br>Stable robots: 3<br>Unstable robots: 0 | Round 1:<br>42 $\rightarrow$ bigger = 4, smaller = 2 $\rightarrow$ difference = 2 $\rightarrow$ Stable<br>97 $\rightarrow$ bigger = 9, smaller = 7 $\rightarrow$ bigger divisible by 3 and smaller divisible by 7 $\rightarrow$ Invalid<br>81 $\rightarrow$ taken again for robot 2 $\rightarrow$ bigger = 8, smaller = 1 $\rightarrow$ difference = 7 $\rightarrow$ Unstable<br>65 $\rightarrow$ bigger = 6, smaller = 5 $\rightarrow$ difference = 1 $\rightarrow$ Stable<br>So, in Round 1:<br>Stable robots = 2<br>Unstable robots = 1 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

A game company is testing  $N$  mini game bots over  $R$  rounds. In each round, each bot sends a **two-digit performance code**. From each **valid code**, the **bigger digit** is the **attack level**, and the **smaller digit** is the **defense level**. If the **difference** between them is **less than 4**, the bot is **balanced**. Otherwise, it is **imbalanced**. You are required to write a Java program that takes  $R$  and  $N$  as **input**, followed by the **performance code** of each bot in every round. For each code, **extract the two digits** and determine the **bigger and smaller digit**. If the **bigger digit is divisible by 5** and the **smaller digit is divisible by 3**, the code is **invalid**, and **input must be taken again** for that bot in that round. After each round, print the number of **balanced and imbalanced bots**.

[Assume all inputs are valid two-digit integers from 10 to 99.]

| Sample Input                                                                                                                                                                              | Sample Output                                                                                            | Explanation                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rounds (R)= 2<br>Bots (N)= 3<br>Round 1:<br>Bot 1 code = 39<br>Bot 2 code = 11<br>Bot 3 Code = 81<br>Round 2:<br>Bot 1 code = 42<br>Bot 2 code = 53<br>Bot 2 code = 87<br>Bot 3 Code = 64 | Round 1:<br>Balanced bots: 1<br>Imbalanced bots: 2<br>Round 2:<br>Balanced bots: 3<br>Imbalanced bots: 0 | Round 2:<br>42 → bigger = 4, smaller = 2 → difference = 2 → Balanced<br>53 → bigger = 5, smaller = 3 → bigger is divisible by 5 and smaller is divisible by 3 → Invalid<br>87 → taken again for bot 2 → bigger = 8, smaller = 7 → difference = 1 → Balanced<br>64 → bigger = 6, smaller = 4 → difference = 2 → Balanced<br>So, in Round 2:<br>Balanced bots = 3<br>Imbalanced bots = 0 |

## SET - [A]

## SOLUTION

```

import java.util.Scanner;

public class setA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Rounds (R)= ");
        int R = sc.nextInt();
        System.out.print("Robots (N)= ");
        int N = sc.nextInt();

        for (int round = 1; round <= R; round++) {
            int stable = 0;
            int unstable = 0;

            System.out.println("Round " + round + ":");
            for (int robot = 1; robot <= N; robot++) {
                System.out.print("Robot "+robot+" code = ");
                int code = sc.nextInt();

                int d1 = code / 10;
                int d2 = code % 10;

                int bigger, smaller;
                if (d1 > d2) {
                    bigger = d1;
                    smaller = d2;
                } else {
                    bigger = d2;
                    smaller = d1;
                }

                if (bigger % 3 == 0 && smaller % 7 == 0) {
                    robot--; // take input again for same robot
                    continue;
                }

                int diff = bigger - smaller;

                if (diff >= 5) {
                    unstable++;
                } else {
                    stable++;
                }
            }
            System.out.println("Round " + round + ":");
            System.out.println("Stable robots: " + stable);
            System.out.println("Unstable robots: " + unstable);
        }
    }
}

```

| RUBRIC |                                                                          |               |
|--------|--------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                           | Marks<br>[10] |
| 1      | Takes R and N as input correctly                                         | 1             |
| 2      | Uses 2 nested loops correctly for rounds and robots                      | 2             |
| 3      | Extracts the two digits correctly                                        | 1             |
| 4      | Finds bigger and smaller digits correctly                                | 1             |
| 5      | Checks invalid condition correctly (bigger % 3 == 0 && smaller % 7 == 0) | 1             |
| 6      | Retakes input for the same robot in the same round                       | 1             |
| 7      | Checks balanced/imbalanced condition correctly (difference >= 5)         | 1             |
| 8      | Counts stable and unstable robots correctly for each round               | 1             |
| 9      | Prints the correct round-wise output                                     | 1             |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;

public class SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Rounds (R) = ");
        int R = sc.nextInt();
        System.out.print("Bots (B) = ");
        int N = sc.nextInt();

        for (int round = 1; round <= R; round++) {
            int balanced = 0;
            int imbalanced = 0;

            System.out.println("Round "+round+":");
            for (int bot = 1; bot <= N; bot++) {
                System.out.print("Bot "+bot+" code = ");
                int code = sc.nextInt();

                int d1 = code / 10;
                int d2 = code % 10;

                int bigger, smaller;
                if (d1 > d2) {
                    bigger = d1;
                    smaller = d2;
                } else {
                    bigger = d2;
                    smaller = d1;
                }

                if (bigger % 5 == 0 && smaller % 3 == 0) {
                    bot--;
                    continue;
                }

                int diff = bigger - smaller;

                if (diff < 4) {
                    balanced++;
                } else {
                    imbalanced++;
                }
            }
            System.out.println("Round " + round + ":");
            System.out.println("Balanced bots: " + balanced);
            System.out.println("Imbalanced bots: " + imbalanced);
        }
    }
}
```

# RUBRIC

| SL | Points to Meet                                                           | Marks<br>[10] |
|----|--------------------------------------------------------------------------|---------------|
| 1  | Takes R and N as input correctly                                         | 1             |
| 2  | Uses 2 nested loops correctly for rounds and bots                        | 2             |
| 3  | Extracts the two digits correctly                                        | 1             |
| 4  | Finds bigger and smaller digits correctly                                | 1             |
| 5  | Checks invalid condition correctly (bigger % 5 == 0 && smaller % 3 == 0) | 1             |
| 6  | Retakes input for the same bot in the same round                         | 1             |
| 7  | Checks balanced/imbalanced condition correctly (difference < 4)          | 1             |
| 8  | Counts balanced and imbalanced bots correctly for each round             | 1             |
| 9  | Prints the correct round-wise output                                     | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A cybersecurity team is checking encrypted message logs to identify hidden access codes. A valid code **starts** with **an uppercase letter (A-Z)** and **ends** with **an odd digit (0-9)**, including all characters in between. Once a valid code is found, **it is separated by #**, and the search for the next valid code must continue from the next character. Write a Java program that takes the encrypted message as input from the user, extracts all valid codes according to the rules, and displays them. If no valid code is found, "No valid code found." is displayed.

**You are only allowed to use .charAt() and .length() as built-in methods.**

| Sample Input    | Sample Output                                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| abAxy3mnBz2pqC1 | Axy3#C1<br><br><b>Explanation</b><br>Axy3 → starts with A, ends with an odd digit 3 (valid)<br>Bz2 → starts with B, ends with an even digit 2 (invalid)<br>C1 → starts with C, ends with an odd digit 1 (valid)<br>Both valid codes are separated by #. |
| helloXabcYz     | No valid code found.                                                                                                                                                                                                                                    |

**SET-B**

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A research team is scanning DNA samples to identify hidden sample labels. A valid label **starts with an even digit (0-9)** and **ends with a lowercase letter (a-z)**, including all characters in between. Once a valid label is found, **it is separated by @**, and the search for the next valid code must continue from the next character. Write a Java program that takes a DNA sequence as input from the user, extracts all valid labels according to the rules, and displays them. If no valid label is found, "No valid label found." is displayed.

**You are only allowed to use .charAt() and .length() as built-in methods.**

| Sample Input    | Sample Output                                                                                                                                                                                                                                           |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2ADxBc5mnDe6qfs | 2ADx@6q<br><br><b>Explanation</b><br>2ADx → starts with an even digit 2, ends with x (valid)<br>5m → starts with an odd digit 5, ends with m (invalid)<br>6q → starts with an even digit 6, ends with q (valid)<br>Both valid codes are separated by @. |
| 135BCD79EFG     | No valid label found.                                                                                                                                                                                                                                   |

# Solution & Rubric

**SET - [A]****SOLUTION**

```

import java.util.Scanner;
public class LabExam3A {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String message = sc.nextLine();
        String result = "";
        int start = -1;
        for (int i = 0; i < message.length(); i++) {
            char ch = message.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                start = i;
            }
            if (start != -1 && (ch=='1' || ch=='3' || ch=='5' || ch=='7' || ch=='9')) {
                String code = "";
                for (int k = start; k <= i; k++) {
                    code = code + message.charAt(k);
                }
                if (result.length() > 0) {
                    result = result + "#";
                }
                result = result + code;
                start = -1;
            }
        }
        if (result.length() == 0) {
            System.out.println("No valid code found.");
        } else {
            System.out.println(result);
        }
    }
}

```

**RUBRIC**

| SL | Points to Meet                                                               | Marks [10] |
|----|------------------------------------------------------------------------------|------------|
| 1  | Correctly taking user input using Scanner                                    | 1          |
| 2  | Identifying the start (uppercase letter) and end (odd digit) of a valid code | 2.5        |
| 3  | Correctly extracting each valid code                                         | 3          |
| 4  | Appending # to separate each valid code                                      | 2          |
| 5  | Appropriate print statements for valid code and "No valid code found."       | 1.5        |

# SET - [B]

## SOLUTION

```
import java.util.Scanner;
public class DNALabelExtractor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String sequence = sc.nextLine();
        String result = "";
        int start = -1;
        for (int i = 0; i < sequence.length(); i++) {
            char ch = sequence.charAt(i);
            if (ch == '0' || ch == '2' || ch == '4' || ch == '6' || ch == '8') {
                start = i;
            }
            if (start != -1 && ch >= 'a' && ch <= 'z') {
                String label = "";
                for (int k = start; k <= i; k++) {
                    label = label + sequence.charAt(k);
                }
                if (result.length() > 0) {
                    result = result + "@";
                }
                result = result + label;
                start = -1;
            }
        }
        if (result.length() == 0) {
            System.out.println("No valid label found.");
        } else {
            System.out.println(result);
        }
    }
}
```

## RUBRIC

| SL | Points to Meet                                                                 | Marks [10] |
|----|--------------------------------------------------------------------------------|------------|
| 1  | Correctly taking user input using Scanner                                      | 1          |
| 2  | Identifying the start (even digit) and end (lowercase letter) of a valid label | 2.5        |
| 3  | Correctly extracting each valid label                                          | 3          |
| 4  | Appending @ to separate each valid label                                       | 2          |
| 5  | Appropriate print statements for valid label and "No valid label found."       | 1.5        |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **ascending order** and elements at **odd indices** are in **descending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                              | Expected Output                           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| int [] marks = {9, -5, 7, 9, -5, 5};                                                                                                                                                               | Unique count: 4<br>Sorted array: 7 5 9 -5 |
| <b>Explanation:</b><br>The unique elements are 9, -5, 7, and 5, so the unique count is 4.<br>Even indices are sorted in ascending order: 7, 9<br>Odd indices are sorted in descending order: 5, -5 |                                           |

**SET-B**

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

Write a Java program that counts the unique (distinct) elements in a given array and then creates a new array containing only those unique elements. Next, your program sorts the new array **in place** so that elements at **even indices** are in **descending order** and elements at **odd indices** are in **ascending order**. Finally, it displays the count of unique elements and the sorted array.

| Given                                                                                                                                                                                            | Expected Output                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <code>int [] scores= {4, 2, 6, 1, 4, 1};</code>                                                                                                                                                  | Unique count: 4<br>Sorted array: 6 1 4 2 |
| <b>Explanation:</b><br>The unique elements are 4, 2, 6, and 1, so the unique count is 4.<br>Even indices are sorted in descending order: 6, 4<br>Odd indices are sorted in ascending order: 1, 2 |                                          |

# Solution & Rubric

**SET - [A]**

**SOLUTION**

```
public class SetA {
    public static void main(String[] args) {

        int[] marks = {9, -5, 7, 9, -5, 5};
        int[] unique = new int[marks.length];
        int size = 0;

        for (int i = 0; i < marks.length; i++) {
            boolean isDuplicate = false;
            for (int j = 0; j < size; j++) {
                if (marks[i] == unique[j]) {
                    isDuplicate = true;
                    break;
                }
            }
            if (isDuplicate==false) {
                unique[size] = marks[i];
                size++;
            }
        }

        for (int i = 0; i < size; i += 2) {
            int minIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] < unique[minIndex]) {
                    minIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[minIndex];
            unique[minIndex] = tempVal;
        }

        for (int i = 1; i < size; i += 2) {
            int maxIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] > unique[maxIndex]) {
                    maxIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[maxIndex];
            unique[maxIndex] = tempVal;
        }

        System.out.println("Unique count: " + size);
        System.out.print("Sorted array: ");
        for (int i = 0; i < size; i++) {
            System.out.print(unique[i] + " ");
        }
    }
}
```

| RUBRIC |                                                            |               |
|--------|------------------------------------------------------------|---------------|
| S<br>L | Points to Meet                                             | Marks<br>[10] |
| 1      | Input Handling & Initialization                            | 1             |
| 2      | Counting unique elements and creating a final unique array | 3             |
| 3      | Ascending sorting in even places                           | 2.5           |
| 4      | Descending sorting in an odd place                         | 2.5           |
| 5      | Prints the unique count and sorted array                   | 1             |

**SET - [B]****SOLUTION**

```
public class SetB {
    public static void main(String[] args) {

        int[] scores = {4, 2, 6, 1, 4, 1};
        int[] unique = new int[scores.length];
        int size = 0;

        for (int i = 0; i < scores.length; i++) {
            boolean isDuplicate = false;
            for (int j = 0; j < size; j++) {
                if (scores[i] == unique[j]) {
                    isDuplicate = true;
                    break;
                }
            }
            if (isDuplicate==false) {
                unique[size] = scores[i];
                size++;
            }
        }

        for (int i = 0; i < size; i += 2) {
            int maxIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] > unique[maxIndex]) {
                    maxIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[maxIndex];
            unique[maxIndex] = tempVal;
        }
        for (int i = 1; i < size; i += 2) {
            int minIndex = i;
            for (int j = i + 2; j < size; j += 2) {
                if (unique[j] < unique[minIndex]) {
                    minIndex = j;
                }
            }
            int tempVal = unique[i];
            unique[i] = unique[minIndex];
            unique[minIndex] = tempVal;
        }

        System.out.println("Unique count: " + size);
        System.out.print("Sorted array: ");
        for (int i = 0; i < size; i++) {
            System.out.print(unique[i] + " ");
        }
    }
}
```

| RUBRIC |                                                          |               |
|--------|----------------------------------------------------------|---------------|
| SL     | Points to Meet                                           | Marks<br>[10] |
| 1      | Input Handling & Initialization                          | 1             |
| 2      | Counting unique elements and creating final unique array | 3             |
| 3      | Ascending sorting in even place                          | 2.5           |
| 4      | Descending sorting in odd place                          | 2.5           |
| 5      | Prints the unique count and sorted array                 | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

An online traditional clothing store is offering special Eid discounts. Write a Java program that asks the user to enter the cloth type (1 for Panjabi at Tk. 2100 per piece, 2 for Salwar Kameez at Tk. 2500 per piece, 3 for Sari at Tk. 4000 per piece), the quantity of items, and the 6-digit membership card number. The program must check that the cloth type is valid and that the quantity is greater than 0. Otherwise, it should display **"Invalid cloth type or quantity."** If the inputs are valid, the program should calculate the total amount and apply an Eid discount: **10% for a total amount between Tk. 5000 and Tk. 10000**, and **15% for a total amount above Tk. 10000**. If the updated price **exceeds Tk. 2000** after the standard discount and the **last 2-digit of the membership card is even**, an **additional 5% Eid discount** should be applied. However, if this 5% discount causes the final price to **fall below Tk. 4300**, then the discount for the membership card number is removed. Finally, the program should display the final payable amount after all applicable discounts.

| Sample Input                                                                                                                       | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 2<br>Enter quantity: 2<br>Enter membership card number (6-digit): 125468 | Total payable amount: Tk. 4500.0<br><br><b>Explanation:</b><br>The user buys 2 Salwar Kameez for Tk. 5000 total, which gets a 10% discount, reducing the price to Tk. 4500. Since Tk. 4500 is over Tk. 2000 and the last 2-digit of the membership card is even, the 5% Eid discount is applied, dropping the price to Tk. 4275. However, this falls below the Tk. 4300 limit and the 5% discount is removed. |
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 1<br>Enter quantity: 3<br>Enter membership card number (6-digit): 125468 | Total payable amount: Tk. 5386.5<br><br><b>Explanation:</b><br>The user buys 3 Panjabis for a total of Tk. 6300, which gets a 10% discount, reducing the price to Tk. 5670. Since Tk. 5670 is over Tk. 2000 and the last 2-digit of the membership card is even, the 5% Eid discount is applied, dropping the price to Tk. 5386.5.                                                                            |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

An online traditional clothing store is offering special Eid reward points. Write a Java program that asks the user to enter the cloth type (1 for Panjabi at Tk. 2100 per piece, 2 for Salwar Kameez at Tk. 2500 per piece, 3 for Sari at Tk. 4000 per piece), the quantity, and the phone number. The program must check that the cloth type is valid and the quantity is greater than 0. Otherwise, it should display **"Invalid cloth type or quantity."** If the inputs are valid, the program should calculate the initial total amount and determine the Eid reward points as follows: the user earns **2 points for every full Tk. 1000** of the initial purchase if the **last 2-digit of the phone number is odd**. Otherwise, the user earns **5 points** if the initial purchase is between **Tk. 5000 and Tk. 10000**. Each reward point is worth **Tk. 50**. However, the reward point discount is **only** adjusted from the bill if the initial total amount exceeds **Tk. 5000**. Finally, the program should display the final payable amount after any applicable point adjustments.

| Sample Input                                                                                                        | Sample Output                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 2<br>Enter quantity: 2<br>Enter phone number: 01234567862 | Total payable amount: Tk. 5000.0<br><br><b>Explanation:</b><br>The user buys 2 Salwar Kameez for a total of Tk. 5000. The last 2-digit of the phone number is even, so they earn 5 points, which has a value of Tk.250. However, the discount is only applied if the total exceeds Tk. 5000. Since the total does not exceed Tk. 5000, no discount is applied. |
| Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): 1<br>Enter quantity: 3<br>Enter phone number: 01234567891 | Total payable amount: Tk. 5670.0<br><br><b>Explanation:</b><br>The user buys 3 Panjabis for a total of Tk. 6300. The last 2-digit of the phone number is odd, so they earn 12.6 points (6300*2/1000), which has a value of Tk. 630. Since the total exceeds Tk. 5000, the discount is applied, resulting in a final payable amount of Tk. 5670.0               |

## SET - [A]

## SOLUTION

```
import java.util.Scanner;
public class LabExam1SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): ");
        int clothType = sc.nextInt();
        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();
        System.out.print("Enter membership card number (6-digit): ");
        int cardNo = sc.nextInt();

        if ((clothType == 1 || clothType == 2 || clothType == 3) && quantity > 0) {

            double totalAmount = 0;
            if (clothType == 1) {
                totalAmount = 2100 * quantity;
            } else if (clothType == 2) {
                totalAmount = 2500 * quantity;
            } else if (clothType == 3) {
                totalAmount = 4000 * quantity;
            }

            if (totalAmount > 10000) {
                totalAmount = totalAmount - (totalAmount * 0.15);
            } else if (totalAmount >= 5000 && totalAmount <= 10000) {
                totalAmount = totalAmount - (totalAmount * 0.10);
            }

            if (totalAmount > 2000 && (cardNo%100)%2 == 0) {
                double tempAmount = totalAmount - (totalAmount * 0.05);
                if (tempAmount >= 4300) {
                    totalAmount = tempAmount;
                }
            }

            System.out.println("\nTotal payable amount: Tk. " + totalAmount);

        } else {
            System.out.println("\nInvalid quantity or cloth type.");
        }
    }
}
```

| RUBRIC |                                                                                                                                       |            |
|--------|---------------------------------------------------------------------------------------------------------------------------------------|------------|
| SL     | Points to Meet                                                                                                                        | Marks [10] |
| 1      | Correctly imports and initializes Scanner, and takes three inputs using appropriate data types                                        | 1          |
| 2      | Checks that cloth type is 1,2 or 3 and quantity is greater than 0                                                                     | 1          |
| 3      | Accurately determines the price per piece and calculates the initial amount                                                           | 1          |
| 4      | Correctly checks and calculates the 15% discount (for > 10000) and the 10% discount (for >= 5000 and <= 10000).                       | 2          |
| 5      | Checks the last two digits of the membership card to be an even and updated amount > 2000, then correctly calculates the 5% discount. | 2          |
| 6      | Implements the logic to apply or remove the premium membership discount based on the minimum limit of 4300.                           | 2          |
| 7      | Correctly prints the final payable amount and invalid inputs                                                                          | 1          |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;
public class LabExam1SetB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter cloth type (1: Panjabi, 2: Salwar Kameez, 3: Sari): ");
        int clothType = sc.nextInt();
        System.out.print("Enter quantity: ");
        int quantity = sc.nextInt();
        System.out.print("Enter phone number: ");
        int phoneNo = sc.nextInt();

        if ((clothType == 1 || clothType == 2 || clothType == 3) && quantity > 0) {

            double totalAmount = 0;
            if (clothType == 1) {
                totalAmount = 2100 * quantity;
            } else if (clothType == 2) {
                totalAmount = 2500 * quantity;
            } else if (clothType == 3) {
                totalAmount = 4000 * quantity;
            }

            double rewardPoints = 0;
            if ((phoneNo%100)%2 == 0) {
                if (totalAmount >= 5000 && totalAmount <= 10000) {
                    rewardPoints = 5;
                }
            } else {
                rewardPoints = (totalAmount / 1000) * 2;
            }
            if (totalAmount > 5000) {
                totalAmount = totalAmount - (rewardPoints * 50);
            }
            System.out.println("\nTotal payable amount: Tk. " + totalAmount);
        } else {
            System.out.println("\nInvalid cloth type or quantity.");
        }
    }
}
```

| RUBRIC |                                                                                                                                  |               |
|--------|----------------------------------------------------------------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                                                                                   | Marks<br>[10] |
| 1      | Correctly imports and initializes Scanner, and takes three inputs using appropriate data types                                   | 1             |
| 2      | Checks that cloth type is 1,2 or 3 and quantity is greater than 0                                                                | 1             |
| 3      | Accurately determines the price per piece and calculates the initial amount                                                      | 1.5           |
| 4      | Correctly checks for odd last 2-digit of phone number.                                                                           | 1.5           |
| 5      | Correctly implements the reward point logic (2 points for every Tk. 1000 for odd last 2-digit phone number; otherwise, 5 points) | 2             |
| 6      | Implements the logic to apply the point deduction (points x 50) only if the total amount exceeds 5000.                           | 2             |
| 7      | Correctly prints the final payable amount and invalid inputs                                                                     | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze cultural sessions during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of sessions. For each session, the program should first take input of the number of performances. Then, for each performance, the user must input its duration (in minutes).

For every session, the program should count how many performances have a duration greater than 30 minutes and calculate the total duration of that session. If more than half of the performances in a session have a duration greater than 30 minutes, the program should print "Long Session!". After processing all sessions, the program should display the total duration of all sessions combined.

| Input                                                                                                                                                                                                                                                                                                                                    | Output                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of sessions: 1<br><br>Session 1:<br>Enter number of performances: 4<br>Enter duration of performance 1: 10<br>Enter duration of performance 2: 35<br>Enter duration of performance 3: 45<br>Enter duration of performance 4: 20                                                                                             | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><br>Overall Total Duration: 110                                                                                               |
| Enter number of sessions: 2<br><br>Session 1:<br>Enter number of performances: 3<br>Enter duration of performance 1: 20<br>Enter duration of performance 2: 40<br>Enter duration of performance 3: 50<br><br>Session 2:<br>Enter number of performances: 2<br>Enter duration of performance 1: 25<br>Enter duration of performance 2: 30 | Session 1:<br>Long performances count: 2<br>Total duration: 110<br><b>Long Session!</b><br><br>Session 2:<br>Long performances count: 0<br>Total duration: 55<br><br>Overall Total Duration: 165 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to analyze food sales during the celebration of Pahela Baishakh at BRAC University. The program should ask the user to enter the number of student stalls in the festival. For each student stall, the number of items sold is equal to the stall number. For example, Stall 1 sells 1 item, Stall 2 sells 2 items, Stall 3 sells 3 items, and so on. For each item, the user must input its price.

For every student stall, the program should display all items that cost more than 200 taka (premium items) and calculate the total sales of that stall. If a stall has 2 or more premium items, the program should print **"Popular Stall!"**. After processing all stalls, the program should display the overall total sales of the festival.

| Input                                                                                                                                                                                                                                                        | Output                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of stalls: 2<br><br>Stall 1:<br>Enter price of item 1: 220<br><br>Stall 2:<br>Enter price of item 1: 150<br>Enter price of item 2: 250                                                                                                          | Stall 1:<br>Premium items: 220<br>Total sales: 220<br><br>Stall 2:<br>Premium items: 250<br>Total sales: 400<br><br>Overall Festival Sales: 620                                                                                     |
| Enter number of stalls: 3<br><br>Stall 1:<br>Enter price of item 1: 150<br><br>Stall 2:<br>Enter price of item 1: 250<br>Enter price of item 2: 300<br><br>Stall 3:<br>Enter price of item 1: 100<br>Enter price of item 2: 220<br>Enter price of item 3: 50 | Stall 1:<br>Premium items:<br>Total sales: 150<br><br>Stall 2:<br>Premium items: 250 300<br>Total sales: 550<br><b>Popular Stall!</b><br><br>Stall 3:<br>Premium items: 220<br>Total sales: 370<br><br>Overall Festival Sales: 1070 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a string from the user and produces its run-length encoded form. In run-length encoding, each group of consecutive identical characters is replaced by the character followed by its count. If a character appears only once in a row, write the character without a number. After producing the encoded string, compare its length with the original. If the encoded version is shorter, print the encoded string. Otherwise, print the original string unchanged.

| Sample Input | Sample Output                                                |
|--------------|--------------------------------------------------------------|
| aabbbccdddee | a2b3c2d4e2                                                   |
| abcd         | abcd<br>(encoded "a1b1c1d1" is longer, so<br>print original) |
| aaabbaaa     | a3b2a3                                                       |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that reads a main string and a pattern string, then prints how many times the pattern appears in the main string (non-overlapping occurrences only). You must implement the search manually, no built-in method may be used to find or detect the substring.

| Sample Input                      | Sample Output |
|-----------------------------------|---------------|
| main: ababab<br>pattern: ab       | Count: 3      |
| main: aaaaaa<br>pattern: aa       | Count: 3      |
| main: hello world<br>pattern: xyz | Count: 0      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

A weekly attendance system stores 7 digits in an array, where **each index** represents a day of the week in order from Saturday to Friday. In this system, classes on **Saturday** (index 0) and **Tuesday** (index 3) are conducted **online**, while all other days are conducted **in-person**. A day is considered balanced if the number of elements before it (to the left) with lower values is equal to the number of elements after it (to the right) with higher values.

Write a Java program to determine and print **all** balanced days **only among the in-person days**, and also print the **total** number of such balanced in-person days.

| Given Array                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Output                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <pre>int[] arr = {40, 10, 20, 100, 15, 30, 5};</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Balanced in-person days are: 2 4 6</p> <p>Total balanced in-person days: 3</p> |
| <p><b>Explanation:</b></p> <p>Monday (index 2, value 20): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> <p>Wednesday (index 4, value 15): left lower count = 1 (10), right higher count = 1 (30), ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> <p>Friday (index 6, value 5): left lower count = 0, right higher count = 0, ignoring indices 0 and 3 (online). Since both are equal, the day is balanced.</p> |                                                                                   |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

A FIFA World Cup performance tracker stores the number of goal chances created for 7 matches in an array, where **each index** represents a match in sequence from Match 1 to Match 7. In this system, matches played on **Monday** (index 1) and **Thursday** (index 4) are against **rival teams**, while all other matches are **non-rival**. A match is considered Growth-Stable if the number of non-rival matches before it (to the left) with higher values is equal to the number of non-rival matches after it (to the right) with lower values.

Write a Java program to determine and print **all** Growth-Stable matches **only** among the **non-rival** matches, and also print the **total** number of such Growth-Stable matches.

| Given Array                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Output                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <pre>int[] scores = {40, 10, 20, 25, 15, 30, 5};</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <p>Growth-Stable matches are: 2 3 5</p> <p>Total Growth-Stable matches: 3</p> |
| <p><b>Explanation:</b></p> <p>Match 3 (index 2, value 20): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> <p>Match 4 (index 3, value 25): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> <p>Match 6 (index 5, value 30): left higher count = 1 (40), right lower count = 1 (5), ignoring indices 1 and 4. Since both are equal, the match is Growth-Stable.</p> |                                                                               |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate calorie intake during Iftar. The program should ask the user to enter a six-digit number, and the product of the last three digits of this number will be used as the total calorie limit. The user must then input how many dates they want to eat and how many jilapis they want to eat. Each date contains 80 calories and each jilapi contains 140 calories, and the person must always eat dates first. The program should first calculate the total calories from the dates; if the calories from the dates exceed the calorie limit, then the person cannot eat any jilapis and must drink water instead. If the calories from the dates are within the limit, the program should calculate how many jilapis can be eaten without exceeding the total calorie limit.

| Input                                                                                    | Output                                                                                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Enter a 6-digit number: 123456<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 120<br>Calories from dates: 160<br>Dates exceed the calorie limit.<br>You cannot eat jilapis. Drink water instead. |
| Enter a 6-digit number: 243798<br>Enter number of dates: 2<br>Enter number of jilapis: 3 | Calorie limit: 504<br>Calories from dates: 160<br>You can eat up to 2 jilapis.<br>Total calories consumed: 440                    |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a **Java program** that calculates calorie intake during Sehri. The program should take input for the number of rice plates, glasses of milk, and bananas a person wants to eat. Each plate of rice has 200 calories, each glass of milk has 120 calories, and each banana has 100 calories, with a total calorie limit of 800. The program must calculate rice calories first, and if rice alone exceeds 800 calories, the person can only drink water. If not, then calculate the calories from milk and bananas and make sure the total does not go over 800 calories.

| Sample Input                                                                                          | Sample Output                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter number of plates of rice: 5<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 1000<br>Calorie limit exceeded by rice alone!<br>You should drink only water.                                                                      |
| Enter number of plates of rice: 2<br>Enter number of glasses of milk: 1<br>Enter number of bananas: 1 | Rice calories = 400<br>Milk calories = 120<br>Banana calories = 100<br>Total calorie intake = 620<br>You are within the 800 calorie limit. Good Sehri!             |
| Enter number of plates of rice: 3<br>Enter number of glasses of milk: 2<br>Enter number of bananas: 2 | Rice calories = 600<br>Adding milk and bananas exceeds the 800 calorie limit.<br>You should reduce milk or bananas.<br>Maximum remaining calories after rice = 200 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

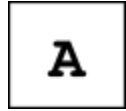
### Question 1 [C01] [10 Points]

Write a Java program that takes an integer input N from the user and prints a pattern given below:

| Input | Output                                                                                                                                                        |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| N = 5 | <pre> 12345 12345 12345 12345 12345 123456789  1*****7    1***5     1*3      1 </pre>                                                                         |
| N = 7 | <pre> 1234567 1234567 1234567 1234567 1234567 1234567 1234567 1234567 12345678910111213  1*****11    1*****9     1*****7      1***5       1*3        1 </pre> |



CSE110: Programming Language I (Lab)  
Spring 2025  
Lab Exam - 02  
Duration: 25 Minutes



|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

A secret network system logs packet identifiers as a string of **lowercase letters**. During stable transmission, packet IDs appear in **continuous alphabetical order**, where each next character is exactly **one greater than the previous one**. Your task is to: Find the **longest continuous alphabetical sequence** in the string. If the **length of that sequence is greater than 5**, convert the sequence to **UPPERCASE** and print **every alternate character**, starting from the **first character**. If the length is **5 or less**, print the sequence normally (no skipping, no case change)

- You may only use `length()`, `charAt()`, loops, and conditions

| Sample Input      | Sample Output |
|-------------------|---------------|
| xyzmnopqrstuvwxyz | MOQS UW       |
| awdxyz            | xyz           |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

In a live cricket match analytics system, individual player scores are being recorded in real time as the match progresses. The system can track a maximum of 11 players.

Each time a player's score is entered, the system must immediately display the current Top-3 highest scores, showing the three highest run-scorers in descending order. If fewer than 3 player scores have been entered, the system should display:

Minimum 3 scores needed to see the leaderboard assume that the user will give a minimum 4 scores.

Score input continues until:

- The user enters -1, or
- A total of 11 player scores have been recorded

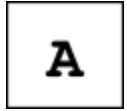
(whichever occurs first)

After input ends, the system must display the final Top-3 highest scores (Top Run Scorers).

| Sample Input                                 | Sample Output                                                                                                                                                                                                                                                          |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 45<br>78<br>30<br>92<br>65<br>21<br>-1       | Minimum 3 scores needed to see the<br>leaderboard<br>Minimum 3 scores needed to see the<br>leaderboard<br>Current Top 3: 78 45 30<br>Current Top 3: 92 78 45<br>Current Top 3: 92 78 65<br>Current Top 3: 92 78 65<br>Final Top 3: 92 78 65                            |
| 50<br>88<br>34<br>75<br>60<br>41<br>95<br>-1 | Minimum 3 scores needed to see the<br>leaderboard<br>Minimum 3 scores needed to see the<br>leaderboard<br>Current Top 3: 88 50 34<br>Current Top 3: 88 75 50<br>Current Top 3: 88 75 60<br>Current Top 3: 88 75 60<br>Current Top 3: 95 88 75<br>Final Top 3: 95 88 75 |



CSE110: Programming Language I (Lab)  
Spring 2026  
Lab Exam - 4  
Duration: 25 Minutes



|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Peter Parker is buying snacks from a smart vending machine at his university. The machine sells snack boxes. Each snack box costs 37 taka. The machine first calculates the total cost based on the number of snack boxes purchased.

- If the total cost is more than 1000 taka, the machine gives a 10% discount, and no service charge is added.
- If the total cost is less than 200 taka, the machine adds an extra 20 taka service charge.

After calculating the final payable amount, the machine displays how the customer should pay the amount using only 50 taka, 20 taka, 5 taka, 2 taka, and 1 taka notes.

Assume that the final amount will always be payable exactly using these notes.

| Sample Input                   | Sample Output                                                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Enter the number of snacks: 20 | Final Bill: 740<br>50 taka notes: 14<br>20 taka notes: 2<br>5 taka notes: 0<br>2 taka notes: 0<br>1 taka notes: 0 |
| Enter the number of snacks: 5  | Final Bill: 205<br>50 taka notes: 4<br>20 taka notes: 0<br>5 taka notes: 1<br>2 taka notes: 0<br>1 taka notes: 0  |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Tony Stark recently installed an “Energy Booster Dispenser” in the engineering building because students keep falling asleep during lectures. The machine sells energy drink cans. Each drink costs 42 taka. The machine first calculates the total cost based on the number of cans selected.

If the total cost is more than 1200 taka, the machine gives a 12% discount as a “VIP Bulk Buyer Reward”. If the total cost is less than 250 taka, the machine adds a 30 taka maintenance charge because “small orders waste power”.

After calculating the final payable amount, the machine shows how the customer should pay using only 100 taka, 20 taka, 5 taka, 2 taka, and 1 taka notes.

| Sample Input                  | Sample Output                                                                                                     |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Enter the number of cans: 10  | Final Bill: 420<br>100 taka notes: 4<br>20 taka notes: 1<br>5 taka notes: 0<br>2 taka notes: 0<br>1 taka notes: 0 |
| Enter the number of snacks: 4 | Final Bill: 198<br>100 taka notes: 1<br>20 taka notes: 4<br>5 taka notes: 3<br>2 taka notes: 1<br>1 taka notes: 1 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

In the futuristic city of **Energon Prime**, engineers have discovered a powerful **P-digit** Energy Code(Number) that controls the entire city's power grid. This code must be carefully analyzed to unlock its hidden energy potential. The system responds only to a **special digit N** embedded within the code.

As you examine the Energy Code from Right to Left:

- If the digit N is not found anywhere in the number, then the system shuts down and no energy is generated.
- If the digit N is found, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as  $\text{digit} \times (\text{frequency})$ . **All nonzero contributions** are multiplied to get the **final energy**.

Write a Java program to print whether N was found and the resulting energy value, or indicate that no energy is generated.

| Sample Input 1 | Sample Output 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 122331<br>3    | Digit 0 -> Frequency: 0, Contribution: 0<br>Digit 1 -> Frequency: 2, Contribution: 2<br>Digit 2 -> Frequency: 2, Contribution: 4<br>Digit 3 -> Frequency: 2, Contribution: 6<br>Digit 4 -> Frequency: 0, Contribution: 0<br>Digit 5 -> Frequency: 0, Contribution: 0<br>Digit 6 -> Frequency: 0, Contribution: 0<br>Digit 7 -> Frequency: 0, Contribution: 0<br>Digit 8 -> Frequency: 0, Contribution: 0<br>Digit 9 -> Frequency: 0, Contribution: 0<br>Total Energy: 48<br><b>Explanation:</b><br>Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times<br>$(1 \times 2) \times (2 \times 2) \times (3 \times 2)$<br>= 48 |
| Sample Input 2 | Sample Output 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 456789<br>2    | 2 NOT FOUND<br>NO ENERGY GENERATED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

In the ancient cyber realm of **Aetherion Grid**, engineers have discovered a powerful **P-digit Signal Key (Number)** that controls the planetary communication network. This code must be analyzed to unlock its hidden energy. The system responds only to a special digit **N** embedded within the number.

As you examine the Signal Key from Right to Left:

- If the digit **N** is not found anywhere in the number, the system shuts down and no energy is generated.
- If the digit **N** is found anywhere in the number, the system activates. To compute the energy, the system uses a layered frequency analysis method. For each digit from 0 to 9, the system re-scans the entire number to count its occurrences and print frequency. The energy contribution of each digit is calculated as **digit × (frequency<sup>2</sup>)**. All contributions are summed to get the final energy.

Write a Java program to print whether **N** was found and the resulting energy value, or indicate that no energy is generated.

| Sample Input 1 | Sample Output 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 122331<br>3    | Digit 0 -> Frequency: 0, Contribution: 0<br>Digit 1 -> Frequency: 2, Contribution: 4<br>Digit 2 -> Frequency: 2, Contribution: 8<br>Digit 3 -> Frequency: 2, Contribution: 12<br>Digit 4 -> Frequency: 0, Contribution: 0<br>Digit 5 -> Frequency: 0, Contribution: 0<br>Digit 6 -> Frequency: 0, Contribution: 0<br>Digit 7 -> Frequency: 0, Contribution: 0<br>Digit 8 -> Frequency: 0, Contribution: 0<br>Digit 9 -> Frequency: 0, Contribution: 0<br>Total Energy: 24<br><u>Explanation:</u> 3 is found!<br>Frequency count: 1 → 2 times, 2 → 2 times, 3 → 2 times<br>$1 \times 2^2 + 2 \times 2^2 + 3 \times 2^2$<br>= 24 |
| Sample Input 2 | Sample Output 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 456789<br>2    | 2 NOT FOUND<br>NO ENERGY GENERATED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## SETA-Solution

```
import java.util.Scanner;
public class SetA {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int signalKey = sc.nextInt();
        int N = sc.nextInt();
        int temp = signalKey;
        boolean found = false;
        // Check digit N from right to left
        while (temp > 0) {
            if (temp % 10 == N) {
                found = true;
            }
            temp /= 10;
        }
        if (found == false) {
            System.out.println("Digit " + N + " not found. No energy generated.");
        }
        else {
            int totalEnergy = 1;
            // For digits 0 to 9
            for (int d = 0; d <= 9; d++) {
                int freq = 0;
                temp = signalKey;
                // count frequency of digit d
                while (temp > 0) {
                    if (temp % 10 == d) {
                        freq++;
                    }
                    temp /= 10;
                }
                int contribution = d * freq;
                System.out.println("Digit " + d +
                    " -> Frequency: " + freq +
                    ", Contribution: " + contribution);
                // multiply only non-zero contributions
                if (contribution != 0) {
                    totalEnergy *= contribution;
                }
            }
            System.out.println("Total Energy: " + totalEnergy);
        }
    }
}
```

## SETB-Solution

```
import java.util.Scanner;
public class SETB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int signalKey = sc.nextInt();
        int N = sc.nextInt();
        int temp = signalKey;
        boolean found = false;
        //check if N exists
        while (temp > 0) {
            int digit = temp % 10;
            if (digit == N) {
                found = true;
            }
            temp /= 10;
        }
        if (!found) {
            System.out.println("Digit " + N + " not found. No energy generated.");
        }
        else{
            int totalEnergy = 0;
            // process digits 0 to 9
            for (int d = 0; d <= 9; d++) {
                int freq = 0;
                temp = signalKey;
                // count frequency of digit d
                while (temp > 0) {
                    if (temp % 10 == d) {
                        freq++;
                    }
                    temp /= 10;
                }
                int contribution = d * (freq * freq);
                totalEnergy += contribution;
                System.out.println("Digit " + d +
                    " -> Frequency: " + freq +
                    ", Contribution: " + contribution);
            }
            System.out.println("Total Energy: " + totalEnergy);
        }
    }
}
```

**RUBRIC for Set A & B**

| SL | Points To Meet                                 | Marks [10] |
|----|------------------------------------------------|------------|
| 1. | Proper Syntax, import Scanner, Scanner Obj     | 1          |
| 2. | Take two user inputs & other variables         | 1.5        |
| 3. | Search the number                              | 2          |
| 4. | Number <b>not found condition &amp; print</b>  | 1          |
| 5. | Loop → If number is <b>found</b>               | 1.5        |
| 6. | count frequency of digit d & other calculation | 3          |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer  $d$ , where  $0 \leq d < n$  and  $n$  is the length of the string. The program should perform a **left rotation** on the given string by  $d$  positions.

In a left rotation:

- The first  **$d$  characters** of the string are moved to the end.
- The remaining characters are shifted to the left.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

| Sample Input:                                                                                                                  | Sample Output:                                |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Enter string: ABCD<br>Enter d: 1<br>Enter string: ABCD<br>Enter d: 2<br>Enter string: HELLO<br>Enter d: 3<br>Enter string: END | BCDA<br>CDAB<br>LOHEL<br>The program stopped. |

SET-B

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

Write a Java program that continuously takes input from the user using a loop. In each iteration, the user will enter a string and an integer  $d$ , where  $0 \leq d < n$  and  $n$  is the length of the string. The program should perform a **right rotation** on the given string by  $d$  positions.

A right rotation means:

- moving the last  **$d$  characters** of the string to the beginning
- shifting the remaining characters to the right.

After performing the rotation, the program should display the updated string. The process should continue until the user enters "END" as the string, at which point the program should stop execution.

| Sample Input:                                                                                                                  | Sample Output:                                |
|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Enter string: ABCD<br>Enter d: 1<br>Enter string: ABCD<br>Enter d: 2<br>Enter string: HELLO<br>Enter d: 3<br>Enter string: END | DABC<br>CDAB<br>LLOHE<br>The program stopped. |

# Solution & Rubric

[CSE110 Spring 2026 Lab Exam - 03 Tentative Solutions and Rubrics](#)

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("Enter string: ");
            String s = sc.nextLine();

            if (s.equals("END")) {
                break;
            }

            System.out.print("Enter d: ");
            int d = sc.nextInt();
            sc.nextLine();

            int n = s.length();
            d = d % n;

            String temp = "";
            int i = 0;
            while (i < d) {
                temp += s.charAt(i);
                i++;
            }

            String result = "";
            i = d;
            while (i < n) {
                result += s.charAt(i);
                i++;
            }

            i = 0;
            while (i < temp.length()) {
                result += temp.charAt(i);
                i++;
            }

            System.out.println(result);
        }

        sc.close();
        System.out.println("The program stopped.");
    }
}
```

| RUBRIC |                                                                                                                                                                                                                                                                                   |               |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| S<br>L | Points to Meet                                                                                                                                                                                                                                                                    | Marks<br>[10] |
| 1      | Proper class declaration                                                                                                                                                                                                                                                          | 0.5           |
| 2      | Input Handling: Correctly uses loop to take continuous string input and correct termination of loop.                                                                                                                                                                              | 2.5           |
| 3      | Left Rotation Logic (5 marks) <ul style="list-style-type: none"> <li>2 mark: Correctly extracts first <b>d</b> characters</li> <li>2 mark: Correctly extracts remaining characters from <b>d</b> to end</li> <li>2 mark: Correctly concatenates parts in rotated order</li> </ul> | 6             |
| 4      | Correct output handling                                                                                                                                                                                                                                                           | 1             |
| 5      |                                                                                                                                                                                                                                                                                   |               |

SET - [B]

| SOLUTION |
|----------|
|----------|

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.print("Enter string: ");
            String s = sc.nextLine();

            if (s.equals("END")) {
                break;
            }

            System.out.print("Enter d: ");
            int d = sc.nextInt();
            sc.nextLine();

            int n = s.length();
            d = d % n;
            String temp = "";
            int i = n-d;
            while (i < n) {
                temp += s.charAt(i);
                i++;
            }

            String result = "";
            i = 0;
            while (i < n-d) {
                result += s.charAt(i);
                i++;
            }

            i = 0;
            while (i < temp.length()) {
                result += temp.charAt(i);
                i++;
            }

            System.out.println(result);
        }

        sc.close();
        System.out.println("The program stopped.");
    }
}

```

#### RUBRIC

| SL | Points to Meet | Marks |
|----|----------------|-------|
|----|----------------|-------|

|   |                                                                                                                                                                                                                                                                                 |      |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
|   |                                                                                                                                                                                                                                                                                 | [10] |
| 1 | Proper class declaration                                                                                                                                                                                                                                                        | 0.5  |
| 2 | Input Handling: Correctly uses loop to take continuous string input and correct termination of loop.                                                                                                                                                                            | 2.5  |
| 3 | <b>Right Rotation Logic (5 marks)</b> <ul style="list-style-type: none"> <li>• 2 mark: Correctly extracts last <b>d</b> characters</li> <li>• 2 mark: Correctly handles remaining part (0 to n-d-1)</li> <li>• 2 mark: Correctly concatenates parts in rotated order</li> </ul> | 6    |
| 4 | Correct output handling                                                                                                                                                                                                                                                         | 1    |
| 5 |                                                                                                                                                                                                                                                                                 |      |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **Friends**. A user is considered a **true Friends binge watcher** only if they watched episodes for at least **3 consecutive days** where the number of episodes watched each day was **strictly increasing**. Among all such increasing streaks, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *Friends* a user watched each day. Write a Java program that prints "True Binge Watcher" if the selected streak has a total of **10 or more episodes**; otherwise print "Casual Watcher".

|                                                                                             |                                                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sample Input 1:</b><br><br>Given Array:<br><pre>int [] watch = {2, 3, 5, 6, 1, 2};</pre> | <b>Sample Output 1:</b><br><br>True Binge Watcher<br><b>Explanation:</b><br>Increasing streaks:<br>2, 3, 5, 6 → length = 4, sum = 16<br>1, 2 → length = 2 (ignored)<br>Selected streak = 2, 3, 5, 6<br>sum = 16 >= 10 |
| <b>Sample Input 2:</b><br><br>Given Array:<br><pre>int [] watch = {1, 2, 3, 0, 1};</pre>    | <b>Sample Output 2:</b><br><br>Casual Watcher<br><b>Explanation:</b><br>Increasing streaks:<br>1, 2, 3 → length = 3, sum = 6<br>0, 1 → length = 2 (ignored)<br>Selected streak = 1, 2, 3<br>sum = 6 < 10              |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

The streaming service is analyzing viewing habits for the TV show **The Big Bang Theory**. A user is considered a true *The Big Bang Theory* binge watcher only if they watched episodes for **at least 3 consecutive days** where the number of episodes watched each day was strictly decreasing. Among all such **decreasing streaks**, consider only the **longest streak**, and if two streaks have the same length, choose the one with the higher total episodes watched. Each element of a given integer array **watch** represents how many episodes of *The Big Bang Theory* a user watched each day. Write a Java program that prints **"True Binge Watcher"** if the selected streak has a total of 12 or more episodes; otherwise print **"Casual Watcher"**.

|                                                                                                |                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sample Input 1:</b><br><br>Given Array:<br><pre>int [] watch = {8, 6, 4, 2, 5, 3, 1};</pre> | <b>Sample Output 1:</b><br><br>True Binge Watcher<br><b>Explanation:</b><br>Decreasing streaks are:<br>8, 6, 4, 2 → length = 4, sum = 20<br>5, 3, 1 → length = 3, sum = 9<br>Longest streak = 8, 6, 4, 2<br>Sum = 20 >= 12                                                       |
| <b>Sample Input 2:</b><br><br>Given Array:<br><pre>int [] watch = {5, 4, 2, 6, 2, 1};</pre>    | <b>Sample Output 2:</b><br><br>Casual Watcher<br><b>Explanation:</b><br>Decreasing streaks are:<br>5, 4, 2 → length = 3, sum = 11<br>6, 2, 1 → length = 3, sum = 9<br>Both streaks have same length, so choose the one with the higher total episodes watched:<br>sum = 11 <= 12 |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
public class BingeWatcher {
    public static void main(String[] args) {
        int[] watch = {2, 3, 5, 6, 1, 2};

        int maxLength = 0;
        int maxSum = 0;

        int currentLength = 1;
        int currentSum = watch[0];

        for (int i = 1; i < watch.length; i++) {

            if (watch[i] > watch[i - 1]) {
                currentLength++;
                currentSum += watch[i];
            } else {
                if (currentLength >= 3) {
                    if (currentLength > maxLength ||
                        (currentLength == maxLength && currentSum > maxSum)) {
                        maxLength = currentLength;
                        maxSum = currentSum;
                    }
                }

                currentLength = 1;
                currentSum = watch[i];
            }
        }

        // checking last streak
        if (currentLength >= 3) {
            if (currentLength > maxLength ||
                (currentLength == maxLength && currentSum > maxSum)) {
                maxLength = currentLength;
                maxSum = currentSum;
            }
        }

        if (maxSum >= 10) {
            System.out.println("True Binge Watcher");
        } else {
            System.out.println("Casual Watcher");
        }
    }
}
```

**RUBRIC**

| SL | Points to Meet                                                                                                                                                                      | Marks<br>[10] |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java syntax (class, main method, array declaration, variables)                                                                                                               | 1             |
| 2  | Correctly identifies strictly increasing consecutive elements ( <code>watch[i]&gt;watch[i-1]</code> )                                                                               | 2             |
| 3  | Properly tracks current streak length and sum                                                                                                                                       | 1             |
| 4  | Applies condition to consider only streaks of length $\geq 3$                                                                                                                       | 1             |
| 5  | Applies tie-breaker using higher sum when lengths are equal ( <code>currentLength == maxLength &amp;&amp; currentSum &gt; maxSum</code> ) and updates maxLength and maxsum properly | 2             |
| 6  | Correctly handles the last streak after loop                                                                                                                                        | 1             |
| 7  | Uses proper <code>maxsum&gt;=10</code> condition and prints correct output                                                                                                          | 2             |

### SET - [B]

|          |
|----------|
| SOLUTION |
|----------|

```

public class BingeWatcher {
    public static void main(String[] args) {
        int[] watch = {8, 6, 4, 2, 5, 3, 1};

        int maxLength = 0;
        int maxSum = 0;

        int currentLength = 1;
        int currentSum = watch[0];

        for (int i = 1; i < watch.length; i++) {
            if (watch[i] < watch[i - 1]) {
                currentLength++;
                currentSum += watch[i];
            } else {
                if (currentLength >= 3) {
                    if (currentLength > maxLength ||
                        (currentLength == maxLength && currentSum > maxSum)) {
                        maxLength = currentLength;
                        maxSum = currentSum;
                    }

                    currentLength = 1;
                    currentSum = watch[i];
                }
            }
            // Check the last streak
            if (currentLength >= 3) {
                if (currentLength > maxLength ||
                    (currentLength == maxLength && currentSum > maxSum)) {
                    maxLength = currentLength;
                    maxSum = currentSum;
                }
            }

            if (maxSum >= 12) {
                System.out.println("True Binge Watcher");
            } else {
                System.out.println("Casual Watcher");
            }
        }
    }
}

```

#### RUBRIC

| SL | Points to Meet                                                                                        | Marks<br>[10] |
|----|-------------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java syntax (class, main method, array declaration, variables)                                 | 1             |
| 2  | Correctly identifies strictly increasing consecutive elements ( <code>watch[i]&lt;watch[i-1]</code> ) | 2             |

|   |                                                                                                                                                                                     |   |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| 3 | Properly tracks current streak length and sum                                                                                                                                       | 1 |
| 4 | Applies condition to consider only streaks of length $\geq 3$                                                                                                                       | 1 |
| 5 | Applies tie-breaker using higher sum when lengths are equal ( <code>currentLength == maxLength &amp;&amp; currentSum &gt; maxSum</code> ) and updates maxlength and maxsum properly | 2 |
| 6 | Correctly handles the last streak after loop                                                                                                                                        | 1 |
| 7 | Uses proper <code>maxsum &gt;= 12</code> condition and prints correct output                                                                                                        | 2 |

|        |     |          |
|--------|-----|----------|
| Name : | ID: | Section: |
|--------|-----|----------|

### Question 1 [C01] [10 Points]

You are preparing iftar plates for family members and each plate must contain **3 dates** and served with **1 glass of water**. However you only have a certain amount of dates and water glasses ready. Write a Java program, that takes as input the total number of people at iftar, total dates available and total water glasses available. Your program must

1. Calculate the total number of dates and glasses of water required to serve all the people
2. Display appropriate message based on if everyone can be served or not:
  - a. Both dates and water glasses are sufficient, display "All plates can be served."
  - b. Only dates are insufficient display "Not enough dates."
  - c. Only water glasses are insufficient display "Not enough water glasses."
  - d. Both are insufficient "Not enough dates and water glasses."
4. Moreover, if all plates can be served, display remaining dates and water glasses and if you can serve additional people or not.

| Sample Input                                                                                                    | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total number of people: 5<br>Enter total dates available: 18<br>Enter total glasses of water available: 6 | Dates Required: 15 , Water Required: 5<br>All plates can be served.<br>Remaining dates: 3<br>Remaining water glasses: 1<br>Can serve additional people.<br><b>Explanation:</b><br>5 people need 5 water glasses and 15 dates. Since given amount of water is 5 and dates are 18, so everyone can be served. Moreover there are 3 additional dates and one additional water. Using 3 dates and 1 glass of water another person can be served. |
| Enter total number of people: 3<br>Enter total dates available: 10<br>Enter total glasses of water available: 3 | Dates Required: 9 , Water Required: 3<br>All plates can be served<br>Remaining dates: 1<br>Remaining water glasses: 0<br>Can NOT serve additional people.<br><b>Explanation:</b><br>At least 3 dates and 1 water needed to serve additional people                                                                                                                                                                                           |
| Enter total number of people: 3<br>Enter total dates available: 8<br>Enter total glasses of water available: 4  | Dates Required: 9 , Water Required: 3<br>Not enough dates.                                                                                                                                                                                                                                                                                                                                                                                   |

|        |     |          |
|--------|-----|----------|
| Name : | ID: | Section: |
|--------|-----|----------|

### Question 1 [C01] [10 Points]

You want to donate clothes and socks to people in need. Each person will receive **1 piece of clothing** and **2 socks**. However, you have a limited number of clothes and socks.

Write a Java program, that takes as input the total number of people you plan on donating to, total clothes available and total socks available. Your program must

1. Calculate the total number of clothes and socks required to serve all the people
2. Display appropriate message based on if everyone can be served or not:
  - a. Both clothes and socks are sufficient, display "All people can receive donation."
  - b. Only clothes are insufficient display "Not enough clothes."
  - c. Only socks are insufficient display "Not enough socks."
  - d. Both are insufficient "Not enough clothes and socks."
4. Moreover, if all people can receive donation, display remaining clothes and socks and if you can donate to additional people or not.

| Sample Input                                                                                           | Sample Output                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Enter total number of people: 6<br>Enter total clothes available: 8<br>Enter total socks available: 16 | Clothes Required: 6 , Socks Required: 12<br>All people can receive donation.<br>Remaining clothes: 2<br>Remaining socks: 4<br>Can donate to additional people.<br><b>Explanation:</b><br>6 people need 6 clothes and 12 socks. Since given amount of clothes is 8 and socks are 16, so everyone can be served.<br>Moreover there are 2 additional clothes and 4 additional socks.<br>Using 2 socks and 1 cloth another person can be served. |
| Enter total number of people: 4<br>Enter total clothes available: 4<br>Enter total socks available: 9  | Clothes Required: 4 , Socks Required: 8<br>All people can receive donation.<br>Remaining clothes: 0<br>Remaining socks: 1<br>Can NOT donate to additional people.<br><b>Explanation:</b><br>At least 1 cloth and 2 socks are needed to serve one more person                                                                                                                                                                                 |
| Enter total number of people: 6<br>Enter total clothes available: 8<br>Enter total socks available: 5  | Clothes Required: 6 , Socks Required: 12<br>Not enough socks.                                                                                                                                                                                                                                                                                                                                                                                |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class IftarPlates {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total number of people: ");
        int people = sc.nextInt();

        System.out.print("Enter total dates available: ");
        int datesAvailable = sc.nextInt();

        System.out.print("Enter total glasses of water available: ");
        int waterAvailable = sc.nextInt();

        int datesRequired = people * 3;
        int waterRequired = people * 1;

        System.out.println("Dates Required: "+datesRequired+ " , Water Required: "+ waterRequired);

        if (datesAvailable >= datesRequired && waterAvailable >= waterRequired) {

            System.out.println("All plates can be served.");
            int remainingDates = datesAvailable - datesRequired;
            int remainingWater = waterAvailable - waterRequired;

            System.out.println("Remaining dates: " + remainingDates);
            System.out.println("Remaining water glasses: " + remainingWater);

            if (remainingDates >= 3 && remainingWater >= 1) {
                System.out.println("Can serve additional people.");
            } else {
                System.out.println("Can NOT serve additional people.");
            }
        }
        else if (datesAvailable < datesRequired && waterAvailable < waterRequired) {
            System.out.println("Not enough dates and water glasses.");
        }
        else if (datesAvailable < datesRequired) {
            System.out.println("Not enough dates.");
        }
        else {
            System.out.println("Not enough water glasses.");
        }
    }
}
```

| RUBRIC |                                                                                                                                                     |               |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| SL     | Points to Meet                                                                                                                                      | Marks<br>[10] |
| 1      | Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object)                                                  | 1             |
| 2      | Taking 3 inputs properly                                                                                                                            | 1             |
| 3      | Calculating required dates and requires water and properly displaying them                                                                          | 1             |
| 4      | Correctly checking if all people can be served, not enough dates and water, not enough dates only and not enough water only using if-else branching | 4             |
| 5      | Calculating remaining dates and water                                                                                                               | 1             |
| 6      | Correctly checking and displaying whether at least 1 water and 3 dates remain for one additional person                                             | 2             |

**SET - [B]**

| SOLUTION |
|----------|
|----------|

```

import java.util.Scanner;

public class DonationProgram {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter total number of people: ");
        int people = sc.nextInt();

        System.out.print("Enter total clothes available: ");
        int clothesAvailable = sc.nextInt();

        System.out.print("Enter total socks available: ");
        int socksAvailable = sc.nextInt();

        int clothesRequired = people * 1;
        int socksRequired = people * 2;

        System.out.println("Clothes Required: "+clothesRequired+" , Socks Required: "+socksRequired);
        if (clothesAvailable >= clothesRequired && socksAvailable >= socksRequired) {

            System.out.println("All people can receive donation.");

            int remainingClothes = clothesAvailable - clothesRequired;
            int remainingSocks = socksAvailable - socksRequired;
            System.out.println("Remaining clothes: " + remainingClothes);
            System.out.println("Remaining socks: " + remainingSocks);

            if (remainingClothes >= 1 && remainingSocks >= 2) {
                System.out.println("Can donate to additional people.");
            } else {
                System.out.println("Can NOT donate to additional people.");
            }
        }
        else if (clothesAvailable < clothesRequired && socksAvailable < socksRequired) {
            System.out.println("Not enough clothes and socks.");
        }
        else if (clothesAvailable < clothesRequired) {
            System.out.println("Not enough clothes.");
        }
        else {
            System.out.println("Not enough socks.");
        }
    }
}

```

RUBRIC

| SL | Points to Meet                                                                                                                                                 | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object)                                                             | 1             |
| 2  | Taking 3 inputs properly                                                                                                                                       | 1             |
| 3  | Calculating required clothes and requires socks and properly displaying them                                                                                   | 1             |
| 4  | Correctly checking if all people can receive donation, not enough clothes and socks, not enough clothes only and not enough socks only using if-else branching | 4             |
| 5  | Calculating remaining clothes and socks                                                                                                                        | 1             |
| 6  | Correctly checking and displaying whether at least 1 cloth and 2 socks remain for one additional person                                                        | 2             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate electricity bills using a slab system. The program should take **total electricity consumption** (kWh), **connected load** (kW), and **customer type** (1 = Residential, 2 = Commercial, 3 = Agricultural) as input. Then calculate the bill as described below:

| Energy Charge Slabs                                                                                                                                       | Apply discount/rebate based on customer type                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>0 - 100 KWh → 7 Tk per KWh</li> <li>101 - 300 KWh → 9 Tk per KWh</li> <li>Above 300 KWh → 12 Tk per KWh</li> </ul> | <ul style="list-style-type: none"> <li>Residential → 5%</li> <li>Commercial → 3%</li> <li>Agricultural → 10%</li> </ul> |
| <ul style="list-style-type: none"> <li>Demand charge = 30 Tk per kW of connected load.</li> </ul>                                                         |                                                                                                                         |

Bill Calculation:

- Energy charge = Calculated according to slab rates
- Demand charge = Connected load × 30
- Discount = (Energy charge + Demand charge) × Discount Rate
- Final bill = (Energy charge + Demand charge) – Discount

| Input                                                                          | output                                                                                                                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Total electricity consumption = 309<br>Connected load = 2<br>Customer type = 1 | <b>BILL DETAILS:</b><br><br>Energy consumption charge = 2608 Tk<br><br>Demand charge = 60 Tk<br><br>Discount = 133.4 Tk<br><br>Total bill = 2534.6 Tk | <b>Energy consumption charge:</b><br>100*7 = 700tk (for first 100kwh)<br>200*9 = 1800 (for next 200 kwh)<br>9*12= 108 (above 300kwh)<br><br><b>Total energy consumption charge = 700+1800+108 = 2608</b><br><br><b>Demand charge = 2*30 = 60</b><br><br><b>Discount / rebate =</b><br>(2608+60)*5% = 133.4<br><br><b>Total bill = ((2608+ 60)-(2608+ 60)*.05) = 2534.6</b> |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

Write a Java program to calculate water bills using a slab system. The program should take **total water consumption** (in cubic meters, m<sup>3</sup>), **connection size**, and **customer type** (1 = Residential, 2 = Commercial, 3 = Agricultural) as input. Then calculate the bill as described below:

| Water Charge Slabs                                                                                                                                                                                                   | Apply discount/rebate based on customer type                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>0 - 50 m<sup>3</sup> → 10 Tk per m<sup>3</sup></li> <li>51- 100 m<sup>3</sup> → 15 Tk per m<sup>3</sup></li> <li>Above 100 m<sup>3</sup> → 20 Tk per m<sup>3</sup></li> </ul> | <ul style="list-style-type: none"> <li>Residential → 5%</li> <li>Commercial → 3%</li> <li>Agricultural → 10%</li> </ul> |
| <ul style="list-style-type: none"> <li>Connection charge = 50 Tk per unit</li> </ul>                                                                                                                                 |                                                                                                                         |

Bill Calculation:

- Water charge = Calculated according to slab rates
- Connection charge = Connection size × 50
- Discount = (Water charge + Connection charge) × Discount Rate
- Final bill = (Water charge + Connection charge) – Discount

| Input                                                                    | output                                                                                                                                                | Explanation                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Total water consumption = 109<br>Connected size = 2<br>Customer type = 1 | <b>BILL DETAILS:</b><br><br>Water consumption charge = 1430 Tk<br><br>Connection charge = 100 Tk<br><br>Discount = 76.5<br><br>Total bill = 1453.5 Tk | <b>Water consumption charge:</b><br>50*10 = 500tk (for first 50m <sup>3</sup> )<br>50*15 = 750 (for next 50 m <sup>3</sup> )<br>9*20= 180 (above 100 m <sup>3</sup> )<br><br><b>Total water consumption charge</b><br>= 500+750+180 = 1430<br><br><b>Connection charge</b> = 2*50=100<br><br><b>Discount / rebate =</b><br>(1430+100)*5% = 76.5<br><br><b>Total bill =</b><br>((1430+100)-(1430+100)*.05) = 1453.5 |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;
public class ElectricityBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Total electricity consumption = ");
        int consumption = sc.nextInt();

        System.out.print("Connected load = ");
        int load = sc.nextInt();

        System.out.print("Customer type = ");
        int type = sc.nextInt();

        double energyCharge = 0;

        // Slab Calculation
        if (consumption <= 100) {
            energyCharge = consumption * 7;
        } else if (consumption <= 300) {
            energyCharge = (100 * 7) + (consumption - 100) * 9;
        } else {
            energyCharge = (100 * 7) + (200 * 9) + (consumption - 300) * 12;
        }

        // Demand Charge
        double demandCharge = load * 30;

        double total = energyCharge + demandCharge;

        // Discount
        double discount = 0;
        if (type == 1) {
            discount = total * 0.05;    // Residential
        } else if (type == 2) {
            discount = total * 0.03;    // Commercial
        } else if (type == 3) {
            discount = total * 0.10;    // Agricultural
        } else {
            discount = 0;
        }

        double finalBill = total - discount;

        // Output
        System.out.println("\n----- BILL DETAILS -----");
        System.out.println("Energy Charge = " + energyCharge + " Tk");
        System.out.println("Demand Charge = " + demandCharge + " Tk");
    }
}
```

```

        System.out.println("Discount = " + discount + " Tk");
        System.out.println("Total Bill = " + finalBill + " Tk");
    }
}

```

#### RUBRIC

| SL | Points to Meet                                                                                     | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object) | 2             |
| 2  | Taking 3 inputs properly                                                                           | 1             |
| 3  | Calculate energy consumption charge properly                                                       | 2             |
| 4  | Calculate demand charge properly                                                                   | 1             |
| 5  | Calculate discount properly                                                                        | 2             |
| 6  | Calculate final bill properly                                                                      | 1             |
| 7  | Proper print statements according to sample input and output                                       | 1             |

#### SET - [B]

#### SOLUTION

```

import java.util.Scanner;
public class WaterBill {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Total water consumption = ");
        int consumption = sc.nextInt();

        System.out.print("Enter connection size = ");
        int size = sc.nextInt();

        System.out.print("Customer type = ");
        int type = sc.nextInt();

        double waterCharge = 0;

        // Slab Calculation
        if (consumption <= 50) {
            waterCharge = consumption * 10;
        } else if (consumption <= 100) {
            waterCharge = (50 * 10) + (consumption - 50) * 15;
        } else {
            waterCharge = (50 * 10) + (50 * 15) + (consumption - 100) * 20;
        }

        // Connection Charge
        double connectionCharge = size * 50;

        double total = waterCharge + connectionCharge;

        // Discount
        double discount = 0;

        if (type == 1) {
            discount = total * 0.05;    // Residential
        } else if (type == 2) {
            discount = total * 0.03;    // Commercial
        } else if (type == 3) {
            discount = total * 0.10;    // Agricultural
        } else {
            discount = 0;
        }

        double finalBill = total - discount;

        // Output
        System.out.println("\n----- BILL DETAILS -----");
        System.out.println("Water Consumption charge = " + waterCharge + " Tk");
        System.out.println("Connection Charge = " + connectionCharge + " Tk");
        System.out.println("Discount = " + discount + " Tk");
        System.out.println("Total Bill = " + finalBill + " Tk");
    }
}

```

**RUBRIC**

| SL | Points to Meet                                                                                     | Marks<br>[10] |
|----|----------------------------------------------------------------------------------------------------|---------------|
| 1  | Proper Java Syntax (Importing Scanner, creating class and main method and creating Scanner Object) | 2             |
| 2  | Taking 3 inputs properly                                                                           | 1             |
| 3  | Calculate Water consumption charge properly                                                        | 2             |
| 4  | Calculate Connection charge properly                                                               | 1             |
| 5  | Calculate discount properly                                                                        | 2             |
| 6  | Calculate final bill properly                                                                      | 1             |
| 7  | Proper print statements according to sample input and output                                       | 1             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C01] [10 Points]

You are provided with a number N from the user. Your task is to print a pattern of numbers in descending order, where each row starts from n and decreases by 1 until the row number. The total number of rows will be equal to the value of N.

| Sample Input | Sample Output                             |
|--------------|-------------------------------------------|
| 4            | 4<br>43<br>432<br>4321                    |
| 6            | 6<br>65<br>654<br>6543<br>65432<br>654321 |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C01] [10 Points]**

You are given a number N from the user. Your task is to print a pattern where the number of characters in each row increases by 1, and the symbols alternate between two characters. The pattern should continue for N rows.

| Sample Input | Sample Output                                         |
|--------------|-------------------------------------------------------|
| 4            | &<br>\$\$<br>&&&<br>\$\$\$\$                          |
| 6            | &<br>\$\$<br>&&&<br>\$\$\$\$<br>&&&&&<br>\$\$\$\$\$\$ |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are building a text filtering tool for a data cleaning application. Users often input strings where words are separated by a special character. Some characters in each word appear only once (considered noise), while others appear multiple times (considered significant).

Your task is to develop a program that removes all unique (single-occurrence) characters from each word while keeping all occurrences of characters that appear more than once.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, remove characters that appear only once
- Print the filtered words

| Input                   | Output                            |
|-------------------------|-----------------------------------|
| heellllloo-worrlld<br>- | The words are:<br>eellllloo<br>rr |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are developing a text analyzer where users input strings with repeated characters and special separators. In this case, characters that appear only once are considered meaningful, while repeated characters are treated as noise.

Your task is to develop a program that keeps only the characters that appear exactly once in each word and removes all repeated characters.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, keep only characters that appear once
- Print the cleaned words

| Input                   | Output                      |
|-------------------------|-----------------------------|
| heellllloo#worrlld<br># | The words are:<br>h<br>wold |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to update the array by replacing the duplicate values of the array with zero. The updated array should be sorted in ascending order. Finally, you have to print the position of the number (you took as input from the user) from the updated array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem. Can't create a new array for this problem.**

| Input                                             | Output                                                                                                      |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=2</pre> | <pre>Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 2 = 5</pre>         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=5</pre> | <pre>Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 5 = Not found</pre> |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to create a new array that will contain only the unique (distinct) elements of the given array. The new array should be sorted in descending order. Finally, you have to print the position of the number (you took as input from the user) in the new array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem.**

| Input                                             | Output                                                                                   |
|---------------------------------------------------|------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=6</pre> | <pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 6 = 2</pre>         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=7</pre> | <pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 7 = Not found</pre> |



CSE110: Programming Language I (Lab)  
Spring 2026  
Lab Exam - 02  
Duration: 15 Minutes

**A**

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C01] [10 Points]**

You are provided with a number N from the user. Your task is to print a pattern of numbers in descending order, where each row starts from n and decreases by 1 until the row number. The total number of rows will be equal to the value of N.

| Sample Input | Sample Output                             |
|--------------|-------------------------------------------|
| 4            | 4<br>43<br>432<br>4321                    |
| 6            | 6<br>65<br>654<br>6543<br>65432<br>654321 |



|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

**Question 1 [C01] [10 Points]**

You are given a number N from the user. Your task is to print a pattern where the number of characters in each row increases by 1, and the symbols alternate between two characters. The pattern should continue for N rows.

| Sample Input | Sample Output                                         |
|--------------|-------------------------------------------------------|
| 4            | &<br>\$\$<br>&&&<br>\$\$\$\$                          |
| 6            | &<br>\$\$<br>&&&<br>\$\$\$\$<br>&&&&&<br>\$\$\$\$\$\$ |

[CSE110 Spring 2025 Lab Exam - 0X Tentative Solutions and Rubrics](#)

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class SetA {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        for (int i = 1; i <= N; i++) {
            for (int j = N; j >= N - i + 1; j--) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

**RUBRIC**

| SL | Points to Meet                    | Marks [10] |
|----|-----------------------------------|------------|
| 1  | Take input from user              | 1          |
| 2  | Properly handle outer loop        | 2          |
| 3  | Properly handle inner loop        | 4          |
| 4  | Correct output pattern generation | 3          |
| 5  |                                   |            |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;

public class SetB {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                if (i % 2 == 1) {
                    System.out.print("&");
                } else {
                    System.out.print("$");
                }
            }
            System.out.println();
        }
    }
}
```

**RUBRIC**

| SL | Points to Meet                    | Marks [10] |
|----|-----------------------------------|------------|
| 1  | Take input from user              | 1          |
| 2  | Properly handle outer loop        | 2          |
| 3  | Properly handle inner loop        | 4          |
| 4  | Correct output pattern generation | 3          |
| 5  |                                   |            |



|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are building a text filtering tool for a data cleaning application. Users often input strings where words are separated by a special character. Some characters in each word appear only once (considered noise), while others appear multiple times (considered significant).

Your task is to develop a program that removes all unique (single-occurrence) characters from each word while keeping all occurrences of characters that appear more than once.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, remove characters that appear only once
- Print the filtered words

| Input                   | Output                            |
|-------------------------|-----------------------------------|
| heellllloo-worrrld<br>- | The words are:<br>eellllloo<br>rr |



|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C02] [10 Points]

You are developing a text analyzer where users input strings with repeated characters and special separators. In this case, characters that appear only once are considered meaningful, while repeated characters are treated as noise.

Your task is to develop a program that keeps only the characters that appear exactly once in each word and removes all repeated characters.

The program should:

- Read a string containing multiple words separated by a user-specified character
- Read the separator character
- Split the string manually (no built-in split)
- For each word, keep only characters that appear once
- Print the cleaned words

| Input                   | Output                      |
|-------------------------|-----------------------------|
| heellllloo#worrlld<br># | The words are:<br>h<br>wold |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.Scanner;

public class Question1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            Count++;
                        }
                    }

                    // keep only repeated characters
                    if (count > 1) {
                        result = result + current;
                    }

                    System.out.println(result);
                    word = "";
                } else {
                    word = word + ch;
                }
            }
        }
    }
}
```

| RUBRIC |                                                     |               |
|--------|-----------------------------------------------------|---------------|
| SL     | Points to Meet                                      | Marks<br>[10] |
| 1      | Correct input handling (string + separator)         | 2             |
| 2      | Proper manual splitting using loop (no built-in)    | 2             |
| 3      | Correct logic for counting character frequency      | 2             |
| 4      | Correct filtering condition (Set A: >1, Set B: ==1) | 2             |
| 5      | Correct output formatting and handling all words    | 2             |

**SET - [B]****SOLUTION**

```
import java.util.Scanner;

public class Question2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();
        char separator = sc.nextLine().charAt(0);

        // add separator at end
        text = text + separator;

        System.out.println("The words are:");

        String word = "";

        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);

            if (ch == separator) {
                String result = "";

                for (int j = 0; j < word.length(); j++) {
                    char current = word.charAt(j);
                    int count = 0;

                    for (int k = 0; k < word.length(); k++) {
                        if (word.charAt(k) == current) {
                            count++;
                        }
                    }

                    // keep only unique characters
                    if (count == 1) {
                        result = result + current;
                    }
                }

                System.out.println(result);
                word = "";
            } else {
                word = word + ch;
            }
        }
    }
}
```

| RUBRIC |                                                     |               |
|--------|-----------------------------------------------------|---------------|
| SL     | Points to Meet                                      | Marks<br>[10] |
| 1      | Correct input handling (string + separator)         | 2             |
| 2      | Proper manual splitting using loop (no built-in)    | 2             |
| 3      | Correct logic for counting character frequency      | 2             |
| 4      | Correct filtering condition (Set A: >1, Set B: ==1) | 2             |
| 5      | Correct output formatting and handling all words    | 2             |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to update the array by replacing the duplicate values of the array with zero. The updated array should be sorted in ascending order. Finally, you have to print the position of the number (you took as input from the user) from the updated array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem. Can't create a new array for this problem.**

| Input                                             | Output                                                                                                      |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=2</pre> | <pre>Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 2 = 5</pre>         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=5</pre> | <pre>Updated array: 6, 2, 0, 1, 0, 9, 0 Sorted array: 0,0,0,1,2,6,9 Output: Position of 5 = Not found</pre> |

|       |     |          |
|-------|-----|----------|
| Name: | ID: | Section: |
|-------|-----|----------|

### Question 1 [C03] [10 Points]

You are given an integer array and you need to take a number as input from the user. Your task is to create a new array that will contain only the unique (distinct) elements of the given array. The new array should be sorted in descending order. Finally, you have to print the position of the number (you took as input from the user) in the new array. Print **"Not found"** if the number is not in the array.

**Can't use built-in sort for the problem.**

| Input                                             | Output                                                                                   |
|---------------------------------------------------|------------------------------------------------------------------------------------------|
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=6</pre> | <pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 6 = 2</pre>         |
| <pre>int arr [] = {6, 2, 6, 1, 2, 9, 1} N=7</pre> | <pre>New array: 6, 2, 1, 9 Sorted array: 9,6,2,1 Output: Position of 7 = Not found</pre> |

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

**SET - [A]**

**SOLUTION**

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 5;

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < i; j++) {
                if (arr[i] == arr[j]) {
                    arr[i] = 0;
                    break;
                }
            }
        }

        System.out.println("Updated Array:");
        for (int num : arr) {
            System.out.print(num + " ");
        }

        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
            if (arr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }else{
            System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
        }
    }
}
```

```
}
```

**RUBRIC**

| SL | Points to Meet                                                        | Marks<br>[10] |
|----|-----------------------------------------------------------------------|---------------|
| 1  | Handle Duplicates by replacing with 0 in the array                    | 3             |
| 2  | Sort the array in ascending order                                     | 3             |
| 3  | Search and find the position                                          | 2             |
| 4  | Follow the constraint (No built-in sort + No new array)               | 1             |
| 5  | Maintain Output format (updated array, sorted array, output position) | 1             |

**SET - [B]****SOLUTION**

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] arr = {6, 2, 6, 1, 2, 9, 1};
        int elem = 6;

        int[] newArr = new int[arr.length];
        int count = 0;

        for (int i = 0; i < arr.length; i++) {
            boolean isDuplicate = false;

            for (int j = 0; j < count; j++) {
                if (arr[i] == newArr[j]) {
                    isDuplicate = true;
                    break;
                }
            }

            if (!isDuplicate) {
                newArr[count] = arr[i];
                count++;
            }
        }

        System.out.println("\nNew array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
        }

        for (int i = 0; i < newArr.length - 1; i++) {
            int maxIndex = i;

            for (int j = i + 1; j < newArr.length; j++) {
                if (newArr[j] > newArr[maxIndex]) {
                    maxIndex = j;
                }
            }

            int temp = newArr[i];
            newArr[i] = newArr[maxIndex];
            newArr[maxIndex] = temp;
        }

        int pos = -1;
        System.out.println("\nSorted Array:");
        for (int i = 0; i < count; i++) {
            System.out.print(newArr[i] + " ");
            if (newArr[i] == elem){
                pos = i+1;
            }
        }
        if(pos == -1){
            System.out.printf("\nOutput: \nPosition of %s = Not found", elem);
        }
    }
}

```

```

    }else{
        System.out.printf("\nOutput: \nPosition of %s = %s", elem,pos);
    }
}
}

```

#### RUBRIC

| SL | Points to Meet                                                    | Marks<br>[10] |
|----|-------------------------------------------------------------------|---------------|
| 1  | Create unique elements in new array                               | 3             |
| 2  | Sort the array in descending order                                | 3             |
| 3  | Search and find the position                                      | 2             |
| 4  | Follow the constraint (No built-in sort)                          | 1             |
| 5  | Maintain Output Format (new array, sorted array, output position) | 1             |